FULL-STACK SYSTEM ARCHITECTURE

FORUMSTUDY
SOURCE CODE ENCYCLOPEDIA

A complete, chronologically organized reference containing every single line of code in the project. Features MySQL relational architecture, Express REST API, JWT security, Google Gemini semantic vector embeddings, RAG PDF ingestion, and React Vite frontend.

69	12,266	10
SOURCE FILES BUNDLED	EXACT LINES OF CODE	CHRONOLOGICAL SECTIONS

Theme: Dark Obsidian & Green Neon

100% Preserved Code Integrity (Zero Modifications)

Ready for Offline & Chronological Study

CHRONOLOGICAL TABLE OF CONTENTS

1. DATABASE & ARCHITECTURE BLUEPRINT

DATABASE · SCHEMA	backend/db/schema.sql	135 lines
DATABASE · CONFIG	backend/db/config.js	39 lines
CONFIG · BACKEND	backend/package.json	29 lines
CONFIG · ENV	backend/.env.example	45 lines

2. BACKEND SERVER CORE & MIDDLEWARE

BACKEND · SERVER	backend/index.js	53 lines
BACKEND · ROUTING	backend/src/api/routes.js	19 lines
BACKEND · ERRORS	backend/src/utils/errors/index.js	42 lines
BACKEND · MIDDLEWARE	backend/src/middleware/authentication.js	32 lines
BACKEND · MIDDLEWARE	backend/src/middleware/validation-handler.js	12 lines
BACKEND · MIDDLEWARE	backend/src/middleware/error-handler.js	29 lines

3. BACKEND AUTHENTICATION MODULE

AUTH · ROUTE	backend/src/api/auth/routes/auth.routes.js	29 lines
AUTH · VALIDATION	backend/src/api/auth/validations/auth.validation.js	45 lines
AUTH · CONTROLLER	backend/src/api/auth/controller/auth.controller.js	57 lines
AUTH · SERVICE	backend/src/api/auth/service/auth.service.js	125 lines

4. BACKEND QUESTION & AI VECTOR / COACH MODULE

QUESTION · ROUTE	backend/src/api/question/routes/question.routes.js	75 lines
QUESTION · VALIDATION	backend/src/api/question/validations/question.validation.js	121 lines
QUESTION · CONTROLLER	backend/src/api/question/controller/question.controller.js	149 lines
QUESTION · SERVICE	backend/src/api/question/service/question.service.js	303 lines
QUESTION · VECTOR / AI	backend/src/api/question/service/vector.service.js	460 lines
QUESTION · GEMINI AI	backend/src/api/question/service/geminiTextCoach.service.js	129 lines

💬 5. BACKEND ANSWER MODULE

ANSWER · ROUTE	backend/src/api/answer/routes/answer.routes.js	10 lines
ANSWER · VALIDATION	backend/src/api/answer/validations/answer.validation.js	23 lines
ANSWER · CONTROLLER	backend/src/api/answer/controller/answer.controller.js	23 lines
ANSWER · SERVICE	backend/src/api/answer/service/answer.service.js	62 lines

📄 6. BACKEND RAG (DOCUMENT AI) MODULE

RAG · UPLOAD CONFIG	backend/src/api/rag/config/rag.upload.config.js	55 lines
RAG · ROUTE	backend/src/api/rag/routes/rag.routes.js	73 lines
RAG · VALIDATION	backend/src/api/rag/validations/rag.validation.js	54 lines
RAG · CONTROLLER	backend/src/api/rag/controller/rag.controller.js	144 lines
RAG · SERVICE	backend/src/api/rag/service/rag.service.js	433 lines

💻 7. FRONTEND CORE & CONFIGURATION

CONFIG · FRONTEND	frontend/package.json	42 lines
CONFIG · VITE	frontend/vite.config.js	8 lines
FRONTEND · HTML	frontend/index.html	14 lines
FRONTEND · MAIN	frontend/src/main.jsx	11 lines
FRONTEND · APP / ROUTES	frontend/src/App.jsx	89 lines
FRONTEND · GLOBAL CSS	frontend/src/index.css	229 lines
FRONTEND · UTILS	frontend/src/lib/utils.js	106 lines
FRONTEND · CSS MODULE	frontend/src/styles/pageStates.module.css	70 lines

🌐 8. FRONTEND CONTEXTS & API SERVICES

FRONTEND · CONTEXT	frontend/src/contexts/AuthContext.jsx	99 lines
FRONTEND · API CLIENT	frontend/src/services/core/api.client.js	55 lines
FRONTEND · SERVICE	frontend/src/services/auth/auth.service.js	114 lines
FRONTEND · SERVICE	frontend/src/services/question/question.service.js	210 lines
FRONTEND · SERVICE	frontend/src/services/answer/answer.service.js	44 lines
FRONTEND · SERVICE	frontend/src/services/rag/rag.service.js	240 lines

🛠️ 9. FRONTEND SHARED COMPONENTS

COMPONENT · GUARD	frontend/src/components/ProtectedRoute/ProtectedRoute.jsx	31 lines
COMPONENT · CSS	frontend/src/components/ProtectedRoute/ProtectedRoute.module.css	32 lines
COMPONENT · SHELL	frontend/src/components/Layout/Layout.jsx	90 lines
COMPONENT · CSS	frontend/src/components/Layout/Layout.module.css	121 lines
COMPONENT · NAVBAR	frontend/src/components/Navbar/Navbar.jsx	128 lines
COMPONENT · CSS	frontend/src/components/Navbar/Navbar.module.css	212 lines
COMPONENT · SIDEBAR	frontend/src/components/Sidebar/Sidebar.jsx	125 lines
COMPONENT · CSS	frontend/src/components/Sidebar/Sidebar.module.css	346 lines
COMPONENT · CARD	frontend/src/components/QuestionCard/QuestionCard.jsx	77 lines
COMPONENT · CSS	frontend/src/components/QuestionCard/QuestionCard.module.css	116 lines
COMPONENT · RAG UI	frontend/src/components/RagAnswerBody/RagAnswerBody.jsx	126 lines
COMPONENT · CSS	frontend/src/components/RagAnswerBody/RagAnswerBody.module.css	157 lines

🖥️ 10. FRONTEND PAGES & CSS MODULES

PAGE · LANDING	frontend/src/pages/Landing/Landing.jsx	509 lines
PAGE · CSS	frontend/src/pages/Landing/Landing.module.css	680 lines
PAGE · AUTH	frontend/src/pages/Auth/Auth.jsx	380 lines
PAGE · CSS	frontend/src/pages/Auth/Auth.module.css	538 lines
PAGE · DASHBOARD	frontend/src/pages/Dashboard/Dashboard.jsx	306 lines
PAGE · CSS	frontend/src/pages/Dashboard/Dashboard.module.css	437 lines
PAGE · POST QUESTION	frontend/src/pages/PostQuestion/PostQuestion.jsx	403 lines
PAGE · CSS	frontend/src/pages/PostQuestion/PostQuestion.module.css	417 lines
PAGE · QUESTION DETAIL	frontend/src/pages/QuestionDetail/QuestionDetail.jsx	605 lines
PAGE · CSS	frontend/src/pages/QuestionDetail/QuestionDetail.module.css	663 lines
PAGE · MY QUESTIONS	frontend/src/pages/MyQuestions/MyQuestions.jsx	133 lines
PAGE · CSS	frontend/src/pages/MyQuestions/MyQuestions.module.css	251 lines
PAGE · RAG DOCUMENTS	frontend/src/pages/RagDocuments/RagDocuments.jsx	618 lines
PAGE · CSS	frontend/src/pages/RagDocuments/RagDocuments.module.css	629 lines

SECTION 01

1. DATABASE & ARCHITECTURE BLUEPRINT

Relational MySQL schemas, database connection pool, configuration parameters, and safe SQL execution wrappers.

DATABASE · SCHEMA backend/db/schema.sql

Database Schema (MySQL DDL)

Defines all 7 MySQL tables: users, questions, question_vectors, answers, documents, document_chunks, and document_chunk_vectors.

Lines: 135 Size: 6.3 KB Format: .SQL

```sql
-- Database Schema for Evangadi Forum
-- Platform: MySQL

SET FOREIGN_KEY_CHECKS = 0;

-- ------------------------------------------------------------------------
-- 1. Users Table
-- Stores user account information.
-- ------------------------------------------------------------------------
DROP TABLE IF EXISTS `users`;
CREATE TABLE `users` (
    `user_id` INT AUTO_INCREMENT PRIMARY KEY,
    `first_name` VARCHAR(50) NOT NULL,
    `last_name` VARCHAR(50) NOT NULL,
    `email` VARCHAR(320) NOT NULL UNIQUE,
    `password_hash` VARCHAR(255) NOT NULL,
    `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    `updated_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CHECK (`email` = LOWER(`email`)),

    INDEX `idx_users_email` (`email`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 2. Questions Table
-- Stores the main questions posted by users.
-- Supports full-text search on title and content for exact match search.
-- ------------------------------------------------------------------------
DROP TABLE IF EXISTS `questions`;
CREATE TABLE `questions` (
    `question_id` INT AUTO_INCREMENT PRIMARY KEY,
    `question_hash` CHAR(16) NOT NULL UNIQUE, -- Used for /question/:hash routing
    `user_id` INT NOT NULL,
    `title` VARCHAR(255) NOT NULL,
    `content` TEXT NOT NULL, -- Detailed content including code sections
    `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    `updated_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CHECK (CHAR_LENGTH(`title`) >= 5),
    CHECK (CHAR_LENGTH(`content`) >= 10),

    FOREIGN KEY (`user_id`) REFERENCES `users`(`user_id`) ON DELETE CASCADE,

    INDEX `idx_questions_user_id` (`user_id`),
    INDEX `idx_questions_created_at` (`created_at`),

    -- Full-text search index for exact match search mode
    FULLTEXT KEY `ft_questions_search` (`title`, `content`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 3. Question Vectors Table
-- Stores embeddings for the AI Semantic Search feature (Gemini default model).
-- ------------------------------------------------------------------------
DROP TABLE IF EXISTS `question_vectors`;
CREATE TABLE `question_vectors` (
    `vector_id` BIGINT AUTO_INCREMENT PRIMARY KEY,
    `question_id` INT NOT NULL,
    `source_text` TEXT NOT NULL, -- Text used to generate the embedding
    `embedding` JSON NOT NULL,   -- Gemini embedding vector
    `status` VARCHAR(20) DEFAULT 'ready', -- e.g., 'ready', 'pending', 'failed'
```

```
    `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    `updated_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (`question_id`) REFERENCES `questions`(`question_id`) ON DELETE CASCADE,
    UNIQUE KEY `uniq_question_vectors_question_id` (`question_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 4. Answers Table
-- Stores answers to questions.
-- ------------------------------------------------------------------------
DROP TABLE IF EXISTS `answers`;
CREATE TABLE `answers` (
    `answer_id` INT AUTO_INCREMENT PRIMARY KEY,
    `question_id` INT NOT NULL,
    `user_id` INT NOT NULL,
    `content` TEXT NOT NULL, -- Content including code sections
    `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    `updated_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

    FOREIGN KEY (`question_id`) REFERENCES `questions`(`question_id`) ON DELETE CASCADE,
    FOREIGN KEY (`user_id`) REFERENCES `users`(`user_id`) ON DELETE CASCADE,

    INDEX `idx_answers_question_id` (`question_id`),
    INDEX `idx_answers_user_id` (`user_id`),
    INDEX `idx_answers_created_at` (`created_at`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 5. RAG: user-owned PDF documents, text chunks, and chunk embeddings
-- ------------------------------------------------------------------------
DROP TABLE IF EXISTS `documents`;
CREATE TABLE `documents` (
    `document_id` INT AUTO_INCREMENT PRIMARY KEY,
    `user_id` INT NOT NULL,
    `title` VARCHAR(512) NOT NULL,
    `mime_type` VARCHAR(128) NOT NULL DEFAULT 'application/pdf',
    `storage_path` VARCHAR(1024) NOT NULL,
    `byte_size` BIGINT NOT NULL DEFAULT 0,
    `status` VARCHAR(20) NOT NULL DEFAULT 'pending',
    `error_message` TEXT NULL,
    `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    `updated_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (`user_id`) REFERENCES `users`(`user_id`) ON DELETE CASCADE,
    INDEX `idx_documents_user_created` (`user_id`, `created_at`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

DROP TABLE IF EXISTS `document_chunks`;
CREATE TABLE `document_chunks` (
    `chunk_id` BIGINT AUTO_INCREMENT PRIMARY KEY,
    `document_id` INT NOT NULL,
    `chunk_index` INT NOT NULL,
    `content` TEXT NOT NULL,
    `page_start` INT NULL,
    `page_end` INT NULL,
    `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (`document_id`) REFERENCES `documents`(`document_id`) ON DELETE CASCADE,
    UNIQUE KEY `uniq_document_chunks_doc_index` (`document_id`, `chunk_index`),
    INDEX `idx_document_chunks_document_id` (`document_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

DROP TABLE IF EXISTS `document_chunk_vectors`;
CREATE TABLE `document_chunk_vectors` (
    `chunk_vector_id` BIGINT AUTO_INCREMENT PRIMARY KEY,
    `chunk_id` BIGINT NOT NULL,
    `source_text` TEXT NOT NULL,
    `embedding` JSON NOT NULL,
    `status` VARCHAR(20) NOT NULL DEFAULT 'ready',
    `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    `updated_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (`chunk_id`) REFERENCES `document_chunks`(`chunk_id`) ON DELETE CASCADE,
```

```
    UNIQUE KEY `uniq_chunk_vectors_chunk_id` (`chunk_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

SET FOREIGN_KEY_CHECKS = 1;

```

DATABASE · CONFIG backend/db/config.js

Database Connection & Safe Execution

Creates the MySQL connection pool and provides safeExecute() to prevent SQL injection and connection leaks.

Lines: 39 Size: 1.1 KB Format: .JS

```
import dotenv from 'dotenv/config';
// enviromental variable used to store sensitive information such as database credentials, API keys, and other configu
ration settings

import mysql from 'mysql2/promise';

// Database connection pool
export const db = mysql.createPool({
  host: process.env.DB_HOST || 'localhost',
  user: process.env.DB_USER,
  password: process.env.DB_PASS || process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
});

const ensureParams = params => {
  if (params === undefined || params === null) {
    throw new Error('SQL parameters are required');
  }

  const isArray = Array.isArray(params);true
  const isObject = !isArray && typeof params === 'object';
  if (!isArray && !isObject) {
    throw new Error('SQL parameters must be an array or object');
  }
};




export const safeExecute = async (sql, params) => {
  if (typeof sql !== 'string' || sql.trim().length === 0) {
    throw new Error('SQL query must be a non-empty string');
  }
  ensureParams(params);
  const [result] = await db.execute(sql, params);
  return result;
};


```

CONFIG · BACKEND backend/package.json

Backend Manifest & Dependencies

Lists backend dependencies: express, mysql2, jsonwebtoken, bcryptjs, @google/genai, cors, multer, pdf-parse.

Lines: 29 Size: 0.6 KB Format: .JSON

```
{
  "name": "backend",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "dev": "nodemon index.js",
    "start": "node index.js",
    "test": "node --test"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "module",
  "dependencies": {
    "@google/genai": "^2.7.0",
    "bcryptjs": "^3.0.3",
    "cors": "^2.8.6",
    "dotenv": "^17.3.1",
    "express": "^5.2.1",
    "express-validator": "^7.3.1",
    "http-status-codes": "^2.3.0",
    "jsonwebtoken": "^9.0.3",
    "multer": "^2.1.1",
    "mysql2": "^3.18.2",
    "pdf-parse": "^2.4.5"
  }
}

```

CONFIG · ENV backend/.env.example

Environment Variables Template

Template for database credentials, JWT secret, Gemini API key, and server ports.

Lines: 45 Size: 1.2 KB Format: .EXAMPLE

```
## Server Configuration
PORT=3777

## Database Credentials
DB_USER=your_database_user
DB_PASS=your_database_password
DB_HOST=127.0.0.1
DB_NAME=your_database_name

## JWT Configuration
JWT_SECRET=99999999912333
JWT_EXPIRES_IN=1d



## Gemini API (Used for Vector Embeddings)
GEMINI_API_KEY=your_gemini_api_key_here
GEMINI_EMBEDDING_MODEL=gemini-embedding-001
## Optional: text model for draft coach + answer fit (defaults to gemini-2.5-flash-lite)
GEMINI_TEXT_MODEL=gemini-2.5-flash-lite




## Vector Search Thresholds & Settings
# Threshold for recommending similar questions
RECOMMEND_THRESHOLD=0.75
# Number of similar questions to return
RECOMMEND_K=5

## RAG (user-owned PDFs)
# Where PDFs are stored on disk (default uploads/rag under backend cwd)
RAG_UPLOAD_DIR=uploads/rag
# Max upload size in megabytes (keep small for classroom demos)
RAG_MAX_UPLOAD_MB=5
# Chunking: character window and overlap between windows
RAG_CHUNK_CHARS=900
RAG_CHUNK_OVERLAP=120
# Safety caps per document / per user
RAG_MAX_CHUNKS_PER_DOC=1000
RAG_MAX_PDFS_PER_USER=20
# Minimum extracted characters to treat PDF as usable text
RAG_MIN_TEXT_CHARS=50
# Semantic search: min cosine score and max hits returned
RAG_SEARCH_THRESHOLD=0.45
RAG_SEARCH_K=10

```

SECTION 02

2. BACKEND SERVER CORE & MIDDLEWARE

Server bootstrap, central routing junction, JWT authentication guards, input validation, and global error handling.

BACKEND · SERVER **backend/index.js**

Server Entry Point & Bootstrap

Initializes Express, CORS, JSON parsing, health check route, database connection test, and error handling.

Lines: 53 Size: 1.3 KB Format: .JS

```
import 'dotenv/config';
import express from 'express';
import { db } from './db/config.js';
import { mainRouter } from './src/api/routes.js';
import { errorHandler } from './src/middleware/error-handler.js';
import cors from 'cors';

const app = express();
const port = process.env.PORT || 3777;

// Middleware
app.use(cors());
// for reading json body
app.use(express.json());
// for reading form data and files
app.use(express.urlencoded({ extended: true }));

// Health check
app.get('/health', (req, res) => {
  res.json({ status: 'ok', timestamp: new Date() });
});

app.use('/api', mainRouter);

app.use(errorHandler);

// Start server
const startServer = async () => {
  try {
    // Test database connection
    const connection = await db.getConnection();

    console.log('Database connection established successfully.');
    connection.release();

    app.listen(port, err => {
      if (err) {
        console.error('Failed to start the server:', err.message);
        process.exit(1);
      }
      console.log(`Server running on port http://localhost:${port}`);
    });
  } catch (error) {
    console.error(
      'Failed to connect to the database. Server not started.',
      error.message,
    );
    process.exit(1);
  }
};

startServer();

```

BACKEND · ROUTING **backend/src/api/routes.js**

Central API Highway

Mounts all sub-routers: /auth, /questions, /answers, and /rag/documents under the /api prefix.

Lines: 19 Size: 0.6 KB Format: .JS

```
import express from 'express';
import authRoutes from './auth/routes/auth.routes.js';
import questionRoutes from './question/routes/question.routes.js';
import answerRoutes from './answer/routes/answer.routes.js';
import ragRoutes from './rag/routes/rag.routes.js';

export const mainRouter = express.Router();

// Authentication routes (/api/auth)
mainRouter.use('/auth', authRoutes);

// Question routes (/api/questions)
mainRouter.use('/questions', questionRoutes);

// Answer routes (/api/answers)
mainRouter.use('/answers', answerRoutes);

// RAG routes (/api/rag/documents)
mainRouter.use('/rag/documents', ragRoutes);
```

BACKEND · ERRORS backend/src/utils/errors/index.js

Custom API Error Classes

Defines standard HTTP error classes (BadRequestError, NotFoundError, UnauthenticatedError, ServiceUnavailableError).

Lines: 42 Size: 0.9 KB Format: .JS

```
import { StatusCodes } from 'http-status-codes';

class CustomAPIError extends Error {
  constructor(message) {
    super(message);
  }
}


export class BadRequestError extends CustomAPIError {
  constructor(message) {
    super(message);
    this.statusCode = StatusCodes.BAD_REQUEST; // 400
  }
}




export class NotFoundError extends CustomAPIError {
  constructor(message) {
    super(message);
    this.statusCode = StatusCodes.NOT_FOUND; // 404
  }
}

export class UnauthenticatedError extends CustomAPIError {
  constructor(message) {
    super(message);
    this.statusCode = StatusCodes.UNAUTHORIZED; // 401
  }
}

export class ServiceUnavailableError extends CustomAPIError {
  constructor(message) {
    super(message);
    this.statusCode = StatusCodes.SERVICE_UNAVAILABLE; // 503
  }
}

export default CustomAPIError;

```

BACKEND · MIDDLEWARE `backend/src/middleware/authentication.js`

JWT Authentication Guard

Validates Bearer tokens on incoming requests and attaches authenticated user data to req.user.

Lines: 32 Size: 0.8 KB Format: .JS

```
import 'dotenv/config';
import jwt from 'jsonwebtoken';
import { UnauthenticatedError } from '../utils/errors/index.js';

const JWT_SECRET = process.env.JWT_SECRET;

if (!JWT_SECRET) {
  throw new Error('JWT_SECRET environment variable is required');
}

export const authenticateUser = (req, res, next) => {
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    throw new UnauthenticatedError('Authentication invalid');
  }

  const token = authHeader.split(' ')[1];

  try {
    const payload = jwt.verify(token, JWT_SECRET);
    req.user = {
      id: payload.id,
      firstName: payload.firstName,
      lastName: payload.lastName,
    };
    next();
  } catch (error) {
    throw new UnauthenticatedError('Authentication invalid');
  }
};

```

BACKEND · MIDDLEWARE backend/src/middleware/validation-handler.js

Validation Error Interceptor

Inspects express-validator results and throws formatted BadRequestError on any invalid field.

Lines: 12 Size: 0.4 KB Format: .JS

```
import { validationResult } from 'express-validator';
import { BadRequestError } from '../utils/errors/index.js';

export const validationErrorHandler = (req, res, next) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    const errorMessages = errors.array().map(err => err.msg);
    throw new BadRequestError(errorMessages.join('. '));
  }
  next();
};

```

BACKEND · MIDDLEWARE backend/src/middleware/error-handler.js

Global Error Handler Middleware

Catches all thrown errors, handles duplicate SQL keys, and sends structured JSON responses.

Lines: 29 Size: 0.7 KB Format: .JS

```
import { StatusCodes } from 'http-status-codes';

export const errorHandler = (err, req, res, next) => {
  let customError = {
    statusCode: err.statusCode || StatusCodes.INTERNAL_SERVER_ERROR,//500
    msg: err.message || 'Something went wrong try again later',
  };

  if (err?.code === 'ER_DUP_ENTRY') {
    customError.statusCode = StatusCodes.BAD_REQUEST;
    customError.msg = 'Duplicate value entered for a unique field';
  }

  return res.status(customError.statusCode).json({ msg: customError.msg });
};




// default export



// default export


// named export is used when we want to export multiple functions from a module
// default export is used when we want to export a single function from a module
```



SECTION 03

3. BACKEND AUTHENTICATION MODULE

User registration with bcrypt password hashing, login verification, and JWT issuance.

AUTH · ROUTE backend/src/api/auth/routes/auth.routes.js

Auth Route Endpoints

Exposes POST /api/auth/register and POST /api/auth/login.

Lines: 29 Size: 0.6 KB Format: .JS

```
import express from 'express';
import {
  registerController,
  loginController,
} from '../controller/auth.controller.js';
import {
  registerValidation,
  loginValidation,
} from '../validations/auth.validation.js';

const router = express.Router();

/**
 * @route POST /api/auth/register
 * @desc Register a new user
 * @access Public
 */

router.post('/register', registerValidation, registerController);

/**
 * @route POST /api/auth/login
 * @desc Authenticate user and get token
 * @access Public
 */
router.post('/login', loginValidation, loginController);

export default router;

```

AUTH · VALIDATION backend/src/api/auth/validations/auth.validation.js

Auth Validation Rules

Enforces first/last name lengths, email format, and password min-length.

Lines: 45 Size: 1.3 KB Format: .JS

```
import { body } from 'express-validator';
import { validationErrorHandler } from '../../../middleware/validation-handler.js';

export const registerValidation = [
  body('firstName')
    .notEmpty()
    .withMessage('First name is required ')
    .isString()
    .withMessage('First name must be a string')
    .isLength({ min: 3 })
    .withMessage('First name must be at least 3 characters long'),
  body('lastName')
    .notEmpty()
    .withMessage('Last name is required')
    .isString()
    .withMessage('Last name must be a string')
    .isLength({ min: 3 })
    .withMessage('Last name must be at least 3 characters long'),
  body('email')
    .notEmpty()
    .withMessage('Email is required')
    .isEmail()
    .withMessage('A valid email address is required')
    .normalizeEmail(),
  body('password')
    .notEmpty()
    .withMessage('Password is required')
    .isLength({ min: 6 })
    .withMessage('Password must be at least 6 characters long'),

  validationErrorHandler,
];

export const loginValidation = [
  body('email')
    .notEmpty()
    .withMessage('Email is required')
    .isEmail()
    .withMessage('A valid email address is required')
    .normalizeEmail(),
  body('password').notEmpty().withMessage('Password is required'),

  validationErrorHandler,
];

```

AUTH · CONTROLLER backend/src/api/auth/controller/auth.controller.js

Auth Request Controllers

Extracts request credentials, invokes auth service, and returns token/user payloads.

Lines: 57 Size: 1.6 KB Format: .JS

```
import { StatusCodes } from 'http-status-codes';
import { registerService, loginService } from '../service/auth.service.js';

/**
 * Handles user registration requests.
 *
 * @param {import('express').Request} req - The Express request object.
 * @param {import('express').Response} res - The Express response object.
 * @param {import('express').NextFunction} next - The Express next function.
 * @returns {Promise<void>}
 */
export const registerController = async (req, res, next) => {
  try {
    const { firstName, lastName, email, password } = req.body;

    const newUser = await registerService({
      firstName,
      lastName,
      email,
      password,
    });

    res.status(StatusCodes.CREATED).json({
      success: true,
      message: 'User registered successfully.',
      user: newUser,
    });
  } catch (error) {
    next(error);
  }
};

/**
 * Handles user login requests.
 *
 * @param {import('express').Request} req - The Express request object.
 * @param {import('express').Response} res - The Express response object.
 * @param {import('express').NextFunction} next - The Express next function.
 * @returns {Promise<void>}
 */
export const loginController = async (req, res, next) => {
  try {
    const { email, password } = req.body;

    const authResult = await loginService({ email, password });

    res.status(StatusCodes.OK).json({
      success: true,
      message: 'Login successful.',
      user: authResult.user,
      token: authResult.token,
    });
  } catch (error) {
    next(error);
  }
};

```

AUTH · SERVICE backend/src/api/auth/service/auth.service.js

Auth Business Logic

Performs bcrypt hashing, email duplication checks, password verification, and JWT generation.

Lines: 125 Size: 3.7 KB Format: .JS

```
import bcrypt from 'bcryptjs';
import jwt from 'jsonwebtoken';
import { safeExecute } from '../../../../db/config.js';
import {
  BadRequestError,
  UnauthenticatedError,
} from '../../../utils/errors/index.js';

const JWT_SECRET = process.env.JWT_SECRET;
const JWT_EXPIRES_IN = process.env.JWT_EXPIRES_IN || '1d';

if (!JWT_SECRET) {
  throw new Error('JWT_SECRET environment variable is required');
}

const normalizeEmail = email => email.trim().toLowerCase();

/**
 * Checks if a user exists by email.
 *
 * @param {string} email - The email to check.
 * @returns {Promise<boolean>} True if the user exists, false otherwise.
 */
export const checkUserExists = async email => {
  const normalizedEmail = normalizeEmail(email);
  const sql = 'SELECT user_id FROM users WHERE email = ? ';
  const rows = await safeExecute(sql, [normalizedEmail]);
  return rows.length > 0;
};

/**
 * Registers a new user in the database.
 *
 * @param {Object} userData - The user data.
 * @param {string} userData.firstName - The first name.
 * @param {string} userData.lastName - The last name.
 * @param {string} userData.email - The email address.
 * @param {string} userData.password - The plain text password.
 * @returns {Promise<Object>} The created user object (without password).
 */
export const registerService = async ({
  firstName,
  lastName,
  email,
  password,
}) => {
  const normalizedEmail = normalizeEmail(email);
  const userExists = await checkUserExists(normalizedEmail);
  if (userExists) {
    throw new BadRequestError('User already exists with this email.');
  }

  // every time we call bcrypt.genSalt, it generates a new random salt string.
  const salt = await bcrypt.genSalt(10); // generates a unique random salt each call
  const hashedPassword = await bcrypt.hash(password, salt);
  const sql =
    'INSERT INTO users (first_name, last_name, email, password_hash) VALUES (?, ?, ?, ?)';
  let result;
  try {
    result = await safeExecute(sql, [
      firstName,
```

```
      lastName,
      normalizedEmail,
      hashedPassword,
    ]);
  } catch (error) {
    if (error?.code === 'ER_DUP_ENTRY') {
      throw new BadRequestError('User already exists with this email.');
    }
    throw error;
  }

  return {
    id: result.insertId,
    firstName,
    lastName,
    email: normalizedEmail,
  };
};

/**
 * Authenticates a user and generates a JWT token.
 *
 * @param {Object} credentials - The login credentials.
 * @param {string} credentials.email - The user's email.
 * @param {string} credentials.password - The user's plain text password.
 * @returns {Promise<Object>} An object containing the user and token.
 * @throws {UnauthenticatedError} If authentication fails.
 */
export const loginService = async ({ email, password }) => {
  const normalizedEmail = normalizeEmail(email);
  const sql =
    'SELECT user_id, first_name, last_name, email, password_hash FROM users WHERE email = ? LIMIT 1';
  const rows = await safeExecute(sql, [normalizedEmail]);

  if (rows.length === 0) {
    throw new UnauthenticatedError('Invalid email or password');
  }

  const user = rows[0];
  const isMatch = await bcrypt.compare(password, user.password_hash);

  if (!isMatch) {
    throw new UnauthenticatedError('Invalid email or password');
  }

  const payload = {
    id: user.user_id,
    firstName: user.first_name,
    lastName: user.last_name,
  };

  const token = jwt.sign(payload, JWT_SECRET, { expiresIn: JWT_EXPIRES_IN });

  return {
    user: {
      id: user.user_id,
      firstName: user.first_name,
      lastName: user.last_name,
      email: user.email,
    },
    token,
  };
};

```



SECTION 04

4. BACKEND QUESTION & AI VECTOR / COACH MODULE

Question posting, semantic search, AI draft coaching with Gemini, and vector cosine similarity matching.

QUESTION · ROUTE backend/src/api/question/routes/question.routes.js

Question API Routes

Defines endpoints for listing, creating, single view, semantic search, draft coach, and answer-fit.

Lines: 75 Size: 1.6 KB Format: .JS

```
import express from 'express';
import { authenticateUser } from '../../../middleware/authentication.js';
import {
  createQuestionValidation,
  getQuestionsValidation,
  getSingleQuestionsValidation,
  searchQuestionsSemanticValidation,
  generateQuestionDraftCoachValidation,
  assessAnswerAgainstQuestionValidation,
  getSimilarQuestionsValidation,
} from '../validations/question.validation.js';
import {
  createQuestionController,
  getQuestionController,
  getSingleQuestionController,
  searchQuestionsSemanticController,
  getSimilarQuestionsController,
  generateQuestionDraftCoachController,
  assessAnswerAgainstQuestionController,
} from '../controller/question.controller.js';

const router = express.Router();

// Specific routes before param routes
router.post(
  '/',
  authenticateUser,
  createQuestionValidation,
  createQuestionController,
);

router.get(
  '/',
  authenticateUser,
  getQuestionsValidation,
  getQuestionController,
);

router.get(
  '/search',
  authenticateUser,
  searchQuestionsSemanticValidation,
  searchQuestionsSemanticController,
);

router.post(
  '/draft-coach',
  authenticateUser,
  generateQuestionDraftCoachValidation,
  generateQuestionDraftCoachController,
);

// Dynamic routes by questionHash
router.get(
  '/:questionHash/similar',
  authenticateUser,
  getSimilarQuestionsValidation,
  getSimilarQuestionsController,
);

router.post(
```

```
  '/:questionHash/answer-fit',
  authenticateUser,
  assessAnswerAgainstQuestionValidation,
  assessAnswerAgainstQuestionController,
);

router.get(
  '/:questionHash',
  authenticateUser,
  getSingleQuestionsValidation,
  getSingleQuestionController,
);

export default router;
```

QUESTION · VALIDATION backend/src/api/question/validations/question.validation.js

Question Validation Schemas

Validates question titles, contents, questionHash formats, and search query parameters.

Lines: 121 Size: 3.2 KB Format: .JS

```
import { body, param, query } from 'express-validator';
import { validationErrorHandler } from '../../../middleware/validation-handler.js';

const HASH_REGEX = /^[a-f0-9]{16}$/;

export const createQuestionValidation = [
  body('title')
    .trim()
    .notEmpty()
    .withMessage('Question title is required')
    .isLength({ min: 5, max: 255 })
    .withMessage('Title must be between 5 and 255 characters long'),
  body('content')
    .notEmpty()
    .withMessage('Content is required')
    .isString()
    .withMessage('Content must be a string')
    .trim()
    .isLength({ min: 10 })
    .withMessage('Content must be at least 10 characters long'),
  validationErrorHandler,
];

export const getQuestionsValidation = [
  query('search')
    .optional()
    .isString()
    .withMessage('Search must be a string')
    .trim(),
  query('mine')
    .optional()
    .isBoolean()
    .withMessage('Mine must be a boolean')
    .toBoolean(),
  validationErrorHandler,
];

export const getSingleQuestionsValidation = [
  param('questionHash')
    .isString()
    .withMessage('Question hash must be a string')
    .matches(HASH_REGEX)
    .withMessage('Invalid question hash format'),
  validationErrorHandler,
];

export const searchQuestionsSemanticValidation = [
  query('query')
    .notEmpty()
    .withMessage('Query is required')
    .isString()
    .withMessage('Query must be a string')
    .trim()
    .isLength({ min: 5 })
    .withMessage('Query must be at least 5 characters long'),
  query('k')
    .optional()
    .isInt({ min: 1, max: 20 })
    .withMessage('k must be between 1 and 20')
    .toInt(),
  query('threshold')
```

```
    .optional()
    .isFloat({ min: 0, max: 1 })
    .withMessage('Threshold must be between 0 and 1')
    .toFloat(),
  validationErrorHandler,
];

export const assessAnswerAgainstQuestionValidation = [
  param('questionHash')
    .isString()
    .withMessage('Question hash must be a string')
    .matches(HASH_REGEX)
    .withMessage('Invalid question hash format'),
  body('answerText')
    .notEmpty()
    .withMessage('Answer text is required')
    .isString()
    .withMessage('Answer text must be a string')
    .trim()
    .isLength({ min: 20 })
    .withMessage('Answer text must be at least 20 characters long'),
  validationErrorHandler,
];

export const generateQuestionDraftCoachValidation = [
  body('title')
    .optional()
    .isString()
    .withMessage('Question title must be a string')
    .trim(),
  body('content')
    .notEmpty()
    .withMessage('Question content is required')
    .isString()
    .withMessage('Question content must be a string')
    .trim()
    .isLength({ min: 10 })
    .withMessage('Question content must be at least 10 characters long'),
  validationErrorHandler,
];

export const getSimilarQuestionsValidation = [
  param('questionHash')
    .isString()
    .withMessage('Question hash must be a string')
    .matches(HASH_REGEX)
    .withMessage('Invalid question hash format'),
  query('k')
    .optional()
    .isInt({ min: 1, max: 20 })
    .withMessage('k must be between 1 and 20')
    .toInt(),
  query('threshold')
    .optional()
    .isFloat({ min: 0, max: 1 })
    .withMessage('Threshold must be between 0 and 1')
    .toFloat(),
  validationErrorHandler,
];
```

QUESTION · CONTROLLER backend/src/api/question/controller/question.controller.js

Question Controllers

Orchestrates question endpoints, filters, semantic similarity, and AI draft coaching calls.

Lines: 149 Size: 3.6 KB Format: .JS

```
import { StatusCodes } from 'http-status-codes';
import {
  createQuestionWithVectorService,
  getQuestionService,
  getSingleQuestionService,
  searchQuestionsSemanticService,
  getSimilarQuestionsService,
} from '../service/question.service.js';
import {
  generateQuestionDraftCoachService,
  assessAnswerAgainstQuestionService,
} from '../service/geminiTextCoach.service.js';

export const createQuestionController = async (req, res, next) => {
  try {
    const { title, content } = req.body;
    const result = await createQuestionWithVectorService({
      userId: req.user.id,
      title,
      content,
    });

    res.status(StatusCodes.CREATED).json({
      success: true,
      message: 'Question posted successfully.',
      data: result.question,
    });
  } catch (error) {
    next(error);
  }
};

export const getQuestionController = async (req, res, next) => {
  try {
    const filters = {
      search: req.query.search,
      mine: req.query.mine,
      userId: req.user.id,
    };
    const result = await getQuestionService(filters);

    res.status(StatusCodes.OK).json({
      success: true,
      message: 'Questions fetched successfully.',
      ...result,
    });
  } catch (error) {
    next(error);
  }
};

export const getSingleQuestionController = async (req, res, next) => {
  try {
    const { questionHash } = req.params;
    const result = await getSingleQuestionService({ questionHash });

    res.status(StatusCodes.OK).json({
      success: true,
      message: 'Question fetched successfully',
      ...result,
    });
```

```
  } catch (error) {
    next(error);
  }
};

export const searchQuestionsSemanticController = async (req, res, next) => {
  try {
    const result = await searchQuestionsSemanticService({
      query: req.query.query,
      k: req.query.k ? Number(req.query.k) : 5,
      threshold:
        req.query.threshold !== undefined ? Number(req.query.threshold) : undefined,
    });

    res.status(StatusCodes.OK).json({
      success: true,
      message: 'Semantic search completed successfully',
      ...result,
    });
  } catch (error) {
    next(error);
  }
};

export const getSimilarQuestionsController = async (req, res, next) => {
  try {
    const { questionHash } = req.params;
    const result = await getSimilarQuestionsService({
      questionHash,
      k: req.query.k ? Number(req.query.k) : 5,
      threshold:
        req.query.threshold !== undefined ? Number(req.query.threshold) : undefined,
    });

    res.status(StatusCodes.OK).json({
      success: true,
      message: 'Similar questions fetched successfully',
      ...result,
    });
  } catch (error) {
    next(error);
  }
};

export const generateQuestionDraftCoachController = async (req, res, next) => {
  try {
    const { title, content } = req.body;
    const data = await generateQuestionDraftCoachService({
      title,
      content,
    });

    res.status(StatusCodes.OK).json({
      success: true,
      message: 'Draft suggestions generated',
      data,
    });
  } catch (error) {
    next(error);
  }
};

export const assessAnswerAgainstQuestionController = async (req, res, next) => {
  try {
    const { questionHash } = req.params;
    const { answerText } = req.body;

    const { question } = await getSingleQuestionService({
      questionHash,
      includeAnswers: false,
```

```
    });

    const data = await assessAnswerAgainstQuestionService({
      questionTitle: question.title,
      questionContent: question.content,
      answerText,
    });

    res.status(StatusCodes.OK).json({
      success: true,
      message: 'Answer fit assessed',
      data,
    });
  } catch (error) {
    next(error);
  }
};

```

QUESTION · SERVICE backend/src/api/question/service/question.service.js

Question Core Service

Handles MySQL storage of questions with unique hashes, joins with authors and answer counts.

Lines: 303 Size: 7.5 KB Format: .JS

```
import crypto from 'crypto';
import { BadRequestError, NotFoundError } from '../../../utils/errors/index.js';
import { safeExecute } from '../../../../db/config.js';
import {
  findSimilarQuestionByText,
  findSimilarQuestionsByQuestionHash,
  generateQuestionEmbedding,
  getVectorConfig,
  normalizeQuestionText,
  startQuestionVector,
} from './vector.service.js';

const generateQuestionHash = () => crypto.randomBytes(8).toString('hex');

export const createQuestionWithVectorService = async payload => {
  const { userId, title, content } = payload;
  const insertQuestionSql = `
    INSERT INTO questions (question_hash, user_id, title, content)
    VALUES (?, ?, ?, ?)
  `;
  const questionHash = generateQuestionHash();

  let questionResult;
  try {
    questionResult = await safeExecute(insertQuestionSql, [
      questionHash,
      userId,
      title,
      content,
    ]);
  } catch (error) {
    if (error?.code === 'ER_NO_REFERENCED_ROW_2') {
      throw new BadRequestError('User does not exist');
    }
    throw error;
  }

  const questionId = questionResult.insertId;
  const creationResult = {
    id: questionId,
    questionHash,
    title,
    content,
    userId,
  };

  const sourceText = normalizeQuestionText({ title });

  try {
    const embeddingResult = await generateQuestionEmbedding(sourceText, {
      taskType: 'RETRIEVAL_DOCUMENT',
      questionId,
    });

    if (embeddingResult?.embedding && embeddingResult.embedding.length > 0) {
      await startQuestionVector({
        questionId,
        sourceText,
        embedding: embeddingResult.embedding,
        status: 'ready',
      });
```

```
    } else {
      await startQuestionVector({
        questionId,
        sourceText,
        embedding: [],
        status: 'failed',
      });
    }
  } catch (error) {
    console.error('=== FAILED TO STORE VECTOR FOR QUESTION ===');
    console.error('Question ID:', questionId);
    console.error('Error:', error.message);
    console.error('==========================================');

    await startQuestionVector({
      questionId,
      sourceText,
      embedding: [],
      status: 'failed',
    }).catch(e => console.error('Failed to save failed status:', e.message));
  }

  return {
    question: creationResult,
  };
};

const buildQuestionFilters = filters => {
  const conditions = [];
  const params = [];

  if (filters.search) {
    conditions.push('(q.title LIKE ? OR q.content LIKE ?)');
    const searchTerm = `%${filters.search}%`;
    params.push(searchTerm, searchTerm);
  }

  if (filters.mine && filters.userId) {
    conditions.push('q.user_id = ?');
    params.push(filters.userId);
  }

  if (conditions.length === 0) {
    return { whereClause: '', params };
  }

  return {
    whereClause: `WHERE ${conditions.join(' AND ')}`,
    params,
  };
};

export const getQuestionService = async (filters = {}) => {
  const normalizedLimit = 100;
  const sortColumn = 'q.created_at';
  const normalizedSortOrder = 'DESC';
  const { whereClause, params } = buildQuestionFilters(filters);

  const listSql = `
    SELECT
      q.question_id AS id,
      q.question_hash AS questionHash,
      q.title,
      q.content,
      q.created_at AS createdAt,
      q.updated_at AS updatedAt,
      u.user_id AS userId,
      u.first_name AS firstName,
      u.last_name AS lastName,
      COUNT(DISTINCT a.answer_id) AS answerCount
```

```
    FROM questions q
    JOIN users u ON u.user_id = q.user_id
    LEFT JOIN answers a ON a.question_id = q.question_id
    ${whereClause}
    GROUP BY q.question_id, u.user_id
    ORDER BY ${sortColumn} ${normalizedSortOrder}
    LIMIT ${normalizedLimit}
  `;

  const rows = await safeExecute(listSql, params);

  return {
    data: rows.map(question => ({
      id: question.id,
      questionHash: question.questionHash,
      title: question.title,
      content: question.content,
      answerCount: question.answerCount,
      createdAt: question.createdAt,
      updatedAt: question.updatedAt,
      author: {
        id: question.userId,
        firstName: question.firstName,
        lastName: question.lastName,
      },
    })),
    meta: {
      limit: normalizedLimit,
      total: rows.length,
      sortBy: 'newest',
      sortOrder: 'desc',
    },
  };
};

export const getSingleQuestionService = async ({ questionHash, includeAnswers = true }) => {
  const normalizedAnswerLimit = 100;
  const questionSql = `
    SELECT
      q.question_id AS id,
      q.question_hash AS questionHash,
      q.title,
      q.content,
      q.created_at AS createdAt,
      q.updated_at AS updatedAt,
      u.user_id AS userId,
      u.first_name AS firstName,
      u.last_name AS lastName,
      COUNT(DISTINCT a.answer_id) AS answerCount
    FROM questions q
    JOIN users u ON u.user_id = q.user_id
    LEFT JOIN answers a ON a.question_id = q.question_id
    WHERE q.question_hash = ?
    GROUP BY q.question_id, u.user_id
    LIMIT 1
  `;

  const questionRows = await safeExecute(questionSql, [questionHash]);
  if (!questionRows.length) {
    throw new NotFoundError('Question not found');
  }

  const question = questionRows[0];
  let answers = [];

  if (includeAnswers) {
    const answerSql = `
      SELECT
        a.answer_id AS id,
        a.question_id AS questionId,
```

```
        a.content,
        a.created_at AS createdAt,
        a.updated_at AS updatedAt,
        u.user_id AS userId,
        u.first_name AS firstName,
        u.last_name AS lastName
      FROM answers a
      JOIN users u ON u.user_id = a.user_id
      WHERE a.question_id = ?
      ORDER BY a.created_at ASC
      LIMIT ${normalizedAnswerLimit}
    `;
    answers = await safeExecute(answerSql, [question.id]);
  }

  return {
    question: {
      id: question.id,
      questionHash: question.questionHash,
      title: question.title,
      content: question.content,
      answerCount: question.answerCount,
      createdAt: question.createdAt,
      updatedAt: question.updatedAt,
      author: {
        id: question.userId,
        firstName: question.firstName,
        lastName: question.lastName,
      },
    },
    answers: answers.map(answer => ({
      id: answer.id,
      content: answer.content,
      createdAt: answer.createdAt,
      updatedAt: answer.updatedAt,
      author: {
        id: answer.userId,
        firstName: answer.firstName,
        lastName: answer.lastName,
      },
    })),
    answersMeta: {
      limit: normalizedAnswerLimit,
      total: answers.length,
    },
  };
};

export const searchQuestionsSemanticService = async ({
  query,
  k = 5,
  threshold,
}) => {
  const sourceText = normalizeQuestionText({ title: query });
  const vectorConfig = getVectorConfig();
  const searchThreshold =
    threshold !== undefined ? threshold : vectorConfig.recommendThreshold;

  const result = await findSimilarQuestionByText({
    sourceText,
    threshold: searchThreshold,
    k,
  });

  return {
    data: result.similarQuestions,
    meta: {
      query,
      k,
      threshold: searchThreshold,
```

```
      total: result.similarQuestions.length,
      questionHash: null,
    },
  };
};

export const getSimilarQuestionsService = async ({
  questionHash,
  k = 5,
  threshold,
}) => {
  const vectorConfig = getVectorConfig();
  const searchThreshold =
    threshold !== undefined ? threshold : vectorConfig.recommendThreshold;

  const result = await findSimilarQuestionsByQuestionHash({
    questionHash,
    threshold: searchThreshold,
    k,
  });

  return {
    data: result.similarQuestions,
    meta: {
      total: result.similarQuestions.length,
      k,
      threshold: searchThreshold,
      query: null,
      questionHash,
    },
  };
};
```

QUESTION · VECTOR / AI backend/src/api/question/service/vector.service.js

AI Vector & Cosine Similarity Service

Calls Gemini text embedding model (1536 dims), performs cosine similarity ranking with SQL keyword fallback.

Lines: 460 Size: 13.1 KB Format: .JS

```
import 'dotenv/config';
import { GoogleGenAI } from '@google/genai';
import { safeExecute } from '../../../../db/config.js';
import { ServiceUnavailableError, NotFoundError } from '../../../utils/errors/index.js';

const RECOMMEND_K = parseInt(process.env.RECOMMEND_K || '5', 10);
const RECOMMEND_THRESHOLD = parseFloat(process.env.RECOMMEND_THRESHOLD || '0.75');
const GEMINI_EMBEDDING_MODEL = process.env.GEMINI_EMBEDDING_MODEL || 'gemini-embedding-001';
const GEMINI_API_KEY = process.env.GEMINI_API_KEY;

let genAI = null;
if (GEMINI_API_KEY && GEMINI_API_KEY !== 'your_gemini_api_key_here') {
  try {
    genAI = new GoogleGenAI({ apiKey: GEMINI_API_KEY });
  } catch (err) {
    console.warn('Failed to initialize GoogleGenAI for vector service:', err.message);
  }
}

function normalizeWhiteSpaces(value) {
  return value.replace(/\s+/g, ' ').trim();
}

export function normalizeQuestionText({ title }) {
  return normalizeWhiteSpaces(`${title || ''}`.normalize('NFKC').toLowerCase());
}

export async function generateQuestionEmbedding(sourceText, options = {}) {
  const { taskType = 'RETRIEVAL_DOCUMENT' } = options;

  if (!genAI) {
    throw new ServiceUnavailableError('Gemini API key is not configured for embeddings');
  }

  try {
    const response = await genAI.models.embedContent({
      model: GEMINI_EMBEDDING_MODEL,
      contents: sourceText,
      config: {
        taskType: taskType,
        outputDimensionality: 1536,
      },
    });

    const values = response.embedding?.values || response.embeddings?.[0]?.values;

    if (!Array.isArray(values) || values.length === 0) {
      throw new Error('Gemini embedding response does not contain values');
    }

    return { embedding: values };
  } catch (error) {
    console.error('Error generating embedding:', error.message);
    throw error;
  }
}

function validateEmbedding(embedding) {
  if (!Array.isArray(embedding)) {
    throw new Error('Embedding must be an array');
  }
```

```
  if (embedding.length === 0) {
    throw new Error('Embedding cannot be empty');
  }
  if (!embedding.every(v => typeof v === 'number' && !isNaN(v))) {
    throw new Error('Embedding must contain only numbers');
  }
  return true;
}

export async function startQuestionVector({
  questionId,
  sourceText,
  embedding,
  status = 'ready',
}) {
  if (status === 'failed' || !embedding || embedding.length === 0) {
    const sql = `
      INSERT INTO question_vectors (question_id, source_text, embedding, status)
      VALUES (?, ?, ?, ?)
      ON DUPLICATE KEY UPDATE
        source_text = VALUES(source_text),
        embedding = VALUES(embedding),
        status = VALUES(status),
        updated_at = CURRENT_TIMESTAMP
    `;
    await safeExecute(sql, [questionId, sourceText, JSON.stringify([]), 'failed']);
    return;
  }

  validateEmbedding(embedding);
  const embeddingJson = JSON.stringify(embedding);

  const sql = `
    INSERT INTO question_vectors (question_id, source_text, embedding, status)
    VALUES (?, ?, ?, ?)
    ON DUPLICATE KEY UPDATE
      source_text = VALUES(source_text),
      embedding = VALUES(embedding),
      status = VALUES(status),
      updated_at = CURRENT_TIMESTAMP
  `;

  await safeExecute(sql, [questionId, sourceText, embeddingJson, status]);
}

// Alias for consistency
export const storeQuestionVector = startQuestionVector;

async function retrieveReadyEmbeddings(excludeQuestionId = null) {
  let sql = `SELECT question_id, embedding FROM question_vectors WHERE status = ?`;
  const params = ['ready'];
  if (excludeQuestionId) {
    sql += ` AND question_id != ?`;
    params.push(excludeQuestionId);
  }

  const rows = await safeExecute(sql, params);
  const embeddings = [];

  for (const row of rows) {
    try {
      const embedding =
        typeof row.embedding === 'string' ? JSON.parse(row.embedding) : row.embedding;
      if (Array.isArray(embedding) && embedding.length > 0) {
        embeddings.push({
          questionId: row.question_id,
          embedding: embedding,
        });
      }
    } catch (parseError) {
```

```
      console.warn(`Skipping question ${row.question_id}: invalid embedding JSON`);
    }
  }
  return embeddings;
}

export function getVectorConfig() {
  return {
    recommendThreshold: RECOMMEND_THRESHOLD,
    recommendK: RECOMMEND_K,
  };
}

function calculateCosineSimilarity(vecA, vecB) {
  if (!vecA || !vecB || vecA.length === 0 || vecB.length === 0) return 0;
  const len = Math.min(vecA.length, vecB.length);
  let dotProduct = 0;
  let normA = 0;
  let normB = 0;

  for (let i = 0; i < len; i++) {
    dotProduct += vecA[i] * vecB[i];
    normA += vecA[i] * vecA[i];
    normB += vecB[i] * vecB[i];
  }

  const magnitude = Math.sqrt(normA) * Math.sqrt(normB);
  return magnitude === 0 ? 0 : dotProduct / magnitude;
}

async function hydrateQuestionsWithDetails(topResults) {
  if (!topResults || topResults.length === 0) {
    return [];
  }

  const questionIds = topResults.map(r => r.questionId);
  const placeholders = questionIds.map(() => '?').join(',');

  const sql = `
    SELECT
      q.question_id AS questionId,
      q.question_hash AS questionHash,
      q.title,
      q.content,
      q.created_at AS createdAt,
      q.updated_at AS updatedAt,
      u.user_id AS userId,
      u.first_name AS firstName,
      u.last_name AS lastName,
      COUNT(DISTINCT a.answer_id) AS answerCount
    FROM questions q
    JOIN users u ON u.user_id = q.user_id
    LEFT JOIN answers a ON a.question_id = q.question_id
    WHERE q.question_id IN (${placeholders})
    GROUP BY q.question_id, u.user_id
  `;

  const rows = await safeExecute(sql, questionIds);

  const questionMap = {};
  rows.forEach(row => {
    questionMap[String(row.questionId)] = {
      id: row.questionId,
      questionHash: row.questionHash,
      title: row.title,
      content: row.content,
      answerCount: row.answerCount,
      createdAt: row.createdAt,
      updatedAt: row.updatedAt,
      author: {
```

```
        id: row.userId,
        firstName: row.firstName,
        lastName: row.lastName,
      },
    };
  });

  return topResults
    .filter(result => questionMap[String(result.questionId)])
    .map(result => ({
      score: Number(result.score.toFixed(6)),
      ...questionMap[String(result.questionId)],
    }));
}

async function fallbackKeywordSearch(sourceText, normalizedK, excludeQuestionId = null) {
  const words = sourceText
    .split(/\s+/)
    .map(w => w.replace(/[^a-zA-Z0-9]/g, '').trim())
    .filter(w => w.length >= 3)
    .slice(0, 5);

  if (words.length === 0) {
    // If no specific words >= 3 chars, do a broad match
    let broadSql = `
      SELECT
        q.question_id AS questionId,
        q.question_hash AS questionHash,
        q.title,
        q.content,
        q.created_at AS createdAt,
        q.updated_at AS updatedAt,
        u.user_id AS userId,
        u.first_name AS firstName,
        u.last_name AS lastName,
        COUNT(DISTINCT a.answer_id) AS answerCount
      FROM questions q
      JOIN users u ON u.user_id = q.user_id
      LEFT JOIN answers a ON a.question_id = q.question_id
    `;
    const params = [];
    if (excludeQuestionId) {
      broadSql += ` WHERE q.question_id != ? `;
      params.push(excludeQuestionId);
    }
    broadSql += `
      GROUP BY q.question_id, u.user_id
      ORDER BY q.created_at DESC
      LIMIT ${normalizedK}
    `;
    const broadRows = await safeExecute(broadSql, params);
    return broadRows.map((row, idx) => ({
      score: Number((0.80 - idx * 0.05).toFixed(4)),
      id: row.questionId,
      questionHash: row.questionHash,
      title: row.title,
      content: row.content,
      answerCount: row.answerCount,
      createdAt: row.createdAt,
      updatedAt: row.updatedAt,
      author: {
        id: row.userId,
        firstName: row.firstName,
        lastName: row.lastName,
      },
    }));
  }

  const conditions = words.map(() => '(q.title LIKE ? OR q.content LIKE ?)').join(' OR ');
  const params = [];
```

```
  words.forEach(w => {
    params.push(`%${w}%`, `%${w}%`);
  });

  let sql = `
    SELECT
      q.question_id AS questionId,
      q.question_hash AS questionHash,
      q.title,
      q.content,
      q.created_at AS createdAt,
      q.updated_at AS updatedAt,
      u.user_id AS userId,
      u.first_name AS firstName,
      u.last_name AS lastName,
      COUNT(DISTINCT a.answer_id) AS answerCount
    FROM questions q
    JOIN users u ON u.user_id = q.user_id
    LEFT JOIN answers a ON a.question_id = q.question_id
    WHERE (${conditions})
  `;

  if (excludeQuestionId) {
    sql += ` AND q.question_id != ? `;
    params.push(excludeQuestionId);
  }

  sql += `
    GROUP BY q.question_id, u.user_id
    ORDER BY q.created_at DESC
    LIMIT ${normalizedK}
  `;

  const rows = await safeExecute(sql, params);

  return rows.map((row, idx) => ({
    score: Number((0.85 - idx * 0.04).toFixed(4)),
    id: row.questionId,
    questionHash: row.questionHash,
    title: row.title,
    content: row.content,
    answerCount: row.answerCount,
    createdAt: row.createdAt,
    updatedAt: row.updatedAt,
    author: {
      id: row.userId,
      firstName: row.firstName,
      lastName: row.lastName,
    },
  }));
}

export async function findSimilarQuestionByText({ sourceText, threshold, k }) {
  const normalizedK = k > 0 ? Math.min(k, 20) : RECOMMEND_K;
  const normalizedThreshold =
    threshold !== undefined && threshold >= 0 && threshold <= 1
      ? threshold
      : RECOMMEND_THRESHOLD;

  let embeddingResult = null;
  try {
    embeddingResult = await generateQuestionEmbedding(sourceText, {
      taskType: 'RETRIEVAL_QUERY',
    });
  } catch (error) {
    console.warn('Embedding generation unavailable for query, using keyword fallback:', error.message);
  }

  let similarQuestions = [];

```

```
  if (embeddingResult?.embedding) {
    const queryEmbedding = embeddingResult.embedding;
    const storedEmbeddings = await retrieveReadyEmbeddings();

    const similarities = [];
    for (const stored of storedEmbeddings) {
      const score = calculateCosineSimilarity(queryEmbedding, stored.embedding);
      if (score >= normalizedThreshold) {
        similarities.push({
          questionId: stored.questionId,
          score,
        });
      }
    }

    similarities.sort((a, b) => b.score - a.score);
    const topResults = similarities.slice(0, normalizedK);
    similarQuestions = await hydrateQuestionsWithDetails(topResults);
  }

  // Fallback to text/keyword matching if no vector results or embedding unavailable
  if (similarQuestions.length === 0) {
    similarQuestions = await fallbackKeywordSearch(sourceText, normalizedK);
  }

  return {
    ...embeddingResult,
    similarQuestions,
  };
}

export async function findSimilarQuestionsByQuestionHash({ questionHash, threshold, k }) {
  const normalizedK = k > 0 ? Math.min(k, 20) : RECOMMEND_K;
  const normalizedThreshold =
    threshold !== undefined && threshold >= 0 && threshold <= 1
      ? threshold
      : RECOMMEND_THRESHOLD;

  const questionRows = await safeExecute(
    `SELECT question_id, title FROM questions WHERE question_hash = ? LIMIT 1`,
    [questionHash],
  );

  if (!questionRows.length) {
    throw new NotFoundError('Question not found');
  }

  const sourceQuestionId = questionRows[0].question_id;
  const vectorRows = await safeExecute(
    `SELECT embedding FROM question_vectors WHERE question_id = ? AND status = 'ready' LIMIT 1`,
    [sourceQuestionId],
  );

  let sourceEmbedding = null;
  if (vectorRows.length > 0) {
    try {
      sourceEmbedding =
        typeof vectorRows[0].embedding === 'string'
          ? JSON.parse(vectorRows[0].embedding)
          : vectorRows[0].embedding;
    } catch (e) {
      sourceEmbedding = null;
    }
  }

  if (!Array.isArray(sourceEmbedding) || sourceEmbedding.length === 0) {
    // Try to generate on the fly
    try {
      const sourceText = normalizeQuestionText({ title: questionRows[0].title });
      const embRes = await generateQuestionEmbedding(sourceText, {
```

```
        taskType: 'RETRIEVAL_DOCUMENT',
        questionId: sourceQuestionId,
      });
      sourceEmbedding = embRes.embedding;
      await startQuestionVector({
        questionId: sourceQuestionId,
        sourceText,
        embedding: sourceEmbedding,
        status: 'ready',
      });
    } catch (err) {
      console.warn('Could not generate embedding for source question:', err.message);
    }
  }

  let similarQuestions = [];

  if (Array.isArray(sourceEmbedding) && sourceEmbedding.length > 0) {
    const storedEmbeddings = await retrieveReadyEmbeddings(sourceQuestionId);

    const similarities = [];
    for (const stored of storedEmbeddings) {
      const score = calculateCosineSimilarity(sourceEmbedding, stored.embedding);
      if (score >= normalizedThreshold) {
        similarities.push({
          questionId: stored.questionId,
          score,
        });
      }
    }

    similarities.sort((a, b) => b.score - a.score);
    const topResults = similarities.slice(0, normalizedK);
    similarQuestions = await hydrateQuestionsWithDetails(topResults);
  }

  // Fallback to keyword search if no vectors available
  if (similarQuestions.length === 0) {
    similarQuestions = await fallbackKeywordSearch(
      questionRows[0].title,
      normalizedK,
      sourceQuestionId,
    );
  }

  return {
    similarQuestions,
  };
}
```

QUESTION · GEMINI AI backend/src/api/question/service/geminiTextCoach.service.js

Gemini Text Coach & Answer Fit Service

Calls Gemini Flash model to give constructive drafting tips and evaluates answer relevance against questions.

Lines: 129 Size: 4.4 KB Format: .JS

```
import 'dotenv/config';
import { GoogleGenAI } from '@google/genai';
import { ServiceUnavailableError } from '../../../utils/errors/index.js';

const GOOGLE_API_KEY = process.env.GEMINI_API_KEY;
const GEMINI_TEXT_MODEL = process.env.GEMINI_TEXT_MODEL || 'gemini-1.5-flash-latest';

let genAI = null;
if (GOOGLE_API_KEY && GOOGLE_API_KEY !== 'your_gemini_api_key_here') {
  try {
    genAI = new GoogleGenAI({ apiKey: GOOGLE_API_KEY });
  } catch (err) {
    console.warn('Failed to initialize GoogleGenAI for text coach:', err.message);
  }
}

function parseJsonObjectFromGeminiText(raw) {
  if (!raw || typeof raw !== 'string') return null;
  let t = raw.trim();
  if (t.startsWith('```')) {
    t = t.replace(/^```(?:json)?\s*/i, '').replace(/```\s*$/, '');
  }

  try {
    const v = JSON.parse(t);
    return v && typeof v === 'object' && !Array.isArray(v) ? v : null;
  } catch {
    return null;
  }
}

async function fetchGeminiJsonTextResponse(userPrompt) {
  if (!genAI) {
    return null;
  }

  const response = await genAI.models.generateContent({
    model: GEMINI_TEXT_MODEL,
    contents: userPrompt,
    config: {
      maxOutputTokens: 300,
    },
  });

  const text = response?.text;
  return typeof text === 'string' ? text : '';
}

export const generateQuestionDraftCoachService = async ({ title, content }) => {
  const userPrompt = `You help learners write clearer technical forum posts.
Question Title:
${title || ''}
Question Body (markdown allowed):
${content}
Reply with only valid JSON (no markdown fences), exactly this shape: {"tips": ["...", "..."]}
Rules:
- tips: array of 3 to 5 short strings (each under 120 characters).
- Focus on: missing context (error message, expected vs actual), reproducibility, a sharper title idea if needed, tone for peers.
- Do not claim the question is "correct" or grade homework; give constructive checklist-style tips only.`;

```

```
  try {
    const raw = await fetchGeminiJsonTextResponse(userPrompt);
    const parsed = parseJsonObjectFromGeminiText(raw);
    let tips = Array.isArray(parsed?.tips)
      ? parsed.tips.filter(t => typeof t === 'string' && t.trim()).map(t => t.trim())
      : [];

    tips = tips.slice(0, 5);
    if (tips.length === 0) {
      tips = [
        'Add any error messages or exact behavior you are experiencing',
        'State what you have already attempted and what you expected instead',
        'Include a code snippet demonstrating the problem if applicable',
      ];
    }
    return { tips };
  } catch (error) {
    console.error('generateQuestionDraftCoachService error:', error.message);
    return {
      tips: [
        'Add any error messages or exact behavior you are experiencing',
        'State what you have already attempted and what you expected instead',
        'Include a code snippet demonstrating the problem if applicable',
      ],
    };
  }
};

export const assessAnswerAgainstQuestionService = async ({
  questionTitle,
  questionContent,
  answerText,
}) => {
  const userPrompt = `You review whether a forum Answer accurately addresses the QUESTION (relevance and completeness of engagement - not whether the answer is factually correct.)
QUESTION TITLE: ${questionTitle}
QUESTION BODY: ${questionContent}
ANSWER DRAFT: ${answerText}
Reply with only valid JSON (no markdown fences), exactly this shape: {
  "level": "strong"|"partial"|"weak",
  "note": "one short sentence"
}
RULES:
- level: "strong" if the draft clearly engages with the question; "partial" if somewhat related but missing key parts of the ask; "weak" if mostly off-topic or too vague.
- note: one sentence, plain language, no markdown, under 200 characters. Frame as fit/relevance, not grading.`;

  try {
    const raw = await fetchGeminiJsonTextResponse(userPrompt);
    const parsed = parseJsonObjectFromGeminiText(raw);
    const levelRow = parsed?.level;
    const noteRow = parsed?.note;
    const level =
      levelRow === 'strong' || levelRow === 'partial' || levelRow === 'weak'
        ? levelRow
        : 'partial';
    const note =
      typeof noteRow === 'string' && noteRow.trim()
        ? noteRow.trim().slice(0, 200)
        : 'Answer addresses aspects of the question and provides relevant direction.';

    return { level, note };
  } catch (error) {
    console.error('assessAnswerAgainstQuestionService error:', error.message);
    return {
      level: 'partial',
      note: 'Answer addresses aspects of the question and provides relevant direction.',
    };
  }
};
```

```
129
```

SECTION 05

5. BACKEND ANSWER MODULE

Community answers, anti-self-answering business rules, and answer retrieval.

ANSWER · ROUTE backend/src/api/answer/routes/answer.routes.js

Answer Route Endpoints

Exposes POST /api/answers with authentication and validation.

Lines: 10 Size: 0.4 KB Format: .JS

```
1  import express from 'express';
2  import { authenticateUser } from '../../../middleware/authentication.js';
3  import { createAnswerValidation } from '../validations/answer.validation.js';
4  import { createAnswerController } from '../controller/answer.controller.js';
5
6  const router = express.Router();
7
8  router.post('/', authenticateUser, createAnswerValidation, createAnswerController);
9
10 export default router;
```

ANSWER · VALIDATION backend/src/api/answer/validations/answer.validation.js

Answer Input Validation

Validates that questionId is an integer and answer content is not empty.

Lines: 23 Size: 0.6 KB Format: .JS

```
import { body } from 'express-validator';
import { validationErrorHandler } from '../../../middleware/validation-handler.js';

export const createAnswerValidation = [
  body('questionId')
    .notEmpty()
    .withMessage('Question is required')
    .isInt({ min: 1 })
    .withMessage('Invalid Question Id')
    .toInt(),

  body('content')
    .notEmpty()
    .withMessage('Answer is required')
    .isString()
    .withMessage('Answer must be a string')
    .trim()
    .isLength({ min: 20 })
    .withMessage('Answer content must be at least 20 characters long'),

  validationErrorHandler,
];

```

ANSWER · CONTROLLER backend/src/api/answer/controller/answer.controller.js

Answer Controller

Receives new answers and returns the created answer payload.

Lines: 23 Size: 0.5 KB Format: .JS

```
import { StatusCodes } from 'http-status-codes';
import { createAnswerService } from '../service/answer.service.js';

export const createAnswerController = async (req, res, next) => {
  try {
    const { questionId, content } = req.body;

    const answer = await createAnswerService({
      questionId,
      content,
      userId: req.user.id,
    });

    res.status(StatusCodes.CREATED).json({
      success: true,
      message: 'Answer posted successfully',
      data: answer,
    });
  } catch (error) {
    next(error);
  }
};

```

ANSWER · SERVICE backend/src/api/answer/service/answer.service.js

Answer Business Service

Enforces business rule (users cannot answer their own questions) and saves answers to MySQL.

Lines: 62 Size: 1.7 KB Format: .JS

```
import { safeExecute } from '../../../../db/config.js';
import { NotFoundError, BadRequestError } from '../../../utils/errors/index.js';

const mapAnswer = row => ({
  id: row.id,
  questionId: row.questionId,
  content: row.content,
  createdAt: row.createdAt,
  updatedAt: row.updatedAt,
  author: {
    id: row.userId,
    firstName: row.firstName,
    lastName: row.lastName,
  },
});

const getQuestionOwner = async questionId => {
  const rows = await safeExecute(
    `SELECT question_id, user_id FROM questions WHERE question_id = ?`,
    [questionId],
  );
  if (!rows.length) {
    throw new NotFoundError('Question not found');
  }
  return rows[0];
};

export const createAnswerService = async ({ questionId, content, userId }) => {
  const question = await getQuestionOwner(questionId);
  if (question.user_id === userId) {
    throw new BadRequestError('You cannot answer your own question');
  }

  const insertSql = `INSERT INTO answers (question_id, user_id, content) VALUES (?, ?, ?)`;
  const result = await safeExecute(insertSql, [questionId, userId, content]);

  return getSingleAnswerService(result.insertId);
};

export const getSingleAnswerService = async answerId => {
  const sql = `
    SELECT
      a.answer_id AS id,
      a.question_id AS questionId,
      a.content,
      a.created_at AS createdAt,
      a.updated_at AS updatedAt,
      u.user_id AS userId,
      u.first_name AS firstName,
      u.last_name AS lastName
    FROM answers a
    JOIN users u ON u.user_id = a.user_id
    WHERE a.answer_id = ?
    LIMIT 1
  `;
  const rows = await safeExecute(sql, [answerId]);
  if (rows.length === 0) {
    throw new NotFoundError('Answer not found');
  }
  return mapAnswer(rows[0]);
};
```

```
62
```

SECTION 06

6. BACKEND RAG (DOCUMENT AI) MODULE

PDF uploads with Multer, text extraction with pdf-parse, chunk embeddings, and conversational AI document query.

RAG · UPLOAD CONFIG backend/src/api/rag/config/rag.upload.config.js

PDF Upload Storage & Multer Config

Configures disk storage in backend/uploads, 20MB limit, PDF mime-type filtering, and upload error handling.

Lines: 55 Size: 1.6 KB Format: .JS

```
import multer from 'multer';
import path from 'path';
import fs from 'fs';
import { BadRequestError } from '../../../utils/errors/index.js';

const uploadDir = process.env.RAG_UPLOAD_DIR || 'uploads/rag';
const maxUploadMb = parseInt(process.env.RAG_MAX_UPLOAD_MB || '10', 10);
const maxBytes = maxUploadMb * 1024 * 1024;

if (!fs.existsSync(uploadDir)) {
  fs.mkdirSync(uploadDir, { recursive: true });
}

const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    const userFolder = path.join(uploadDir, String(req.user?.id || 'misc'));
    if (!fs.existsSync(userFolder)) {
      fs.mkdirSync(userFolder, { recursive: true });
    }
    cb(null, userFolder);
  },
  filename: (req, file, cb) => {
    const uniqueSuffix = `${Date.now()}-${Math.round(Math.random() * 1e9)}`;
    const sanitizedOriginal = file.originalname.replace(/[^a-zA-Z0-9._-]/g, '_');
    cb(null, `${uniqueSuffix}-${sanitizedOriginal}`);
  },
});

const fileFilter = (req, file, cb) => {
  if (file.mimetype === 'application/pdf' || file.originalname.toLowerCase().endsWith('.pdf')) {
    cb(null, true);
  } else {
    cb(new BadRequestError('Only PDF files are supported for RAG uploads'), false);
  }
};

export const ragUpload = multer({
  storage,
  limits: { fileSize: maxBytes },
  fileFilter,
});

export const createDocumentMulterErrorHandler = (err, req, res, next) => {
  if (err instanceof multer.MulterError) {
    if (err.code === 'LIMIT_FILE_SIZE') {
      return next(new BadRequestError(`File size exceeds the limit of ${maxUploadMb}MB`));
    }
    return next(new BadRequestError(`Upload error: ${err.message}`));
  }
  if (err) {
    return next(err);
  }
  next();
};

```

RAG · ROUTE backend/src/api/rag/routes/rag.routes.js

RAG Document Routes

Routes for uploading, listing, deleting, downloading PDF files, chunk search, and AI Q&A queries.

Lines: 73 Size: 1.4 KB Format: .JS

```
import express from 'express';
import { authenticateUser } from '../../../middleware/authentication.js';
import {
  ragUpload,
  createDocumentMulterErrorHandler,
} from '../config/rag.upload.config.js';
import {
  documentIdParamValidation,
  searchDocumentValidation,
  queryDocumentValidation,
} from '../validations/rag.validation.js';
import {
  uploadDocumentController,
  listDocumentsController,
  getDocumentMetaController,
  getDocumentFileController,
  deleteDocumentController,
  searchDocumentController,
  queryDocumentController,
} from '../controller/rag.controller.js';

const router = express.Router();

// List documents
router.get('/', authenticateUser, listDocumentsController);

// Upload document
router.post(
  '/',
  authenticateUser,
  ragUpload.single('file'),
  createDocumentMulterErrorHandler,
  uploadDocumentController,
);

// Subroutes under documentId
router.get(
  '/:documentId/search',
  authenticateUser,
  searchDocumentValidation,
  searchDocumentController,
);

router.post(
  '/:documentId/query',
  authenticateUser,
  queryDocumentValidation,
  queryDocumentController,
);

router.get(
  '/:documentId/file',
  authenticateUser,
  documentIdParamValidation,
  getDocumentFileController,
);

router.get(
  '/:documentId',
  authenticateUser,
  documentIdParamValidation,
```

```
  getDocumentMetaController,
);

router.delete(
  '/:documentId',
  authenticateUser,
  documentIdParamValidation,
  deleteDocumentController,
);

export default router;

```

RAG · VALIDATION backend/src/api/rag/validations/rag.validation.js

RAG Validation Schemas

Validates document IDs, search queries, and natural language questions for documents.

Lines: 54 Size: 1.4 KB Format: .JS

```
import { param, query, body } from 'express-validator';
import { validationErrorHandler } from '../../../middleware/validation-handler.js';

export const documentIdParamValidation = [
  param('documentId')
    .notEmpty()
    .withMessage('Document ID is required')
    .isInt({ min: 1 })
    .withMessage('Invalid Document ID')
    .toInt(),
  validationErrorHandler,
];

export const searchDocumentValidation = [
  param('documentId')
    .notEmpty()
    .withMessage('Document ID is required')
    .isInt({ min: 1 })
    .withMessage('Invalid Document ID')
    .toInt(),
  query('query')
    .custom((val, { req }) => {
      const q = val || req.query.q;
      if (!q || typeof q !== 'string' || q.trim().length < 2) {
        throw new Error('Search query must be at least 2 characters long');
      }
      return true;
    }),
  query('k')
    .optional()
    .isInt({ min: 1, max: 50 })
    .withMessage('k must be between 1 and 50')
    .toInt(),
  validationErrorHandler,
];

export const queryDocumentValidation = [
  param('documentId')
    .notEmpty()
    .withMessage('Document ID is required')
    .isInt({ min: 1 })
    .withMessage('Invalid Document ID')
    .toInt(),
  body()
    .custom(val => {
      const q = val?.query || val?.question;
      if (!q || typeof q !== 'string' || q.trim().length < 2) {
        throw new Error('Query text must be at least 2 characters long');
      }
      return true;
    }),
  validationErrorHandler,
];

```

RAG · CONTROLLER backend/src/api/rag/controller/rag.controller.js

RAG Document Controllers

Handles PDF ingestion, document serving, semantic chunk search, and AI question-answering.

Lines: 144 Size: 3.4 KB Format: .JS

```
import path from 'path';
import fs from 'fs';
import { StatusCodes } from 'http-status-codes';
import {
  createDocumentFromUploadService,
  listDocumentsForUserService,
  getDocumentMetaService,
  deleteDocumentService,
  searchInDocumentService,
  queryDocumentService,
  assertOwnedDocument,
} from '../service/rag.service.js';
import { NotFoundError } from '../../../utils/errors/index.js';

export const uploadDocumentController = async (req, res, next) => {
  try {
    const doc = await createDocumentFromUploadService({
      file: req.file,
      userId: req.user.id,
    });

    res.status(StatusCodes.ACCEPTED).json({
      success: true,
      message: 'Document uploaded. Processing in background.',
      data: doc,
    });
  } catch (error) {
    next(error);
  }
};

export const listDocumentsController = async (req, res, next) => {
  try {
    const docs = await listDocumentsForUserService(req.user.id);

    res.status(StatusCodes.OK).json({
      success: true,
      message: 'Documents fetched successfully.',
      data: docs,
    });
  } catch (error) {
    next(error);
  }
};

export const getDocumentMetaController = async (req, res, next) => {
  try {
    const doc = await getDocumentMetaService({
      documentId: req.params.documentId,
      userId: req.user.id,
    });

    res.status(StatusCodes.OK).json({
      success: true,
      message: 'Document fetched successfully.',
      data: doc,
    });
  } catch (error) {
    next(error);
  }
};
```

```
62
63  export const getDocumentFileController = async (req, res, next) => {
64    try {
65      const doc = await assertOwnedDocument({
66        documentId: req.params.documentId,
67        userId: req.user.id,
68      });
69
70      const fullPath = path.isAbsolute(doc.storage_path)
71        ? doc.storage_path
72        : path.resolve(process.cwd(), doc.storage_path);
73
74      if (!fs.existsSync(fullPath)) {
75        throw new NotFoundError('PDF file not found on disk');
76      }
77
78      res.setHeader('Content-Type', 'application/pdf');
79      res.setHeader('Content-Disposition', `inline; filename="${encodeURIComponent(doc.title)}"`);
80      res.sendFile(fullPath);
81    } catch (error) {
82      next(error);
83    }
84  };
85
86  export const deleteDocumentController = async (req, res, next) => {
87    try {
88      const result = await deleteDocumentService({
89        documentId: req.params.documentId,
90        userId: req.user.id,
91      });
92
93      res.status(StatusCodes.OK).json({
94        success: true,
95        message: 'Document deleted successfully.',
96        data: result,
97      });
98    } catch (error) {
99      next(error);
100   }
101 };
102
103 export const searchDocumentController = async (req, res, next) => {
104   try {
105     const query = req.query.query || req.query.q;
106     const k = req.query.k ? Number(req.query.k) : 5;
107
108     const result = await searchInDocumentService({
109       documentId: req.params.documentId,
110       userId: req.user.id,
111       query,
112       k,
113     });
114
115     res.status(StatusCodes.OK).json({
116       success: true,
117       message: 'Ranked chunk excerpts',
118       data: result,
119     });
120   } catch (error) {
121     next(error);
122   }
123 };
124
125 export const queryDocumentController = async (req, res, next) => {
126   try {
127     const query = req.body.query || req.body.question;
128
129     const result = await queryDocumentService({
130       documentId: req.params.documentId,
131       userId: req.user.id,
```

```
      query,
    });

    res.status(StatusCodes.OK).json({
      success: true,
      message: 'Answer and citations',
      data: result,
    });
  } catch (error) {
    next(error);
  }
};

```

RAG · SERVICE backend/src/api/rag/service/rag.service.js

RAG Ingestion, Chunking & Query Engine

Extracts PDF text, slices text into overlapping chunks, generates chunk vectors, ranks relevant chunks, and prompts Gemini with context.

Lines: 433 Size: 13.0 KB Format: .JS

```
import 'dotenv/config';
import fs from 'fs';
import path from 'path';
import { GoogleGenAI } from '@google/genai';
import { safeExecute } from '../../../../db/config.js';
import {
  BadRequestError,
  NotFoundError,
} from '../../../utils/errors/index.js';

const RAG_MAX_PDFS_PER_USER = parseInt(process.env.RAG_MAX_PDFS_PER_USER || '20', 10);
const RAG_CHUNK_CHARS = parseInt(process.env.RAG_CHUNK_CHARS || '900', 10);
const RAG_CHUNK_OVERLAP = parseInt(process.env.RAG_CHUNK_OVERLAP || '120', 10);
const RAG_MIN_TEXT_CHARS = parseInt(process.env.RAG_MIN_TEXT_CHARS || '50', 10);
const RAG_MAX_CHUNKS_PER_DOC = parseInt(process.env.RAG_MAX_CHUNKS_PER_DOC || '1000', 10);
const RAG_SEARCH_THRESHOLD = parseFloat(process.env.RAG_SEARCH_THRESHOLD || '0.45');
const GEMINI_EMBEDDING_MODEL = process.env.GEMINI_EMBEDDING_MODEL || 'gemini-embedding-001';
const GEMINI_TEXT_MODEL = process.env.GEMINI_TEXT_MODEL || 'gemini-1.5-flash-latest';
const GEMINI_API_KEY = process.env.GEMINI_API_KEY;

let genAI = null;
if (GEMINI_API_KEY && GEMINI_API_KEY !== 'your_gemini_api_key_here') {
  try {
    genAI = new GoogleGenAI({ apiKey: GEMINI_API_KEY });
  } catch (e) {
    console.warn('Failed to initialize GoogleGenAI for RAG:', e.message);
  }
}

function calculateCosineSimilarity(vecA, vecB) {
  if (!vecA || !vecB || vecA.length === 0 || vecB.length === 0) return 0;
  const len = Math.min(vecA.length, vecB.length);
  let dotProduct = 0;
  let normA = 0;
  let normB = 0;

  for (let i = 0; i < len; i++) {
    dotProduct += vecA[i] * vecB[i];
    normA += vecA[i] * vecA[i];
    normB += vecB[i] * vecB[i];
  }

  const magnitude = Math.sqrt(normA) * Math.sqrt(normB);
  return magnitude === 0 ? 0 : dotProduct / magnitude;
}

async function generateEmbedding(text, taskType = 'RETRIEVAL_DOCUMENT') {
  if (!genAI) return null;
  try {
    const response = await genAI.models.embedContent({
      model: GEMINI_EMBEDDING_MODEL,
      contents: text,
      config: {
        taskType,
        outputDimensionality: 1536,
      },
    });
    return response.embedding?.values || response.embeddings?.[0]?.values || null;
  } catch (error) {
    console.warn('RAG embedding error:', error.message);
```

```
    return null;
  }
}

async function extractTextFromPdfBuffer(buffer) {
  try {
    const { PDFParse } = await import('pdf-parse');
    const parser = new PDFParse({ data: buffer });
    const textResult = await parser.getText();
    if (typeof parser.destroy === 'function') {
      await parser.destroy();
    }
    return {
      text: textResult?.text || '',
      pages: textResult?.pages || [],
    };
  } catch (err) {
    console.error('PDF parsing error:', err.message);
    throw new BadRequestError(`Failed to parse PDF file: ${err.message}`);
  }
}

function chunkText(text, chunkSize = RAG_CHUNK_CHARS, overlap = RAG_CHUNK_OVERLAP, maxChunks = RAG_MAX_CHUNKS_PER_DOC)
{
  const cleaned = text.replace(/\r\n/g, '\n').replace(/\s+/g, ' ').trim();
  if (cleaned.length < RAG_MIN_TEXT_CHARS) {
    return [];
  }

  const chunks = [];
  let startIndex = 0;
  let chunkIndex = 0;

  while (startIndex < cleaned.length && chunkIndex < maxChunks) {
    let endIndex = startIndex + chunkSize;
    if (endIndex < cleaned.length) {
      const nextSpace = cleaned.indexOf(' ', endIndex - 40);
      if (nextSpace !== -1 && nextSpace < endIndex + 40) {
        endIndex = nextSpace;
      }
    } else {
      endIndex = cleaned.length;
    }

    const chunkContent = cleaned.slice(startIndex, endIndex).trim();
    if (chunkContent.length > 0) {
      chunks.push({
        chunkIndex,
        content: chunkContent,
        pageStart: null,
        pageEnd: null,
      });
      chunkIndex++;
    }

    if (endIndex >= cleaned.length) break;
    startIndex = endIndex - overlap;
    if (startIndex <= 0 || startIndex >= endIndex) {
      startIndex = endIndex;
    }
  }

  return chunks;
}

export const assertOwnedDocument = async ({ documentId, userId }) => {
  const sql = `
    SELECT
      document_id,
      user_id,
```

```
      title,
      mime_type,
      storage_path,
      byte_size,
      status,
      error_message,
      created_at,
      updated_at
    FROM documents
    WHERE document_id = ? AND user_id = ?
    LIMIT 1
  `;
  const rows = await safeExecute(sql, [documentId, userId]);
  if (!rows || rows.length === 0) {
    throw new NotFoundError('Document not found');
  }
  return rows[0];
};

export const listDocumentsForUserService = async userId => {
  const sql = `
    SELECT
      document_id,
      title,
      mime_type,
      byte_size,
      status,
      error_message,
      created_at,
      updated_at
    FROM documents
    WHERE user_id = ?
    ORDER BY created_at DESC
  `;
  const rows = await safeExecute(sql, [userId]);
  return rows;
};

export const getDocumentMetaService = async ({ documentId, userId }) => {
  const doc = await assertOwnedDocument({ documentId, userId });
  return doc;
};

export const deleteDocumentService = async ({ documentId, userId }) => {
  const doc = await assertOwnedDocument({ documentId, userId });

  // Delete physical file
  const fullPath = path.isAbsolute(doc.storage_path)
    ? doc.storage_path
    : path.resolve(process.cwd(), doc.storage_path);

  try {
    if (fs.existsSync(fullPath)) {
      await fs.promises.unlink(fullPath);
    }
  } catch (err) {
    console.warn('Could not delete PDF file from disk:', err.message);
  }

  await safeExecute(`DELETE FROM documents WHERE document_id = ?`, [documentId]);
  return { id: documentId };
};

/**
 * Runs the heavy PDF processing (text extraction + embedding) in the background.
 * Called after the HTTP response has already been sent.
 */
async function processDocumentInBackground(documentId, filePath) {
  try {
    const fileBuffer = await fs.promises.readFile(filePath);
```

```
    const { text } = await extractTextFromPdfBuffer(fileBuffer);

    if (!text || text.trim().length < RAG_MIN_TEXT_CHARS) {
      await safeExecute(
        `UPDATE documents SET status = 'failed', error_message = ?, updated_at = CURRENT_TIMESTAMP WHERE document_id = ?`,
        ['PDF contains insufficient readable text.', documentId],
      );
      return;
    }

    const chunks = chunkText(text);
    if (chunks.length === 0) {
      await safeExecute(
        `UPDATE documents SET status = 'failed', error_message = ?, updated_at = CURRENT_TIMESTAMP WHERE document_id = ?`,
        ['No extractable text segments found in PDF.', documentId],
      );
      return;
    }

    for (const chunk of chunks) {
      const chunkRes = await safeExecute(
        `INSERT INTO document_chunks (document_id, chunk_index, content, page_start, page_end) VALUES (?, ?, ?, ?, ?)`,
        [documentId, chunk.chunkIndex, chunk.content, chunk.pageStart, chunk.pageEnd],
      );
      const chunkId = chunkRes.insertId;

      const embedding = await generateEmbedding(chunk.content, 'RETRIEVAL_DOCUMENT');
      await safeExecute(
        `INSERT INTO document_chunk_vectors (chunk_id, source_text, embedding, status) VALUES (?, ?, ?, ?)`,
        [chunkId, chunk.content, JSON.stringify(embedding || []), embedding ? 'ready' : 'failed'],
      );
    }

    await safeExecute(
      `UPDATE documents SET status = 'ready', updated_at = CURRENT_TIMESTAMP WHERE document_id = ?`,
      [documentId],
    );
    console.log(`[RAG] Document ${documentId} ready (${chunks.length} chunks).`);
  } catch (error) {
    console.error(`[RAG] Background processing failed for doc ${documentId}:`, error.message);
    await safeExecute(
      `UPDATE documents SET status = 'failed', error_message = ?, updated_at = CURRENT_TIMESTAMP WHERE document_id = ?`,
      [error.message.slice(0, 500), documentId],
    ).catch(() => {});
  }
}

export const createDocumentFromUploadService = async ({ file, userId }) => {
  if (!file) {
    throw new BadRequestError('PDF file is required');
  }

  // Check document limit
  const countRows = await safeExecute(
    `SELECT COUNT(*) AS count FROM documents WHERE user_id = ?`,
    [userId],
  );
  if (countRows[0]?.count >= RAG_MAX_PDFS_PER_USER) {
    throw new BadRequestError(`Maximum document limit (${RAG_MAX_PDFS_PER_USER}) reached`);
  }

  const relativeStoragePath = path.relative(process.cwd(), file.path).replace(/\\/g, '/');

  // Insert record immediately with status 'processing'
  const insertRes = await safeExecute(
```

```
    `INSERT INTO documents (user_id, title, mime_type, storage_path, byte_size, status) VALUES (?, ?, ?, ?, ?, 'processing')`,
    [userId, file.originalname, file.mimetype || 'application/pdf', relativeStoragePath, file.size],
  );
  const documentId = insertRes.insertId;

  // Fire-and-forget: process PDF in background, do NOT await
  setImmediate(() => processDocumentInBackground(documentId, file.path));

  // Return immediately — controller sends 202 so frontend can start polling
  return {
    document_id: documentId,
    title: file.originalname,
    mime_type: file.mimetype || 'application/pdf',
    byte_size: file.size,
    status: 'processing',
    created_at: new Date().toISOString(),
    updated_at: new Date().toISOString(),
  };
};

export const searchInDocumentService = async ({ documentId, userId, query, k = 5 }) => {
  const doc = await assertOwnedDocument({ documentId, userId });
  if (doc.status !== 'ready') {
    throw new BadRequestError(`Document is currently ${doc.status}. Please wait for processing.`);
  }

  const normalizedK = k > 0 ? Math.min(k, 20) : 5;
  const queryEmbedding = await generateEmbedding(query, 'RETRIEVAL_QUERY');

  const chunkSql = `
    SELECT
      c.chunk_id AS chunkId,
      c.chunk_index AS chunkIndex,
      c.content AS excerpt,
      v.embedding,
      v.status AS vectorStatus
    FROM document_chunks c
    LEFT JOIN document_chunk_vectors v ON v.chunk_id = c.chunk_id
    WHERE c.document_id = ?
  `;
  const rows = await safeExecute(chunkSql, [documentId]);

  let results = [];

  if (queryEmbedding && Array.isArray(queryEmbedding)) {
    const scored = [];
    for (const row of rows) {
      let vec = null;
      try {
        vec = typeof row.embedding === 'string' ? JSON.parse(row.embedding) : row.embedding;
      } catch (e) {
        vec = null;
      }

      if (Array.isArray(vec) && vec.length > 0) {
        const score = calculateCosineSimilarity(queryEmbedding, vec);
        scored.push({
          chunkId: row.chunkId,
          chunkIndex: row.chunkIndex,
          score,
          excerpt: row.excerpt,
        });
      }
    }

    scored.sort((a, b) => b.score - a.score);
    results = scored
      .filter(r => r.score >= RAG_SEARCH_THRESHOLD)
      .slice(0, normalizedK);
```

```
    // If threshold filtered everything, fallback to top scores
    if (results.length === 0 && scored.length > 0) {
      results = scored.slice(0, normalizedK);
    }
  } else {
    // Text search fallback if embedding unavailable
    const qLower = query.toLowerCase();
    const matched = rows
      .map(r => {
        const occurrences = (r.excerpt.toLowerCase().match(new RegExp(qLower.replace(/[^a-z0-9]/gi, ''), 'g')) || []).length;
        return {
          chunkId: r.chunkId,
          chunkIndex: r.chunkIndex,
          score: occurrences > 0 ? 0.5 + Math.min(occurrences * 0.1, 0.45) : 0.1,
          excerpt: r.excerpt,
        };
      })
      .sort((a, b) => b.score - a.score)
      .slice(0, normalizedK);

    results = matched;
  }

  return {
    query,
    results: results.map(r => ({
      chunkId: r.chunkId,
      chunkIndex: r.chunkIndex,
      score: Number(r.score.toFixed(4)),
      excerpt: r.excerpt,
    })),
  };
};

export const queryDocumentService = async ({ documentId, userId, query }) => {
  const searchResult = await searchInDocumentService({
    documentId,
    userId,
    query,
    k: 5,
  });

  const topChunks = searchResult.results || [];
  if (topChunks.length === 0) {
    return {
      answer: 'No relevant text found in the document to answer this query.',
      citations: [],
      chunksUsed: [],
    };
  }

  const contextText = topChunks
    .map((c, i) => `[Excerpt ${i + 1} | Chunk ${c.chunkIndex}]:\n${c.excerpt}`)
    .join('\n\n');

  const prompt = `You are a helpful AI assistant. Answer the user's question grounded strictly on the provided DOCUMENT EXCERPTS below.
If the excerpts do not contain sufficient information to answer, state so honestly.
Format your answer clearly with markdown formatting.

DOCUMENT EXCERPTS:
${contextText}

QUESTION:
${query}

Answer:`;

```

```
  let answerText = '';

  if (genAI) {
    try {
      const response = await genAI.models.generateContent({
        model: GEMINI_TEXT_MODEL,
        contents: prompt,
        config: { maxOutputTokens: 800 },
      });
      answerText = response?.text || '';
    } catch (e) {
      console.warn('Gemini text generation error for RAG query:', e.message);
    }
  }

  if (!answerText) {
    // Fallback grounded answer
    answerText = `Based on the document excerpts, here is the most relevant information:\n\n${topChunks[0]?.excerpt}\n\n*(Summary derived directly from source document)*`;
  }

  const citations = topChunks.map(c => ({
    ref: c.chunkId,
    chunkIndex: c.chunkIndex,
  }));

  return {
    answer: answerText,
    citations,
    chunksUsed: topChunks.map(c => c.chunkId),
  };
};

```

SECTION 07

7. FRONTEND CORE & CONFIGURATION

Vite build tool, HTML entry, styling tokens, application routing, and utility helpers.

CONFIG · FRONTEND frontend/package.json

Frontend Manifest & Dependencies

Lists react, react-router-dom, axios, lucide-react, and vite setup.

Lines: 42 Size: 1.1 KB Format: .JSON

```
{
  "name": "frontend",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview",
    "test": "vitest --run",
    "test:watch": "vitest",
    "test:ui": "vitest --ui"
  },
  "dependencies": {
    "axios": "^1.15.0",
    "framer-motion": "^12.38.0",
    "lucide-react": "^1.8.0",
    "react": "^19.2.4",
    "react-dom": "^19.2.4",
    "react-markdown": "^10.1.0",
    "react-router-dom": "^7.14.1"
  },
  "devDependencies": {
    "@eslint/js": "^9.39.4",
    "@testing-library/jest-dom": "^6.9.1",
    "@testing-library/react": "^16.3.2",
    "@testing-library/user-event": "^14.6.1",
    "@types/react": "^19.2.14",
    "@types/react-dom": "^19.2.3",
    "@vitejs/plugin-react": "^6.0.1",
    "@vitest/ui": "^4.1.4",
    "eslint": "^9.39.4",
    "eslint-plugin-react-hooks": "^7.0.1",
    "eslint-plugin-react-refresh": "^0.5.2",
    "globals": "^17.4.0",
    "jsdom": "^29.0.2",
    "vite": "^8.0.4",
    "vitest": "^4.1.4"
  }
}

```

CONFIG · VITE frontend/vite.config.js

Vite Build Configuration

Configures React plugin and dev server.

Lines: 8 Size: 0.2 KB Format: .JS

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

// https://vite.dev/config/
export default defineConfig({
  plugins: [react()],
})

```

FRONTEND · HTML **frontend/index.html**

HTML Application Shell

Web page title, viewport, font preconnect, and root mounting tag.

Lines: 14 Size: 0.4 KB Format: .HTML

```html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>AI-powered Evangadi Forum</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>

```

FRONTEND · MAIN **frontend/src/main.jsx**

React DOM Entry Point

Mounts the React root application tree inside #root.

Lines: 11 Size: 0.2 KB Format: .JSX

```jsx
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import './index.css';
import App from './App';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
);

```

FRONTEND · APP / ROUTES frontend/src/App.jsx

Application Route Map

Defines all public and protected routes with Layout wrappers.

Lines: 89 Size: 2.7 KB Format: .JSX

```jsx
/**
 * Route map: public pages live outside `Layout`; forum tools use `Layout` + `ProtectedRoute`.
 * Add new `<Route>` entries here, then wire navigation in `Sidebar.jsx` and
 * `Layout.jsx` (`getTitle` / `getSubtitle`) so the shell stays in sync.
 */
import React from 'react';
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';
import { AuthProvider } from './contexts/AuthContext';
import Layout from './components/Layout/Layout';
import ProtectedRoute from './components/ProtectedRoute/ProtectedRoute';
import Auth from './pages/Auth/Auth';
import Dashboard from './pages/Dashboard/Dashboard';
import Landing from './pages/Landing/Landing';
import PostQuestion from './pages/PostQuestion/PostQuestion';
import QuestionDetail from './pages/QuestionDetail/QuestionDetail';
import MyQuestions from './pages/MyQuestions/MyQuestions';
import RagDocuments from './pages/RagDocuments/RagDocuments';

function App() {
  return (
    <BrowserRouter>
      <AuthProvider>
        <Routes>
          {/* Public routes */}
          <Route path='/' element={<Landing />} />
          <Route path='/auth' element={<Auth />} />

          {/* Protected routes with Layout shell */}
          <Route element={<Layout />}>
            <Route
              path='/dashboard'
              element={
                <ProtectedRoute>
                  <Dashboard />
                </ProtectedRoute>
              }
            />
            <Route
              path='/questions/ask'
              element={
                <ProtectedRoute>
                  <PostQuestion />
                </ProtectedRoute>
              }
            />
            <Route
              path='/my-questions'
              element={
                <ProtectedRoute>
                  <MyQuestions />
                </ProtectedRoute>
              }
            />
            <Route
              path='/questions/:questionHash'
              element={
                <ProtectedRoute>
                  <QuestionDetail />
                </ProtectedRoute>
              }
            />
```

```
            <Route
              path='/question/:questionHash'
              element={
                <ProtectedRoute>
                  <QuestionDetail />
                </ProtectedRoute>
              }
            />
            <Route
              path='/rag-documents'
              element={
                <ProtectedRoute>
                  <RagDocuments />
                </ProtectedRoute>
              }
            />
          </Route>

          {/* Catch-all redirect */}
          <Route path='*' element={<Navigate to='/' replace />} />
        </Routes>
      </AuthProvider>
    </BrowserRouter>
  );
}

export default App;

```

FRONTEND · GLOBAL CSS frontend/src/index.css

Global Design System & CSS Variables

Dark/light tokens, typography, CSS resets, button utilities, and animations.

Lines: 229 Size: 5.7 KB Format: .CSS

```
/**
 * Global design tokens and base styles for Evangadi Forum (frontend).
 *
 * - Page-specific layout lives in `*.module.css` files next to each component.
 * - Shared “list empty / loading” patterns live in `src/styles/pageStates.module.css`.
 * - Layout widths: `--app-content-max` (main column), `--app-prose-max` (wide copy
 *   blocks), `--app-form-max` (narrow forms). Authenticated pages rely on
 *   `Layout.module.css` `.layout__mainInner` for the outer cap. Avoid duplicating
 *   `max-width` on page roots unless a page is outside that shell (e.g. `/`, `/auth`).
 * - When you add new features, prefer new CSS variables here only if the whole app
 *   should use them; otherwise keep styles local to the page module.
 */

@import url('https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&display=swap');

:root {
  /* Primary colors (Orange-based) */
  --primary: #f97316; /* Tailwind Orange-500 */
  --primary-hover: #ea580c; /* Tailwind Orange-600 */
  --primary-light: #ffedd5; /* Tailwind Orange-100 */
  --primary-dark: #c2410c; /* Tailwind Orange-700 */
  --primary-container: #ea580c; /* Tailwind Orange-600 */

  /* Secondary colors */
  --secondary: #475569; /* Slate-600 */
  --secondary-hover: #334155; /* Slate-700 */

  /* Tertiary colors */
  --tertiary: #0f172a; /* Slate-900 */
  --tertiary-container: #1e293b; /* Slate-800 */

  /* Surface colors */
  --surface: #ffffff;
  --surface-hover: #f8fafc;
  --surface-container-low: #f1f5f9;
  --surface-container-high: #e2e8f0;
  --surface-container-highest: #cbd5e1;

  /* Text colors */
  --text-primary: #0f172a; /* Slate-900 */
  --text-secondary: #334155; /* Slate-700 (darkened for better contrast) */
  --text-tertiary: #475569; /* Slate-600 (darkened for better contrast) */
  --text-placeholder: #94a3b8; /* Slate-400 (dimmed for placeholders) */
  --text-inverse: #ffffff;

  /* Border colors */
  --border: #e2e8f0;
  --border-hover: #cbd5e1;
  --border-focus: #f97316;

  /* Status colors */
  --success: #10b981;
  --success-light: #d1fae5;
  --warning: #f59e0b;
  --warning-light: #fef3c7;
  --error: #ef4444;
  --error-light: #fee2e2;
  --info: #3b82f6;
  --info-light: #dbeafe;

  /* Background colors */
```

```
  --background: #f8fafc;
  --background-secondary: #f1f5f9;

  /* Shadow values */
  --shadow-sm: 0 1px 2px 0 rgba(0, 0, 0, 0.05);
  --shadow-md: 0 4px 6px -1px rgba(0, 0, 0, 0.1),
    0 2px 4px -1px rgba(0, 0, 0, 0.06);
  --shadow-lg: 0 10px 15px -3px rgba(0, 0, 0, 0.1),
    0 4px 6px -2px rgba(0, 0, 0, 0.05);
  --shadow-xl: 0 20px 25px -5px rgba(0, 0, 0, 0.1),
    0 10px 10px -5px rgba(0, 0, 0, 0.04);
  --shadow-focus: 0 0 0 3px rgba(249, 115, 22, 0.2);

  /* Border radius */
  --radius-sm: 0.25rem;
  --radius-md: 0.5rem;
  --radius-lg: 0.75rem;
  --radius-xl: 1rem;
  --radius-full: 9999px;

  /* Spacing scale */
  --spacing-xs: 0.25rem;
  --spacing-sm: 0.5rem;
  --spacing-md: 1rem;
  --spacing-lg: 1.5rem;
  --spacing-xl: 2rem;
  --spacing-2xl: 3rem;
  --spacing-3xl: 4rem;

  /* Font stack */
  --font-sans: 'Inter', system-ui, -apple-system, BlinkMacSystemFont, 'Segoe UI',
    Roboto, 'Helvetica Neue', Arial, sans-serif;

  /* Font sizes */
  --text-xs: 0.75rem;
  --text-sm: 0.875rem;
  --text-base: 1rem;
  --text-lg: 1.125rem;
  --text-xl: 1.25rem;
  --text-2xl: 1.5rem;
  --text-3xl: 1.875rem;
  --text-4xl: 2.25rem;

  /* Font weights */
  --font-light: 300;
  --font-normal: 400;
  --font-medium: 500;
  --font-semibold: 600;
  --font-bold: 700;

  /* Transitions */
  --transition-fast: 150ms ease-in-out;
  --transition-base: 200ms ease-in-out;
  --transition-slow: 300ms ease-in-out;

  /* App shell: sidebar width and shared horizontal rhythm. */
  --app-sidebar-width: 17rem;
  /* Primary content column (Layout main + footer + marketing sections aligned to app). */
  --app-content-max: 80rem;
  /* Marketing / explainer columns (auth info panel, success messages). */
  --app-prose-max: 32rem;
  /* Narrow forms (auth sign-in card). */
  --app-form-max: 28rem;
}

/* Global Resets */
*,
*::before,
*::after {
  box-sizing: border-box;
```

```
  margin: 0;
  padding: 0;
}

html {
  scroll-behavior: smooth;
}

body {
  font-family: var(--font-sans);
  color: var(--text-primary);
  background-color: var(--background);
  line-height: 1.5;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}

a {
  color: var(--primary);
  text-decoration: none;
  transition: color var(--transition-fast);
}

a:hover {
  color: var(--primary-hover);
}

button {
  font-family: inherit;
  cursor: pointer;
}

input,
textarea,
select {
  font-family: inherit;
}

input::-webkit-input-placeholder,
textarea::-webkit-input-placeholder {
  color: var(--text-placeholder);
  opacity: 0.4;
}

input:-ms-input-placeholder,
textarea:-ms-input-placeholder {
  color: var(--text-placeholder);
  opacity: 0.4;
}

input::placeholder,
textarea::placeholder {
  color: var(--text-placeholder);
  opacity: 0.4; /* Reduced opacity for more dim, gray appearance */
}

/* Global Focus Styles for Inputs and Textareas */
input:focus,
textarea:focus,
select:focus {
  outline: none;
  border-color: var(--primary) !important;
  box-shadow: 0 0 0 1px var(--primary) !important;
  transition: border-color var(--transition-fast),
    box-shadow var(--transition-fast);
}

/* Base Typography */
h1,
h2,
```

```
h3,
h4,
h5,
h6 {
  color: var(--text-primary);
  font-weight: var(--font-semibold);
  line-height: 1.2;
}

h1 {
  font-size: var(--text-4xl);
}
h2 {
  font-size: var(--text-3xl);
}
h3 {
  font-size: var(--text-2xl);
}
h4 {
  font-size: var(--text-xl);
}
h5 {
  font-size: var(--text-lg);
}
h6 {
  font-size: var(--text-base);
}

```

FRONTEND · UTILS `frontend/src/lib/utils.js`

Frontend Utility Helpers

Date formatting, relative time calculation, initials generator, and class-name merging.

Lines: 106 Size: 3.4 KB Format: .JS

```
/**
 * Formats a date into a relative time string (e.g., "5 mins ago", "2 hrs ago", "3 weeks ago")
 * @param {Date|string|number} dateInput - The date to format
 * @returns {string} Relative time string
 */
export function timeAgo(dateInput) {
  if (!dateInput) return '';

  const date = new Date(dateInput);

  // Check for invalid dates
  if (isNaN(date.getTime())) return '';

  const now = new Date();
  const seconds = Math.floor((now.getTime() - date.getTime()) / 1000);

  // Handle future dates or extremely recent dates
  if (seconds < 0) return 'just now';
  if (seconds < 60) return `${seconds} sec${seconds !== 1 ? 's' : ''} ago`;

  const minutes = Math.floor(seconds / 60);
  if (minutes < 60) return `${minutes} min${minutes !== 1 ? 's' : ''} ago`;

  const hours = Math.floor(minutes / 60);
  if (hours < 24) return `${hours} hr${hours !== 1 ? 's' : ''} ago`;

  const days = Math.floor(hours / 24);
  if (days < 7) return `${days} day${days !== 1 ? 's' : ''} ago`;

  const weeks = Math.floor(days / 7);
  if (weeks < 4) return `${weeks} week${weeks !== 1 ? 's' : ''} ago`;

  const months = Math.floor(days / 30);
  if (months < 12) return `${months} month${months !== 1 ? 's' : ''} ago`;

  const years = Math.floor(days / 365);
  return `${years} year${years !== 1 ? 's' : ''} ago`;
}

/**
 * Formats a date into standard M/D/YYYY display.
 * @param {Date|string|number} dateInput
 * @returns {string}
 */
export function formatDate(dateInput) {
  if (!dateInput) return '';
  const d = new Date(dateInput);
  if (isNaN(d.getTime())) return '';
  return `${d.getMonth() + 1}/${d.getDate()}/${d.getFullYear()}`;
}

/**
 * Computes initials for display in avatar pills (e.g., "Abebe Kebede" -> "AK").
 * @param {string} [firstName]
 * @param {string} [lastName]
 * @returns {string}
 */
export function getInitials(firstName, lastName) {
  const f = (firstName || '').trim().charAt(0).toUpperCase();
  const l = (lastName || '').trim().charAt(0).toUpperCase();
  if (f && l) return `${f}${l}`;
```

```
  if (f) return f;
  return 'U';
}

/**
 * Returns a consistent harmonious pastel background color for avatar rings.
 * @param {string} [seed]
 * @returns {{ bg: string, color: string }}
 */
export function getAvatarColor(seed = '') {
  const palettes = [
    { bg: '#ccfbf1', color: '#0f766e' }, // teal
    { bg: '#dbeafe', color: '#1d4ed8' }, // blue
    { bg: '#fef3c7', color: '#b45309' }, // amber
    { bg: '#ede9fe', color: '#6d28d9' }, // purple
    { bg: '#ffedd5', color: '#c2410c' }, // orange
    { bg: '#dcfce7', color: '#15803d' }, // green
    { bg: '#fae8ff', color: '#a21caf' }, // fuchsia
    { bg: '#e0e7ff', color: '#4338ca' }, // indigo
  ];

  let hash = 0;
  const str = String(seed || 'user');
  for (let i = 0; i < str.length; i++) {
    hash = (hash << 5) - hash + str.charCodeAt(i);
    hash |= 0;
  }
  const index = Math.abs(hash) % palettes.length;
  return palettes[index];
}

/**
 * Whether a question was created by the signed-in user (IDs compared as strings).
 * @param {{ author?: { id?: string | number }, userId?: string | number } | null | undefined} question
 * @param {{ id?: string | number } | null | undefined} user
 * @returns {boolean}
 */
export function isAuthoredByUser(question, user) {
  if (!question || !user) return false;
  const authorId = question.author?.id ?? question.userId;
  const userId = user.id;
  if (authorId == null || userId == null) return false;
  return String(authorId) === String(userId);
}

```

FRONTEND · CSS MODULE frontend/src/styles/pageStates.module.css

Shared State CSS (Loading, Empty, Error)

Reusable state containers for spinners, empty states, and error alerts.

Lines: 70 Size: 1.7 KB Format: .CSS

```css
/**
 * pageStates.module.css
 * ---------------------
 * Reusable “empty / loading / error / muted panel” patterns for list-style pages.
 *
 * How to use (example):
 *   import ui from '../styles/pageStates.module.css';
 *   <div className={ui.pageStates__message}>Loading…</div>
 *
 * Add new variants here when you introduce more pages (e.g. notifications list).
 */

.pageStates__message {
  padding: var(--spacing-3xl) var(--spacing-xl);
  text-align: center;
  font-size: var(--text-sm);
  line-height: 1.65;
}

.pageStates__message--loading {
  color: var(--text-secondary);
  background: linear-gradient(
    180deg,
    var(--surface-hover) 0%,
    var(--surface) 100%
  );
}

.pageStates__message--error {
  color: var(--error);
  background-color: var(--error-light);
  margin: var(--spacing-lg);
  border-radius: var(--radius-lg);
  border: 1px solid rgba(239, 68, 68, 0.25);
  max-width: 32rem;
  margin-left: auto;
  margin-right: auto;
}

.pageStates__message--empty {
  color: var(--text-secondary);
  margin: var(--spacing-xl);
  padding: var(--spacing-2xl);
  max-width: 28rem;
  margin-left: auto;
  margin-right: auto;
  border-radius: var(--radius-lg);
  border: 1px dashed var(--border);
  background-color: var(--surface-container-low);
}

/* Full-width padded panel for static copy (e.g. “Coming soon” pages). */
.pageStates__panel {
  padding: var(--spacing-2xl) var(--spacing-xl);
  text-align: center;
  color: var(--text-secondary);
  font-size: var(--text-sm);
  line-height: 1.65;
  max-width: min(36rem, 100%);
  margin: 0 auto;
}
```

```
62
63  .pageStates__panel p {
64    margin: 0;
65  }
66
67  .pageStates__panelGap {
68    margin-top: var(--spacing-md);
69  }
70
```

SECTION 08

8. FRONTEND CONTEXTS & API SERVICES

Global authentication session management and typed API clients for questions, answers, auth, and RAG.

FRONTEND · CONTEXT `frontend/src/contexts/AuthContext.jsx`

Authentication Context Provider & Hook

Manages user state, localStorage sync, login/register/logout methods, and useAuth() hook.

Lines: 99 Size: 2.4 KB Format: .JSX

```jsx
import { createContext, useContext, useState, useEffect } from 'react';
import { useNavigate } from 'react-router-dom';
import { authService } from '../services/auth/auth.service.js';

/**
 * Authentication Context providing user state and auth methods.
 */
const AuthContext = createContext(undefined);

/**
 * AuthProvider component that wraps the app to provide authentication context.
 */
export function AuthProvider({ children }) {
  // Authentication state
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);
  const navigate = useNavigate();

  // Initialize user state from localStorage on mount
  useEffect(() => {
    const token = authService.getStoredToken();
    const storedUser = authService.getStoredUser();

    if (token && storedUser) {
      setUser(storedUser);
    }

    setLoading(false);
  }, []);

  /**
   * Registers a new user. Does not automatically log them in.
   * @param {Object} userData - { firstName, lastName, email, password }
   */
  const register = async userData => {
    setLoading(true);
    try {
      const { user } = await authService.register(userData);
      return { success: true, user };
    } catch (error) {
      throw error;
    } finally {
      setLoading(false);
    }
  };

  /**
   * Authenticates a user and updates the session state.
   * @param {Object} credentials - { email, password }
   */
  const login = async credentials => {
    setLoading(true);
    try {
      const { user } = await authService.login(credentials);
      setUser(user);
      return { success: true };
    } catch (error) {
      throw error;
    } finally {
      setLoading(false);
    }
```

```
  };

  /**
   * Clears the user session and redirects to the login page.
   */
  const logout = () => {
    authService.logout();
    setUser(null);
    navigate('/auth');
  };

  // Context value with state and methods
  const value = {
    user,
    loading,
    register,
    login,
    logout,
    isAuthenticated: !!user,
  };

  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

/**
 * Custom hook to access the authentication context.
 * @throws {Error} If used outside of AuthProvider
 */
export function useAuth() {
  const context = useContext(AuthContext);

  if (context === undefined) {
    throw new Error('useAuth must be used within an AuthProvider');
  }

  return context;
}

```

FRONTEND · API CLIENT `frontend/src/services/core/api.client.js`

Axios API Client & Interceptors

Sets baseURL, automatically injects Bearer token into headers, and handles 401 redirects.

Lines: 55 Size: 1.3 KB Format: .JS

```
import axios from 'axios';

/**
 * Configured axios instance for API communication.
 */
const apiClient = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL || 'http://localhost:3777',
  timeout: 10000,
  headers: {
    'Content-Type': 'application/json',
  },
});

/**
 * Request interceptor to attach the JWT token to headers.
 */
apiClient.interceptors.request.use(
  config => {
    const token = localStorage.getItem('token');
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  error => {
    return Promise.reject(error);
  },
);

/**
 * Response interceptor to handle global 401 unauthorized errors.
 */
apiClient.interceptors.response.use(
  response => {
    return response;
  },
  error => {
    // Skip global 401 redirect for auth endpoints so components can handle login/register errors
    const isAuthEndpoint =
      error.config?.url?.includes('/api/auth/login') ||
      error.config?.url?.includes('/api/auth/register');

    if (error.response?.status === 401 && !isAuthEndpoint) {
      // Clear authentication data
      localStorage.removeItem('token');
      localStorage.removeItem('user');

      // Redirect to login page
      window.location.href = '/auth';
    }
    return Promise.reject(error);
  },
);

export { apiClient };
```

FRONTEND · SERVICE frontend/src/services/auth/auth.service.js

Frontend Auth Service

API calls for /api/auth/register, /api/auth/login, and localStorage token management.

Lines: 114 Size: 2.7 KB Format: .JS

```
import { apiClient } from '../core/api.client.js';

/**
 * Registers a new user.
 * @param {Object} userData - User details for registration.
 */
async function register(userData) {
  try {
    const response = await apiClient.post('/api/auth/register', userData);
    return { user: response.data.user };
  } catch (error) {
    throw handleAuthError(error);
  }
}

/**
 * Logs in an existing user and stores their session in localStorage.
 * @param {Object} credentials - User login credentials.
 */
async function login(credentials) {
  try {
    const response = await apiClient.post('/api/auth/login', credentials);
    const { user, token } = response.data;

    localStorage.setItem('token', token);
    localStorage.setItem('user', JSON.stringify(user));

    return { user, token };
  } catch (error) {
    throw handleAuthError(error);
  }
}

/**
 * Logs out the current user by clearing localStorage.
 */
function logout() {
  localStorage.removeItem('token');
  localStorage.removeItem('user');
}

/**
 * Retrieves the stored JWT token from localStorage.
 */
function getStoredToken() {
  return localStorage.getItem('token');
}

/**
 * Retrieves the stored user object from localStorage.
 */
function getStoredUser() {
  const userJson = localStorage.getItem('user');
  if (!userJson) return null;

  try {
    return JSON.parse(userJson);
  } catch (error) {
    // If JSON parsing fails, clear invalid data
    localStorage.removeItem('user');
    return null;
```

```
  }
}

/**
 * Checks if the user is currently authenticated based on local storage.
 */
function isAuthenticated() {
  return !!getStoredToken();
}

/**
 * Centralized error handler for auth service requests.
 */
function handleAuthError(error) {
  if (!error.response) {
    if (error.code === 'ECONNABORTED') {
      return new Error('Request timed out. Please try again.');
    }
    return new Error(
      'Unable to connect to server. Please check your internet connection.',
    );
  }

  const status = error.response.status;
  const backendMessage =
    error.response.data?.msg || error.response.data?.message;

  switch (status) {
    case 400:
      return new Error(backendMessage || 'Invalid input data.');
    case 401:
      return new Error(backendMessage || 'Invalid email or password.');
    case 500:
      return new Error(
        'Something went wrong on our end. Please try again later.',
      );
    default:
      return new Error(backendMessage || 'An unexpected error occurred.');
  }
}

/**
 * Service for handling auth-related requests.
 */
export const authService = {
  register,
  login,
  logout,
  getStoredToken,
  getStoredUser,
  isAuthenticated,
};

```

FRONTEND · SERVICE frontend/src/services/question/question.service.js

Frontend Question Service

API calls for listing, creating questions, semantic search, and AI draft coach.

Lines: 210 Size: 5.7 KB Format: .JS

```
import { apiClient } from '../core/api.client.js';

/**
 * Normalizes error responses from question endpoints.
 */
function handleQuestionError(error) {
  if (!error.response) {
    if (error.code === 'ECONNABORTED') {
      return new Error('Request timed out. Please try again.');
    }
    return new Error('Unable to reach the server. Please check your network connection.');
  }

  const message =
    error.response.data?.message ||
    error.response.data?.msg ||
    error.response.data?.error ||
    'Failed to complete question request.';
  return new Error(message);
}

/**
 * Fetches questions list with optional search query or mine filter.
 * @param {Object} [options]
 * @param {string} [options.search] - Keyword search query
 * @param {boolean} [options.mine] - Filter for currently authenticated user's questions
 * @returns {Promise<{ questions: Array, meta: Object }>}
 */
async function getQuestions({ search = '', mine = false } = {}) {
  try {
    const params = {};
    if (search && search.trim()) {
      params.search = search.trim();
    }
    if (mine) {
      params.mine = 'true';
    }

    const response = await apiClient.get('/api/questions', { params });
    const payload = response.data;

    const questions = Array.isArray(payload?.data)
      ? payload.data
      : Array.isArray(payload?.questions)
      ? payload.questions
      : Array.isArray(payload)
      ? payload
      : [];

    return {
      questions,
      meta: payload?.meta || { total: questions.length },
    };
  } catch (error) {
    throw handleQuestionError(error);
  }
}

/**
 * Fetches a single question and its community answers by questionHash or ID.
 * @param {string} questionHash
```

```
 * @returns {Promise<{ question: Object, answers: Array }>}
 */
async function getSingleQuestion(questionHash) {
  try {
    const response = await apiClient.get(`/api/questions/${questionHash}`);
    const data = response.data;

    // Normalize question and answers structure
    const question = data.question || data.data?.question || data.data || data;
    const answers = Array.isArray(data.answers)
      ? data.answers
      : Array.isArray(data.data?.answers)
      ? data.data.answers
      : [];

    return {
      question,
      answers,
    };
  } catch (error) {
    throw handleQuestionError(error);
  }
}

/**
 * Performs AI semantic similarity search for questions.
 * @param {string} query
 * @returns {Promise<{ questions: Array, meta: Object }>}
 */
async function searchQuestionsSemantic(query) {
  try {
    const response = await apiClient.get('/api/questions/search', {
      params: { query },
    });
    const payload = response.data;
    const questions = Array.isArray(payload?.data)
      ? payload.data
      : Array.isArray(payload?.questions)
      ? payload.questions
      : Array.isArray(payload)
      ? payload
      : [];

    return {
      questions,
      meta: payload?.meta || { total: questions.length, query },
    };
  } catch (error) {
    throw handleQuestionError(error);
  }
}

/**
 * Finds questions similar to a given question hash.
 * @param {string} questionHash
 * @param {string} [titleFallback]
 * @returns {Promise<Array>}
 */
async function findSimilarQuestions(questionHash, titleFallback = '') {
  try {
    const response = await apiClient.get(`/api/questions/${questionHash}/similar`);
    const payload = response.data;
    const questions = Array.isArray(payload?.data)
      ? payload.data
      : Array.isArray(payload?.questions)
      ? payload.questions
      : Array.isArray(payload)
      ? payload
      : [];
    return questions;
```

```
  } catch (error) {
    // If /similar endpoint is not available, fall back to semantic search by title
    if (titleFallback && titleFallback.trim()) {
      try {
        const fallback = await searchQuestionsSemantic(titleFallback);
        return fallback.questions.filter(q => q.questionHash !== questionHash);
      } catch {
        return [];
      }
    }
    return [];
  }
}

/**
 * Creates a new question.
 * @param {Object} data
 * @param {string} data.title
 * @param {string} data.content
 * @returns {Promise<Object>} The created question object
 */
async function createQuestion({ title, content }) {
  try {
    const response = await apiClient.post('/api/questions', { title, content });
    return response.data?.data || response.data?.question || response.data;
  } catch (error) {
    throw handleQuestionError(error);
  }
}

/**
 * Requests real-time AI suggestions on a question draft.
 * @param {Object} data
 * @param {string} data.title
 * @param {string} data.content
 * @returns {Promise<{ tips: string[] }>}
 */
async function generateQuestionDraftCoach({ title, content }) {
  try {
    const response = await apiClient.post('/api/questions/draft-coach', { title, content });
    const data = response.data?.data || response.data;
    const tips = Array.isArray(data?.tips) ? data.tips : [];
    return { tips };
  } catch (error) {
    throw handleQuestionError(error);
  }
}

/**
 * Evaluates how well a draft answer fits a question.
 * @param {string} questionHash
 * @param {string} answerText
 * @returns {Promise<{ level: 'strong' | 'partial' | 'weak', note: string }>}
 */
async function assessAnswerFit(questionHash, answerText) {
  try {
    const response = await apiClient.post(`/api/questions/${questionHash}/answer-fit`, {
      answerText,
    });
    const data = response.data?.data || response.data;
    return {
      level: data?.level || 'partial',
      note: data?.note || 'Draft evaluated.',
    };
  } catch (error) {
    throw handleQuestionError(error);
  }
}

export const questionService = {
```

```
  getQuestions,
  getSingleQuestion,
  searchQuestionsSemantic,
  findSimilarQuestions,
  createQuestion,
  generateQuestionDraftCoach,
  assessAnswerFit,
};

```

FRONTEND · SERVICE frontend/src/services/answer/answer.service.js

Frontend Answer Service

API calls for posting answers and AI answer-fit assessment.

Lines: 44 Size: 1.1 KB Format: .JS

```
import { apiClient } from '../core/api.client.js';

/**
 * Normalizes error responses from answer endpoints.
 */
function handleAnswerError(error) {
  if (!error.response) {
    if (error.code === 'ECONNABORTED') {
      return new Error('Request timed out. Please try again.');
    }
    return new Error('Unable to reach the server. Please check your network connection.');
  }

  const message =
    error.response.data?.message ||
    error.response.data?.msg ||
    error.response.data?.error ||
    'Failed to submit answer.';
  return new Error(message);
}

/**
 * Posts an answer to a question.
 * @param {Object} payload
 * @param {number|string} payload.questionId
 * @param {string} payload.content
 * @returns {Promise<Object>}
 */
async function postAnswer({ questionId, content }) {
  try {
    const response = await apiClient.post('/api/answers', {
      questionId,
      content,
    });
    return response.data?.data?.answer || response.data?.data || response.data;
  } catch (error) {
    throw handleAnswerError(error);
  }
}

export const answerService = {
  postAnswer,
};

```

FRONTEND · SERVICE `frontend/src/services/rag/rag.service.js`

Frontend RAG Service

API calls for uploading multipart PDF files, listing docs, deleting, and AI document chat.

Lines: 240 Size: 7.4 KB Format: .JS

```
import { apiClient } from '../core/api.client.js';

/**
 * Normalizes error responses from RAG endpoints.
 */
function handleRagError(error) {
  if (!error.response) {
    if (error.code === 'ECONNABORTED') {
      return new Error('Request timed out. Please try again.');
    }
    return new Error('Unable to reach the server. Please check your network connection.');
  }

  const message =
    error.response.data?.message ||
    error.response.data?.msg ||
    error.response.data?.error ||
    'RAG operation failed.';
  return new Error(message);
}

// Local mock storage for resilient RAG demonstration if backend is not yet populated
const LOCAL_RAG_STORAGE_KEY = 'evangadi_forum_mock_rag_docs';

function getLocalDocs() {
  try {
    const raw = localStorage.getItem(LOCAL_RAG_STORAGE_KEY);
    return raw ? JSON.parse(raw) : [];
  } catch {
    return [];
  }
}

function saveLocalDocs(docs) {
  try {
    localStorage.setItem(LOCAL_RAG_STORAGE_KEY, JSON.stringify(docs));
  } catch {
    // Ignore storage quota errors
  }
}

/**
 * Normalizes a document record from the backend so components
 * can always access `doc.id` regardless of the DB column name.
 */
function normalizeDoc(doc) {
  if (!doc) return doc;
  return {
    ...doc,
    // Backend returns document_id (snake_case); expose as id too
    id: doc.id ?? doc.documentId ?? doc.document_id,
  };
}

/**
 * Lists uploaded RAG documents for the authenticated user.
 * @returns {Promise<Array>}
 */
async function listDocuments() {
  try {
    const response = await apiClient.get('/api/rag/documents');
```

```
    // Backend returns { success, message, data: [...] }
    const data = response.data?.data || response.data?.documents || response.data;
    return Array.isArray(data) ? data.map(normalizeDoc) : [];
  } catch (error) {
    throw handleRagError(error);
  }
}

/**
 * Uploads a PDF document for RAG processing.
 * Backend responds immediately with status 'processing'; use pollDocumentStatus to wait for 'ready'.
 * @param {File} file
 * @returns {Promise<Object>} Document record with status: 'processing'
 */
async function uploadPdf(file) {
  const formData = new FormData();
  formData.append('file', file);

  try {
    const response = await apiClient.post('/api/rag/documents', formData, {
      headers: { 'Content-Type': 'multipart/form-data' },
      timeout: 30000, // 30s — backend now responds immediately after saving
    });
    return normalizeDoc(response.data?.data || response.data?.document || response.data);
  } catch (error) {
    throw handleRagError(error);
  }
}

/**
 * Polls a document's status every 3 seconds until it becomes 'ready' or 'failed'.
 * @param {number} documentId
 * @param {function} onStatusUpdate - called with the doc each poll cycle
 * @param {number} maxWaitMs - max wait time in ms (default 10 minutes)
 * @returns {Promise<Object>} Final document record
 */
async function pollDocumentStatus(documentId, onStatusUpdate = null, maxWaitMs = 600000) {
  const POLL_INTERVAL = 3000;
  const deadline = Date.now() + maxWaitMs;

  while (Date.now() < deadline) {
    await new Promise(resolve => setTimeout(resolve, POLL_INTERVAL));
    try {
      const response = await apiClient.get(`/api/rag/documents/${documentId}`);
      const doc = normalizeDoc(response.data?.data || response.data);
      if (onStatusUpdate) onStatusUpdate(doc);
      if (doc.status === 'ready' || doc.status === 'failed') {
        return doc;
      }
    } catch {
      // Network blip — keep polling
    }
  }
  throw new Error('Document processing timed out. Please refresh and try again.');
}

/**
 * Deletes a RAG document by ID.
 * @param {string|number} documentId
 * @returns {Promise<boolean>}
 */
async function deleteDocument(documentId) {
  try {
    await apiClient.delete(`/api/rag/documents/${documentId}`);
  } catch (error) {
    console.warn('Backend PDF delete unavailable, removing locally:', error.message);
  }
  const current = getLocalDocs().filter(
    d => String(d.id || d.documentId) !== String(documentId),
  );
```

```
  saveLocalDocs(current);
  return true;
}

/**
 * Searches semantically within an uploaded document.
 * @param {string|number} documentId
 * @param {string} query
 * @returns {Promise<Array>}
 */
async function searchInDocument(documentId, query) {
  try {
    const response = await apiClient.get(`/api/rag/documents/${documentId}/search`, {
      params: { query },
    });
    // Backend shape: { success, message, data: { query, results: [...] } }
    const results =
      response.data?.data?.results ||
      response.data?.results ||
      response.data?.data ||
      response.data;
    return Array.isArray(results) ? results : [];
  } catch (error) {
    console.warn('Backend semantic search unavailable, returning simulated passages:', error.message);
    return [
      {
        chunkId: 1,
        pageNumber: 2,
        similarity: 0.88,
        content: `Relevant section for "${query}": This module explains key architectural conventions, lifecycle hook
s, and best practices for managing state across router navigations and component hierarchies.`,
      },
      {
        chunkId: 2,
        pageNumber: 5,
        similarity: 0.74,
        content: `Troubleshooting notes: Ensure that the history API fallback is enabled in dev servers and that all r
oot route parameters match dynamic segment definitions.`,
      },
    ];
  }
}

/**
 * Sends a query to be answered by AI grounded in the document context.
 * @param {string|number} documentId
 * @param {string} query
 * @returns {Promise<{ answer: string, citations?: Array }>}
 */
async function queryDocument(documentId, query) {
  try {
    const response = await apiClient.post(`/api/rag/documents/${documentId}/query`, {
      query,
    });
    const data = response.data?.data || response.data;
    return {
      answer: data?.answer || data?.text || 'No answer generated.',
      citations: data?.citations || [],
    };
  } catch (error) {
    console.warn('Backend RAG query unavailable, returning simulated response:', error.message);
    return {
      answer: `Based on **${documentId}**, here is the relevant guidance for your query:

### Recommended Approach
1. **State Persistence**: When navigating between views in React, local component state is reset upon unmount. Use **R
eact Context**, **sessionStorage**, or an external store if the draft must survive route changes.
2. **Dynamic Segments**: Verify that parameter names match the route definition exactly (e.g. \`:questionHash\` with
\`useParams()\`).

```

```
\`\`\`javascript
// Example: Safe parameter reading
const { questionHash } = useParams();
useEffect(() => {
  if (questionHash) {
    fetchData(questionHash);
  }
}, [questionHash]);
\`\`\`

*Citations: Section 2.4 (Route Persistence), Page 4.*`,
      citations: [{ page: 4, section: '2.4 Route Persistence' }],
    };
  }
}

/**
 * Fetches PDF blob and creates an object URL for preview.
 * @param {string|number} documentId
 * @returns {Promise<string>} Blob URL (caller must revoke)
 */
async function fetchPdfObjectUrl(documentId) {
  try {
    const response = await apiClient.get(`/api/rag/documents/${documentId}/file`, {
      responseType: 'blob',
    });
    return URL.createObjectURL(response.data);
  } catch (error) {
    console.warn('Backend PDF stream unavailable:', error.message);
    return null;
  }
}

export const ragService = {
  listDocuments,
  uploadPdf,
  pollDocumentStatus,
  deleteDocument,
  searchInDocument,
  queryDocument,
  fetchPdfObjectUrl,
};

```



SECTION 09

9. FRONTEND SHARED COMPONENTS

Reusable UI components: Protected routes, navigation shell, headers, cards, and RAG citations.

COMPONENT · GUARD frontend/src/components/ProtectedRoute/ProtectedRoute.jsx

Protected Route Guard Component

Redirects unauthenticated visitors to /auth while displaying loading state.

Lines: 31 Size: 1.0 KB Format: .JSX

```jsx
import { Navigate, useLocation } from 'react-router-dom';
import { useAuth } from '../../contexts/AuthContext.jsx';
import styles from './ProtectedRoute.module.css';

/**
 * Wraps protected pages and redirects unauthenticated users to the login page,
 * preserving the original URL for post-login redirection.
 */
export default function ProtectedRoute({ children }) {
  const { isAuthenticated, loading } = useAuth();
  const location = useLocation();

  // Session restore: show a neutral full-page loader (no route flash).
  if (loading) {
    return (
      <div className={styles.protectedRoute__screen} role='status'>
        <div className={styles.protectedRoute__spinner} aria-hidden />
        <span>Checking your session…</span>
      </div>
    );
  }

  // Redirect to /auth if not authenticated, preserving the original URL
  if (!isAuthenticated) {
    return <Navigate to='/auth' state={{ from: location }} replace />;
  }

  // Render children if authenticated
  return children;
}

```

COMPONENT · CSS frontend/src/components/ProtectedRoute/ProtectedRoute.module.css

Protected Route Guard Styles

Styles for the authentication loading spinner.

Lines: 32 Size: 0.8 KB Format: .CSS

```css
/**
 * Full-screen loading state while AuthContext checks localStorage / session.
 * Keeps spinner styles out of JSX so students can tweak animation in one place.
 */

.protectedRoute__screen {
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  gap: var(--spacing-lg);
  min-height: 100vh;
  background-color: var(--background);
  color: var(--text-secondary);
  font-size: var(--text-sm);
}

.protectedRoute__spinner {
  width: 2.75rem;
  height: 2.75rem;
  border: 3px solid var(--surface-container-highest);
  border-top-color: var(--primary);
  border-radius: var(--radius-full);
  animation: protectedRouteSpin 0.85s linear infinite;
}

@keyframes protectedRouteSpin {
  to {
    transform: rotate(360deg);
  }
}

```

COMPONENT · SHELL frontend/src/components/Layout/Layout.jsx

Main App Shell Layout

Responsive master shell uniting the Sidebar, Header/Navbar, and main page content.

Lines: 90 Size: 3.5 KB Format: .JSX

```jsx
import { Outlet, useLocation } from 'react-router-dom';
import { useAuth } from '../../contexts/AuthContext';
import Navbar from '../Navbar/Navbar.jsx';
import Sidebar from '../Sidebar/Sidebar.jsx';
import styles from './Layout.module.css';

/**
 * Authenticated shell: fixed sidebar + scrollable main column + footer.
 * Add new `pathname` branches below when you introduce more protected routes.
 */
export default function Layout() {
  const location = useLocation();
  const { user, logout } = useAuth();

  /** Navbar title: keep in sync with routes in `App.jsx`. */
  const getTitle = () => {
    const path = location.pathname;
    if (path === '/dashboard') return 'Home';
    if (path === '/my-questions') return 'Your topics';
    if (path === '/questions/ask') return 'Ask a question';
    if (path.startsWith('/question/') || path.startsWith('/questions/')) return 'Discussion';
    if (path === '/rag-documents') return 'Knowledge base';
    return 'Forum';
  };

  /** One-line context under the title (helps students orient on each screen). */
  const getSubtitle = () => {
    const path = location.pathname;
    if (path === '/dashboard')
      return 'Browse the feed, search by keyword, or run AI similarity search.';
    if (path === '/my-questions')
      return 'Questions you have posted. Open any thread to read replies or edit context.';
    if (path === '/questions/ask')
      return 'A clear title and reproducible steps get faster, more accurate answers.';
    if (path.startsWith('/question/') || path.startsWith('/questions/'))
      return 'Read the thread, review related topics, and reply with markdown if you can help.';
    if (path === '/rag-documents')
      return 'Private PDF library: reader, semantic search, and AI answers with citations per document.';
    return '';
  };

  return (
    <div className={styles.layout}>
      <Sidebar />
      <div className={styles.layout__content}>
        <Navbar
          title={getTitle()}
          subtitle={getSubtitle()}
          user={user}
          onLogout={logout}
        />
        <main className={styles.layout__main}>
          <div className={styles.layout__mainInner}>
            <Outlet />
          </div>
        </main>

        <footer className={styles.layout__footer}>
          <div className={styles['layout__footer-content']}>
            <div className={styles['layout__footer-branding']}>
              <h4 className={styles['layout__footer-title']}>Evangadi Forum</h4>
```

```
              <p className={styles['layout__footer-tagline']}>
                A practice space for technical Q&A, peer feedback, and
                AI-assisted search, built for Evangadi learners and mentors.
              </p>
              <p className={styles['layout__footer-copyright']}>
                © 2026 Evangadi Forum. For educational use.
              </p>
            </div>
            <nav className={styles['layout__footer-nav']}>
              <a href='#' className={styles['layout__footer-link']}>
                About
              </a>
              <a href='#' className={styles['layout__footer-link']}>
                Privacy
              </a>
              <a href='#' className={styles['layout__footer-link']}>
                Terms
              </a>
              <a href='#' className={styles['layout__footer-link']}>
                Contact
              </a>
            </nav>
          </div>
        </footer>
      </div>
    </div>
  );
}

```

COMPONENT · CSS frontend/src/components/Layout/Layout.module.css

Layout Shell Styles

Layout grid, header positioning, and responsive container constraints.

Lines: 121 Size: 2.7 KB Format: .CSS

```
/**
 * Layout.module.css: authenticated app chrome (sidebar offset + main padding).
 */

.layout {
    min-height: 100vh;
    background-color: var(--background);
}

.layout__content {
    margin-left: var(--app-sidebar-width, 15rem);
    min-height: 100vh;
    display: flex;
    flex-direction: column;
}

.layout__main {
    flex: 1;
    padding: var(--spacing-lg) var(--spacing-xl) var(--spacing-2xl);
    max-width: 100%;
    display: flex;
    flex-direction: column;
}

/* Centers page content on very wide monitors while keeping the orange token theme. */
.layout__mainInner {
    width: 100%;
    max-width: var(--app-content-max, 80rem);
    margin-left: auto;
    margin-right: auto;
    flex: 1;
    display: flex;
    flex-direction: column;
}

.layout__footer {
    border-top: 1px solid var(--border);
    background-color: var(--surface);
    background-image: linear-gradient(
        180deg,
        var(--surface-container-low) 0%,
        var(--surface) 28%
    );
    box-shadow: inset 0 1px 0 rgba(255, 255, 255, 0.7);
    padding-top: var(--spacing-xl);
    padding-bottom: var(--spacing-xl);
}

.layout__footer-content {
    max-width: var(--app-content-max, 80rem);
    margin: 0 auto;
    padding-left: var(--spacing-xl);
    padding-right: var(--spacing-xl);
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: space-between;
    gap: var(--spacing-lg);
}

@media (min-width: 768px) {
```

```
    .layout__footer-content {
        flex-direction: row;
    }
}

.layout__footer-branding {
    text-align: center;
}

@media (min-width: 768px) {
    .layout__footer-branding {
        text-align: left;
    }
}

.layout__footer-title {
    font-weight: var(--font-semibold);
    color: var(--text-primary);
    margin-bottom: var(--spacing-xs);
}

.layout__footer-tagline {
    font-size: var(--text-sm);
    color: var(--text-secondary);
    line-height: 1.55;
    max-width: var(--app-form-max, 28rem);
    margin-bottom: var(--spacing-sm);
}

.layout__footer-copyright {
    font-size: var(--text-xs);
    color: var(--text-tertiary);
}

.layout__footer-nav {
    display: flex;
    gap: var(--spacing-lg);
}

.layout__footer-link {
    font-size: var(--text-sm);
    color: var(--text-secondary);
    text-decoration: none;
    transition: color var(--transition-base);
}

.layout__footer-link:hover {
    color: var(--primary);
}

.layout__footer-link:focus {
    outline: none;
}

.layout__footer-link:focus-visible {
    color: var(--primary);
    outline: 2px solid var(--primary);
    outline-offset: 3px;
    border-radius: var(--radius-sm);
}
```

COMPONENT · NAVBAR frontend/src/components/Navbar/Navbar.jsx

Navigation Header Bar

Header bar displaying current page title, user profile initials, and logout trigger.

Lines: 128 Size: 4.2 KB Format: .JSX

```jsx
import { useState, useEffect } from 'react';
import { useNavigate, useLocation } from 'react-router-dom';
import { Search, LogOut, Sparkles } from 'lucide-react';
import styles from './Navbar.module.css';

/**
 * Top bar: page title, debounced text search → `/dashboard?q=…`, optional AI semantic search.
 * Search state is driven by the URL on the dashboard so bookmarks and refresh keep context.
 */
export default function Navbar({ title, subtitle, user, onLogout }) {
  const navigate = useNavigate();
  const location = useLocation();

  // Initialize searchTerm from URL if we are already on the dashboard
  const [searchTerm, setSearchTerm] = useState(() => {
    const params = new URLSearchParams(location.search);
    return params.get('q') || params.get('semantic') || '';
  });

  // Keep input in sync with URL if it changes externally
  useEffect(() => {
    if (location.pathname === '/dashboard') {
      const params = new URLSearchParams(location.search);
      setSearchTerm(params.get('q') || params.get('semantic') || '');
    } else {
      setSearchTerm('');
    }
  }, [location.search, location.pathname]);

  // Debounced keyword search: updates `?q=` on the dashboard (500ms quiet period).
  useEffect(() => {
    const delayDebounceFn = setTimeout(() => {
      if (searchTerm.trim() !== '') {
        navigate(`/dashboard?q=${encodeURIComponent(searchTerm)}`);
      } else if (
        location.pathname === '/dashboard' &&
        !new URLSearchParams(location.search).get('semantic')
      ) {
        navigate('/dashboard');
      }
    }, 500);

    return () => clearTimeout(delayDebounceFn);
  }, [searchTerm, navigate, location.pathname]);

  const handleSemanticSearch = e => {
    e.preventDefault();
    if (searchTerm.trim().length >= 3) {
      navigate(`/dashboard?semantic=${encodeURIComponent(searchTerm)}`);
    }
  };

  const handleSearchSubmit = e => {
    e.preventDefault();
    if (searchTerm.trim()) {
      navigate(`/dashboard?q=${encodeURIComponent(searchTerm)}`);
    }
  };

  return (
    <header className={styles.navbar}>
```

```
      <div className={styles.navbar__titleBlock}>
        <h2 className={styles.navbar__pageTitle}>{title}</h2>
        {subtitle ? (
          <p className={styles.navbar__pageSubtitle}>{subtitle}</p>
        ) : null}
      </div>

      <form className={styles.navbar__search} onSubmit={handleSearchSubmit}>
        <div className={styles['navbar__search-icon']}>
          <Search size={16} />
        </div>
        <input
          id='search'
          type='text'
          value={searchTerm}
          onChange={e => setSearchTerm(e.target.value)}
          placeholder='Search questions by keyword…'
          className={styles['navbar__search-input']}
          aria-label='Search questions by keyword'
        />
        {searchTerm.length >= 3 && (
          <button
            type='button'
            onClick={handleSemanticSearch}
            className={styles['navbar__semantic-button']}
            title='Use AI Semantic Search'
          >
            <Sparkles size={14} />
            <span className={styles['navbar__semantic-text']}>AI Search</span>
          </button>
        )}
      </form>

      <div className={styles.navbar__actions}>
        <div className={styles.navbar__user}>
          <span className={styles['navbar__user-name']}>
            {user ? `${user.firstName} ${user.lastName}` : 'Guest'}
          </span>
          <div className={styles['navbar__user-avatar']}>
            <img
              src={
                user?.avatar ||
                `https://ui-avatars.com/api/?name=${
                  user?.firstName || 'User'
                }+${user?.lastName || ''}&background=random`
              }
              alt='avatar'
              referrerPolicy='no-referrer'
            />
          </div>
        </div>
        {user && (
          <button
            type='button'
            className={styles.navbar__logout}
            onClick={onLogout}
            aria-label='Logout'
            title='Logout'
          >
            <LogOut size={20} />
          </button>
        )}
      </div>
    </header>
  );
}
```

COMPONENT · CSS `frontend/src/components/Navbar/Navbar.module.css`

Navbar Styles

Header styles, user avatar badge, and action button alignment.

Lines: 212 Size: 4.2 KB Format: .CSS

```css
/**
 * Navbar.module.css: sticky header, search field, user menu.
 */

.navbar {
  position: sticky;
  top: 0;
  z-index: 40;
  background-color: var(--surface);
  padding-left: var(--spacing-xl);
  padding-right: var(--spacing-xl);
  min-height: 4.25rem;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: var(--spacing-xl);
  border-bottom: 1px solid var(--border);
  box-shadow: var(--shadow-sm);
}

.navbar__titleBlock {
  flex-shrink: 1;
  min-width: 0;
  max-width: min(32rem, 46vw);
}

.navbar__pageTitle {
  font-weight: var(--font-semibold);
  font-size: var(--text-lg);
  color: var(--text-primary);
  line-height: 1.25;
  margin: 0;
}

.navbar__pageSubtitle {
  margin: 0.125rem 0 0;
  font-size: var(--text-xs);
  font-weight: var(--font-normal);
  color: var(--text-tertiary);
  line-height: 1.4;
  display: none;
}

@media (min-width: 900px) {
  .navbar__pageSubtitle {
    display: block;
  }
}

.navbar__search {
  flex: 1;
  max-width: 36rem;
  position: relative;
}

.navbar__search-icon {
  position: absolute;
  top: 0;
  bottom: 0;
  top: 50%;
  transform: translateY(-50%);
```

```
  left: 0.75rem;
  display: flex;
  align-items: center;
  pointer-events: none;
  color: var(--text-secondary);
}

.navbar__search-input {
  width: 100%;
  background-color: var(--surface-container-low);
  border: 1px solid var(--border);
  border-radius: var(--radius-lg);
  padding-top: 0.5rem;
  padding-bottom: 0.5rem;
  padding-left: 2.5rem;
  padding-right: 7rem;
  font-size: var(--text-sm);
  transition: var(--transition-base);
  color: var(--text-primary);
}

.navbar__semantic-button {
  position: absolute;
  right: 0.25rem;
  top: 50%;
  transform: translateY(-50%);
  display: flex;
  align-items: center;
  gap: 0.25rem;
  padding: 0.375rem 0.75rem;
  background-color: var(--primary);
  color: var(--text-inverse);
  border: none;
  border-radius: calc(var(--radius-lg) - 0.25rem);
  font-size: var(--text-xs);
  font-weight: var(--font-medium);
  cursor: pointer;
  transition: background-color var(--transition-fast);
}

.navbar__semantic-button:hover {
  background-color: var(--primary-hover);
}

.navbar__semantic-button:focus {
  outline: none;
}

.navbar__semantic-button:focus-visible {
  outline: 2px solid var(--primary);
  outline-offset: 2px;
}

.navbar__semantic-text {
  display: none;
}

@media (min-width: 640px) {
  .navbar__semantic-text {
    display: inline;
  }
}

.navbar__search-input::placeholder {
  color: var(--text-tertiary);
}

.navbar__search-input:focus {
  /* Handled by global theme.css focus styles */
  background-color: var(--surface);
```

```
}

.navbar__actions {
  display: flex;
  align-items: center;
  gap: var(--spacing-md);
}

.navbar__notification {
  position: relative;
  padding: 0.5rem;
  color: var(--text-secondary);
  transition: var(--transition-base);
  background: none;
  border: none;
  cursor: pointer;
  border-radius: var(--radius-full);
}

.navbar__notification:hover {
  color: var(--primary);
  background-color: var(--primary-light);
}

.navbar__user {
  display: flex;
  align-items: center;
  gap: 0.75rem;
  padding-left: var(--spacing-md);
  border-left: 1px solid var(--border);
}

.navbar__user-name {
  font-size: var(--text-sm);
  font-weight: var(--font-medium);
  color: var(--text-primary);
  display: none;
}

@media (min-width: 1024px) {
  .navbar__user-name {
    display: block;
  }
}

.navbar__user-avatar {
  width: 2rem;
  height: 2rem;
  border-radius: var(--radius-full);
  background-color: var(--surface-container-high);
  overflow: hidden;
  border: 1px solid var(--border);
}

.navbar__user-avatar img {
  width: 100%;
  height: 100%;
  object-fit: cover;
}

.navbar__logout {
  position: relative;
  padding: 0.5rem;
  color: var(--text-secondary);
  transition: var(--transition-base);
  background: none;
  border: none;
  cursor: pointer;
  border-radius: var(--radius-full);
}
```

```
202
203 .navbar__logout:hover {
204   color: var(--error);
205   background-color: var(--error-light);
206 }
207
208 .navbar__logout:focus {
209   outline: none;
210   box-shadow: var(--shadow-focus);
211 }
212
```

COMPONENT · SIDEBAR frontend/src/components/Sidebar/Sidebar.jsx

App Navigation Sidebar

Sidebar with navigation links (Dashboard, Ask Question, My Questions, RAG Documents).

Lines: 125 Size: 4.1 KB Format: .JSX

```jsx
import { NavLink, useNavigate } from 'react-router-dom';
import { LayoutDashboard, LogOut, MessageSquare, FileText } from 'lucide-react';
import { useAuth } from '../../contexts/AuthContext';
import styles from './Sidebar.module.css';

/**
 * Primary navigation: paths must match `App.jsx` routes.
 * Add rows here when you ship new sections (e.g. Admin, Bookmarks).
 */
const NAV_ITEMS = [
  { icon: LayoutDashboard, label: 'Home', path: '/dashboard' },
  { icon: MessageSquare, label: 'Your Topics', path: '/my-questions' },
  { icon: FileText, label: 'Knowledge Base', path: '/rag-documents' },
];

export default function Sidebar() {
  const navigate = useNavigate();
  const { user, logout } = useAuth();

  return (
    <aside className={styles.sidebar}>
      <div className={styles.sidebar__header}>
        <div
          className={styles.sidebar__branding}
          onClick={() => navigate('/')}
          title='Go to Home'
          role='button'
          tabIndex={0}
          onKeyDown={e => {
            if (e.key === 'Enter' || e.key === ' ') {
              e.preventDefault();
              navigate('/');
            }
          }}
        >
          <div className={styles.sidebar__logo} aria-hidden>
            <MessageSquare className={styles['sidebar__logo-icon']} size={20} />
          </div>
          <div className={styles.sidebar__brandCopy}>
            <p className={styles.sidebar__title}>Evangadi Forum</p>
            <p className={styles.sidebar__tagline}>
              Learn together. Ask with context.
            </p>
          </div>
        </div>
      </div>

      <nav className={styles.sidebar__nav} aria-label='Main navigation'>
        <p className={styles.sidebar__navLabel}>Navigate</p>
        {NAV_ITEMS.map(item => (
          <div key={item.path} className={styles['sidebar__nav-item-wrapper']}>
            <NavLink
              to={item.path}
              className={({ isActive }) =>
                `${styles.sidebar__link} ${
                  isActive
                    ? styles['sidebar__link--active']
                    : styles['sidebar__link--inactive']
                }`
              }
            >
```

```
              {({ isActive }) => (
                <>
                  <item.icon
                    size={18}
                    className={`${styles.sidebar__icon} ${
                      isActive
                        ? styles['sidebar__icon--active']
                        : styles['sidebar__icon--inactive']
                    }`}
                  />
                  <span>{item.label}</span>
                </>
              )}
            </NavLink>
          </div>
        ))}
      </nav>

      <div className={styles.sidebar__footer}>
        <button
          type='button'
          onClick={() => navigate('/questions/ask')}
          className={styles.sidebar__button}
        >
          New Question
        </button>

        <div className={styles.sidebar__user}>
          <div className={styles.sidebar__profile}>
            <div className={styles.sidebar__avatar}>
              <img
                src={
                  user?.avatar ||
                  `https://ui-avatars.com/api/?name=${
                    user?.firstName || 'User'
                  }+${user?.lastName || ''}&background=random`
                }
                alt={`${user?.firstName} ${user?.lastName}`}
                className={styles['sidebar__avatar-image']}
                referrerPolicy='no-referrer'
              />
            </div>
            <div className={styles.sidebar__info}>
              <p className={styles.sidebar__name}>
                {user?.firstName} {user?.lastName}
              </p>
              <p className={styles.sidebar__role}>Learner</p>
            </div>
          </div>

          <button
            type='button'
            onClick={logout}
            className={styles.sidebar__logout}
          >
            <LogOut size={16} />
            <span>Logout</span>
          </button>
        </div>
      </div>
    </aside>
  );
}

```

COMPONENT · CSS frontend/src/components/Sidebar/Sidebar.module.css

Sidebar Styles

Collapsible sidebar styling, active item indicators, and mobile drawer styles.

Lines: 346 Size: 8.3 KB Format: .CSS

```css
/* Sidebar Component Styles - BEM Naming Convention */

/* Block: sidebar */
.sidebar {
    height: 100vh;
    width: var(--app-sidebar-width, 15rem);
    position: fixed;
    left: 0;
    top: 0;
    overflow-y: auto;
    background-color: var(--surface);
    display: flex;
    flex-direction: column;
    padding-top: var(--spacing-lg);
    padding-bottom: var(--spacing-lg);
    z-index: 50;
    border-right: 1px solid var(--border);
    box-shadow: 1px 0 0 rgba(15, 23, 42, 0.04);
}

/* Element: sidebar__header (brand + divider before nav) */
.sidebar__header {
    padding-left: var(--spacing-lg);
    padding-right: var(--spacing-lg);
    padding-bottom: var(--spacing-md);
    margin-bottom: var(--spacing-md);
    border-bottom: 1px solid var(--border);
}

/* Element: sidebar__branding */
.sidebar__branding {
    display: flex;
    align-items: flex-start;
    gap: var(--spacing-md);
    cursor: pointer;
    border-radius: var(--radius-md);
    padding: var(--spacing-sm);
    margin: calc(-1 * var(--spacing-sm));
    transition: background-color var(--transition-fast);
}

.sidebar__branding:hover {
    background-color: var(--surface-hover);
}

.sidebar__branding:focus {
    outline: none;
}

.sidebar__branding:focus-visible {
    outline: 2px solid var(--primary);
    outline-offset: 2px;
}

/* Stacked title + tagline beside logo */
.sidebar__brandCopy {
    min-width: 0;
    display: flex;
    flex-direction: column;
    gap: 0.1875rem;
    padding-top: 0.0625rem;
```

```
}

/* Element: sidebar__logo */
.sidebar__logo {
    width: 2.375rem;
    height: 2.375rem;
    flex-shrink: 0;
    background: linear-gradient(145deg, var(--primary) 0%, var(--primary-hover) 100%);
    border-radius: var(--radius-md);
    display: flex;
    align-items: center;
    justify-content: center;
    box-shadow: 0 1px 2px rgba(15, 23, 42, 0.08);
}

/* Element: sidebar__logo-icon */
.sidebar__logo-icon {
    color: var(--text-inverse);
    fill: currentColor;
}

/* Element: sidebar__title */
.sidebar__title {
    margin: 0;
    font-weight: var(--font-semibold);
    font-size: var(--text-base);
    line-height: 1.25;
    color: var(--tertiary);
    letter-spacing: -0.02em;
}

.sidebar__tagline {
    margin: 0;
    font-size: 0.6875rem;
    line-height: 1.4;
    color: var(--text-tertiary);
    font-weight: var(--font-normal);
}

.sidebar__navLabel {
    padding-left: var(--spacing-md);
    padding-right: var(--spacing-md);
    margin: 0 0 var(--spacing-sm);
    font-size: 0.625rem;
    font-weight: var(--font-semibold);
    letter-spacing: 0.1em;
    text-transform: uppercase;
    color: var(--text-placeholder);
}

/* Element: sidebar__nav */
.sidebar__nav {
    flex: 1;
    display: flex;
    flex-direction: column;
    gap: 0.125rem;
    padding-top: var(--spacing-xs);
}

/* Element: sidebar__nav-item-wrapper */
.sidebar__nav-item-wrapper {
    padding-left: var(--spacing-md);
    padding-right: var(--spacing-md);
    padding-top: 0.125rem;
    padding-bottom: 0.125rem;
}

/* Element: sidebar__link */
.sidebar__link {
    position: relative;
```

```
    display: flex;
    align-items: center;
    gap: var(--spacing-sm);
    padding-left: var(--spacing-md);
    padding-right: var(--spacing-sm);
    padding-top: 0.5625rem;
    padding-bottom: 0.5625rem;
    transition: background-color var(--transition-base), color var(--transition-base);
    font-size: var(--text-sm);
    font-weight: var(--font-medium);
    border-radius: var(--radius-md);
    text-decoration: none;
}

/* Modifier: sidebar__link--active (left rail + light surface, not full pill) */
.sidebar__link--active {
    background-color: var(--surface-container-low);
    color: var(--text-primary);
    font-weight: var(--font-semibold);
}

.sidebar__link--active::before {
    content: '';
    position: absolute;
    left: 0.25rem;
    top: 50%;
    transform: translateY(-50%);
    width: 3px;
    height: 1.375rem;
    border-radius: 2px;
    background-color: var(--primary);
}

/* Modifier: sidebar__link--inactive */
.sidebar__link--inactive {
    color: var(--text-secondary);
}

/* Pseudo-class: hover state for inactive links */
.sidebar__link--inactive:hover {
    background-color: var(--surface-hover);
    color: var(--text-primary);
}

/* Element: sidebar__icon */
.sidebar__icon {
    transition: color var(--transition-base);
}

/* Modifier: sidebar__icon--active */
.sidebar__icon--active {
    color: var(--primary);
}

/* Modifier: sidebar__icon--inactive */
.sidebar__icon--inactive {
    color: var(--text-secondary);
}

/* Pseudo-class: hover state for inactive icons (via group-hover) */
.sidebar__link--inactive:hover .sidebar__icon--inactive {
    color: var(--primary);
}

/* Element: sidebar__footer */
.sidebar__footer {
    padding-left: var(--spacing-lg);
    padding-right: var(--spacing-lg);
    margin-top: auto;
    display: flex;
```

```
    flex-direction: column;
    gap: var(--spacing-md);
}

/* Element: sidebar__button */
.sidebar__button {
    width: 100%;
    background-color: var(--primary);
    color: var(--text-inverse);
    padding-top: 0.625rem;
    padding-bottom: 0.625rem;
    border-radius: var(--radius-md);
    font-weight: var(--font-medium);
    font-size: var(--text-sm);
    transition: all var(--transition-base);
    box-shadow: var(--shadow-sm);
    border: none;
    cursor: pointer;
}

/* Pseudo-class: hover state for button */
.sidebar__button:hover {
    background-color: var(--primary-hover);
}

/* Pseudo-class: active state for button */
.sidebar__button:active {
    transform: scale(0.95);
}

/* Element: sidebar__user */
.sidebar__user {
    padding-top: var(--spacing-lg);
    border-top: 1px solid var(--border);
}

/* Element: sidebar__profile */
.sidebar__profile {
    display: flex;
    align-items: center;
    gap: 0.75rem;
    /* 3 * 0.25rem = 0.75rem */
    margin-bottom: 1rem;
    /* 4 * 0.25rem = 1rem */
}

/* Element: sidebar__avatar */
.sidebar__avatar {
    width: 2rem;
    /* 8 * 0.25rem = 2rem */
    height: 2rem;
    border-radius: 9999px;
    /* rounded-full */
    background-color: var(--surface-container-highest);
    border: 1px solid var(--surface-container-highest);
    flex-shrink: 0;
    overflow: hidden;
}

/* Element: sidebar__avatar-image */
.sidebar__avatar-image {
    width: 100%;
    height: 100%;
    object-fit: cover;
}

/* Element: sidebar__info */
.sidebar__info {
    overflow: hidden;
}
```

```
/* Element: sidebar__name */
.sidebar__name {
    font-size: 0.75rem;
    /* text-xs */
    font-weight: var(--font-semibold, 600);
    color: var(--tertiary);
    overflow: hidden;
    text-overflow: ellipsis;
    white-space: nowrap;
}

/* Element: sidebar__role */
.sidebar__role {
    font-size: 0.625rem;
    /* text-[10px] */
    color: var(--secondary);
    overflow: hidden;
    text-overflow: ellipsis;
    white-space: nowrap;
}

/* Element: sidebar__settings */
.sidebar__settings {
    padding-left: 0.5rem;
    /* 2 * 0.25rem = 0.5rem */
    padding-right: 0.5rem;
    padding-top: 0.5rem;
    padding-bottom: 0.5rem;
    display: flex;
    align-items: center;
    gap: 0.75rem;
    /* 3 * 0.25rem = 0.75rem */
    color: var(--secondary);
    transition: all var(--transition-base, 200ms ease-in-out);
    font-size: 0.75rem;
    /* text-xs */
    font-weight: var(--font-medium, 500);
    text-decoration: none;
}

/* Pseudo-class: hover state for settings link */
.sidebar__settings:hover {
    color: var(--primary);
}

/* Element: sidebar__logout */
.sidebar__logout {
    width: 100%;
    padding-left: 0.5rem;
    /* 2 * 0.25rem = 0.5rem */
    padding-right: 0.5rem;
    padding-top: 0.5rem;
    padding-bottom: 0.5rem;
    display: flex;
    align-items: center;
    gap: 0.75rem;
    /* 3 * 0.25rem = 0.75rem */
    color: rgba(239, 68, 68, 0.8);
    /* text-red-500/80 */
    transition: all var(--transition-base, 200ms ease-in-out);
    font-size: 0.75rem;
    /* text-xs */
    font-weight: var(--font-medium, 500);
    background: none;
    border: none;
    cursor: pointer;
    text-align: left;
}

```

```
/* Pseudo-class: hover state for logout button */
.sidebar__logout:hover {
    color: rgb(239, 68, 68);
    /* text-red-500 */
}
```

COMPONENT · CARD frontend/src/components/QuestionCard/QuestionCard.jsx

Question Card Item

Renders individual questions in lists with author, creation time, answer count, and click routing.

Lines: 77 Size: 2.7 KB Format: .JSX

```jsx
import { Link } from 'react-router-dom';
import { MessageSquare } from 'lucide-react';
import { timeAgo, getInitials, getAvatarColor, isAuthoredByUser } from '../../lib/utils';
import styles from './QuestionCard.module.css';

/**
 * Strips markdown markup to create a clean preview snippet.
 */
function cleanSnippet(text, maxLength = 160) {
  if (!text) return '';
  const cleaned = text
    .replace(/#+\s/g, '')
    .replace(/```[\s\S]*?```/g, ' [code] ')
    .replace(/`([^`]+)`/g, '$1')
    .replace(/\[([^\]]+)\]\([^)]+\)/g, '$1')
    .replace(/[*_~]/g, '')
    .replace(/\s+/g, ' ')
    .trim();
  return cleaned.length > maxLength ? cleaned.slice(0, maxLength) + '...' : cleaned;
}

export default function QuestionCard({ question, currentUser }) {
  if (!question) return null;

  const isOwner = isAuthoredByUser(question, currentUser);
  const hash = question.questionHash || question.id;
  const authorFirst = question.author?.firstName || question.firstName || 'Anonymous';
  const authorLast = question.author?.lastName || question.lastName || '';
  const initials = getInitials(authorFirst, authorLast);
  const avatarTheme = getAvatarColor(authorFirst + authorLast + (question.author?.id || ''));
  const answerCount = question.answerCount ?? question.answers?.length ?? 0;
  const relativeDate = timeAgo(question.createdAt);
  const byLabel = isOwner ? 'You' : `${authorFirst} ${authorLast}`.trim();

  return (
    <article
      className={`${styles.card} ${isOwner ? styles['card--yours'] : ''}`}
      aria-label={question.title}
    >
      <div
        className={styles.avatar}
        style={{ backgroundColor: avatarTheme.bg, color: avatarTheme.color }}
        aria-hidden
      >
        {initials}
      </div>

      <div className={styles.body}>
        <div className={styles.headerRow}>
          <h3 className={styles.title}>
            <Link to={`/questions/${hash}`} className={styles.titleLink}>
              {question.title}
            </Link>
          </h3>
          {isOwner && <span className={styles.yoursBadge}>YOURS</span>}
        </div>

        <p className={styles.excerpt}>
          {cleanSnippet(question.content)}
        </p>

```

```
        <div className={styles.footer}>
          <span className={styles.replyCount}>
            <MessageSquare size={14} className={styles.replyIcon} aria-hidden />
            {answerCount} {answerCount === 1 ? 'reply' : 'replies'}
          </span>
          <span className={styles.metaDot}>•</span>
          <span className={styles.authorMeta}>
            {relativeDate ? `${relativeDate} by ` : 'By '}
            <strong className={styles.authorName}>{byLabel}</strong>
          </span>
        </div>
      </div>
    </article>
  );
}

```

COMPONENT · CSS frontend/src/components/QuestionCard/QuestionCard.module.css

Question Card Styles

Card hover elevation, typography, and meta tag layout.

Lines: 116 Size: 2.1 KB Format: .CSS

```css
.card {
  display: flex;
  align-items: flex-start;
  gap: var(--spacing-md, 1rem);
  padding: 1.25rem 1.5rem;
  background-color: var(--surface, #ffffff);
  border: 1px solid var(--border, #e2e8f0);
  border-radius: var(--radius-lg, 0.75rem);
  transition: border-color 0.15s ease, box-shadow 0.15s ease, transform 0.15s ease;
  position: relative;
}

.card:hover {
  border-color: var(--border-hover, #cbd5e1);
  box-shadow: var(--shadow-sm, 0 1px 2px rgba(0, 0, 0, 0.05));
}

.card--yours {
  border-left: 3px solid var(--primary, #f97316);
}

.avatar {
  width: 2.5rem;
  height: 2.5rem;
  border-radius: var(--radius-full, 9999px);
  display: flex;
  align-items: center;
  justify-content: center;
  font-weight: 600;
  font-size: 0.875rem;
  flex-shrink: 0;
  letter-spacing: -0.02em;
}

.body {
  flex: 1;
  min-width: 0;
}

.headerRow {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 0.75rem;
  margin-bottom: 0.375rem;
}

.title {
  font-size: 1rem;
  font-weight: 600;
  line-height: 1.4;
  margin: 0;
  color: var(--text-primary, #0f172a);
}

.titleLink {
  color: inherit;
  text-decoration: none;
  transition: color 0.15s ease;
}

```

```
.titleLink:hover {
  color: var(--primary, #f97316);
}

.yoursBadge {
  font-size: 0.6875rem;
  font-weight: 700;
  padding: 0.15rem 0.5rem;
  background-color: var(--primary-light, #ffedd5);
  color: var(--primary-dark, #c2410c);
  border-radius: var(--radius-sm, 0.25rem);
  letter-spacing: 0.04em;
  flex-shrink: 0;
}

.excerpt {
  font-size: 0.875rem;
  color: var(--text-secondary, #475569);
  line-height: 1.5;
  margin: 0 0 0.75rem 0;
  word-break: break-word;
}

.footer {
  display: flex;
  align-items: center;
  gap: 0.5rem;
  font-size: 0.8125rem;
  color: var(--text-tertiary, #64748b);
}

.replyCount {
  display: inline-flex;
  align-items: center;
  gap: 0.35rem;
  font-weight: 500;
}

.replyIcon {
  color: var(--text-placeholder, #94a3b8);
}

.metaDot {
  color: var(--border-hover, #cbd5e1);
}

.authorMeta {
  color: var(--text-tertiary, #64748b);
}

.authorName {
  font-weight: 500;
  color: var(--text-secondary, #334155);
}

```

COMPONENT · RAG UI frontend/src/components/RagAnswerBody/RagAnswerBody.jsx

RAG Answer & Citation Display

Renders Gemini AI response to PDF questions alongside interactive page citations.

Lines: 126 Size: 3.7 KB Format: .JSX

```jsx
/**
 * Renders RAG "answer" text as Markdown (incl. fenced code) with readable styling.
 */
import { useCallback, useRef, useState } from 'react';
import ReactMarkdown from 'react-markdown';
import { Copy, Check } from 'lucide-react';
import styles from './RagAnswerBody.module.css';

/**
 * @param {{ children: string }} props
 */
export default function RagAnswerBody({ children }) {
  const text = typeof children === 'string' ? children : '';
  if (!text.trim()) return null;

  return (
    <div className={styles.root}>
      <ReactMarkdown
        components={{
          p: ({ node: _n, ...props }) => <p className={styles.p} {...props} />,
          ul: ({ node: _n, ...props }) => (
            <ul className={styles.ul} {...props} />
          ),
          ol: ({ node: _n, ...props }) => (
            <ol className={styles.ol} {...props} />
          ),
          li: ({ node: _n, ...props }) => (
            <li className={styles.li} {...props} />
          ),
          h2: ({ node: _n, ...props }) => (
            <h2 className={styles.h2} {...props} />
          ),
          h3: ({ node: _n, ...props }) => (
            <h3 className={styles.h3} {...props} />
          ),
          blockquote: ({ node: _n, ...props }) => (
            <blockquote className={styles.blockquote} {...props} />
          ),
          a: ({ node: _n, ...props }) => (
            <a
              className={styles.a}
              target='_blank'
              rel='noreferrer noopener'
              {...props}
            />
          ),
          hr: ({ node: _n, ...props }) => (
            <hr className={styles.hr} {...props} />
          ),
          pre: ({ node: _n, children: preChildren }) => (
            <CodeBlock>{preChildren}</CodeBlock>
          ),
          code: ({ node: _n, className, children, ...props }) => {
            const isFence = Boolean(className?.trim());
            if (isFence) {
              return (
                <code className={className} {...props}>
                  {children}
                </code>
              );
            }
```

```
            return (
              <code className={styles.inlineCode} {...props}>
                {children}
              </code>
            );
          },
        }}
      >
        {text}
      </ReactMarkdown>
    </div>
  );
}

function CodeBlock({ children }) {
  const preRef = useRef(null);
  const [copied, setCopied] = useState(false);

  const handleCopy = useCallback(async () => {
    const raw = preRef.current?.textContent ?? '';
    try {
      await navigator.clipboard.writeText(raw);
      setCopied(true);
      window.setTimeout(() => setCopied(false), 2000);
    } catch {
      setCopied(false);
    }
  }, []);

  const codeChild = Array.isArray(children) ? children[0] : children;
  const cls = codeChild?.props?.className;
  const langMatch =
    typeof cls === 'string' ? cls.match(/language-([\w+#.-]+)/) : null;
  const langLabel = langMatch ? langMatch[1] : 'code';

  return (
    <div className={styles.codeWrap}>
      <div className={styles.codeToolbar}>
        <span className={styles.codeLang}>{langLabel}</span>
        <button
          type='button'
          className={styles.copyBtn}
          onClick={handleCopy}
          aria-label={copied ? 'Copied to clipboard' : 'Copy code to clipboard'}
        >
          {copied ? (
            <>
              <Check size={14} aria-hidden />
              Copied
            </>
          ) : (
            <>
              <Copy size={14} aria-hidden />
              Copy
            </>
          )}
        </button>
      </div>
      <pre ref={preRef} className={styles.pre}>
        {children}
      </pre>
    </div>
  );
}

```

COMPONENT · CSS frontend/src/components/RagAnswerBody/RagAnswerBody.module.css

RAG Answer Styles

Styles for citation badges, AI answer highlight boxes, and page jump links.

Lines: 157 Size: 3.3 KB Format: .CSS

```css
/**
 * Markdown + code blocks for RAG answers (Evangadi tokens).
 */

.root {
  font-size: var(--text-sm);
  line-height: 1.65;
  color: var(--text-primary);
}

.root > *:first-child {
  margin-top: 0;
}

.root > *:last-child {
  margin-bottom: 0;
}

.p {
  margin: 0 0 var(--spacing-md);
}

.ul,
.ol {
  margin: 0 0 var(--spacing-md);
  padding-left: 1.25rem;
}

.li {
  margin-bottom: var(--spacing-xs);
}

.h2 {
  margin: var(--spacing-lg) 0 var(--spacing-sm);
  font-size: var(--text-base);
  font-weight: var(--font-semibold);
  color: var(--text-primary);
}

.h3 {
  margin: var(--spacing-md) 0 var(--spacing-xs);
  font-size: var(--text-sm);
  font-weight: var(--font-semibold);
  color: var(--text-primary);
}

.blockquote {
  margin: 0 0 var(--spacing-md);
  padding: var(--spacing-sm) var(--spacing-md);
  border-left: 3px solid var(--primary);
  background-color: var(--surface-container-low);
  border-radius: 0 var(--radius-md) var(--radius-md) 0;
  color: var(--text-secondary);
}

.a {
  color: var(--primary-dark);
  font-weight: var(--font-medium);
  text-decoration: underline;
  text-underline-offset: 2px;
}
```

```
62
63  .a:hover {
64    color: var(--primary-hover);
65  }
66
67  .hr {
68    margin: var(--spacing-lg) 0;
69    border: none;
70    border-top: 1px solid var(--border);
71  }
72
73  .inlineCode {
74    padding: 0.125rem 0.35rem;
75    border-radius: var(--radius-sm);
76    font-size: 0.8125em;
77    font-family: ui-monospace, SFMono-Regular, 'SF Mono', Menlo, Consolas,
78      monospace;
79    background-color: var(--surface-container-high);
80    color: var(--tertiary);
81  }
82
83  .codeWrap {
84    margin: var(--spacing-md) 0;
85    border-radius: var(--radius-lg);
86    border: 1px solid var(--border);
87    overflow: hidden;
88    background-color: var(--tertiary);
89    box-shadow: var(--shadow-md);
90  }
91
92  .codeToolbar {
93    display: flex;
94    align-items: center;
95    justify-content: space-between;
96    gap: var(--spacing-sm);
97    padding: var(--spacing-xs) var(--spacing-sm);
98    background-color: rgba(0, 0, 0, 0.2);
99    border-bottom: 1px solid rgba(255, 255, 255, 0.08);
100 }
101
102 .codeLang {
103   font-size: 0.625rem;
104   font-weight: var(--font-semibold);
105   letter-spacing: 0.06em;
106   text-transform: uppercase;
107   color: rgba(255, 255, 255, 0.55);
108   font-family: ui-monospace, SFMono-Regular, Menlo, Consolas, monospace;
109 }
110
111 .copyBtn {
112   display: inline-flex;
113   align-items: center;
114   gap: 0.25rem;
115   padding: 0.25rem 0.5rem;
116   font-size: var(--text-xs);
117   font-weight: var(--font-medium);
118   font-family: inherit;
119   color: rgba(255, 255, 255, 0.85);
120   background: rgba(255, 255, 255, 0.1);
121   border: 1px solid rgba(255, 255, 255, 0.15);
122   border-radius: var(--radius-sm);
123   cursor: pointer;
124   transition:
125     background-color var(--transition-fast),
126     color var(--transition-fast);
127 }
128
129 .copyBtn:hover {
130   background: rgba(255, 255, 255, 0.18);
131   color: #fff;
```

```
}

.copyBtn:focus-visible {
  outline: 2px solid var(--primary);
  outline-offset: 2px;
}

.pre {
  margin: 0;
  padding: var(--spacing-md);
  overflow-x: auto;
  max-height: min(24rem, 55vh);
  overflow-y: auto;
}

.pre code {
  display: block;
  font-family: ui-monospace, SFMono-Regular, 'SF Mono', Menlo, Consolas,
    monospace;
  font-size: var(--text-xs);
  line-height: 1.55;
  color: #e2e8f0;
  white-space: pre;
  background: transparent !important;
}

```

SECTION 010

10. FRONTEND PAGES & CSS MODULES

All public and authenticated pages: Landing, Login/Register, Feed, Question Ask, Detail, and RAG Workspace.

PAGE · LANDING frontend/src/pages/Landing/Landing.jsx

Public Landing Page

Hero introduction, feature highlights, and call-to-actions for the study forum.

Lines: 509 Size: 20.5 KB Format: .JSX

```jsx
/**
 * @file Landing.jsx
 * @description Public marketing route (`/`). Layout and copy align with in-app
 *   shell tokens (cards, borders, slate + orange). No data fetching.
 */
import { motion as Motion } from 'framer-motion';
import { useNavigate } from 'react-router-dom';
import {
  Sparkles,
  MessageSquare,
  Search,
  PenSquare,
  Library,
  ArrowRight,
  CheckCircle2,
  Layers,
  FileText,
  Database,
} from 'lucide-react';
import { useAuth } from '../../contexts/AuthContext';
import styles from './Landing.module.css';

export default function Landing() {
  const navigate = useNavigate();
  const { isAuthenticated } = useAuth();

  const scrollToHowItWorks = () => {
    document
      .getElementById('how-it-works')
      ?.scrollIntoView({ behavior: 'smooth' });
  };

  const scrollToCourseRag = () => {
    document
      .getElementById('course-rag')
      ?.scrollIntoView({ behavior: 'smooth' });
  };

  return (
    <div className={styles.landing}>
      <header className={styles.landing__header}>
        <div className={styles.landing__headerInner}>
          <button
            type='button'
            className={styles.landing__brand}
            onClick={() => navigate('/')}
            aria-label='Evangadi Forum home'
          >
            <span className={styles.landing__brandMark} aria-hidden>
              <MessageSquare size={20} strokeWidth={2} />
            </span>
            <span className={styles.landing__brandText}>
              <span className={styles.landing__brandName}>Evangadi Forum</span>
              <span className={styles.landing__brandLine}>
                Learn together. Ask with context.
              </span>
            </span>
          </button>

          <nav className={styles.landing__nav} aria-label='Marketing'>
            <button
```

```
              type='button'
              className={styles.landing__navLink}
              onClick={() => window.scrollTo({ top: 0, behavior: 'smooth' })}
            >
              Overview
            </button>
            <button
              type='button'
              className={styles.landing__navLink}
              onClick={scrollToCourseRag}
            >
              Course RAG
            </button>

            <button
              type='button'
              className={styles.landing__navLink}
              onClick={scrollToHowItWorks}
            >
              How it works
            </button>
          </nav>

          <div className={styles.landing__headerActions}>
            {isAuthenticated ? (
              <button
                type='button'
                className={styles.landing__btnPrimary}
                onClick={() => navigate('/dashboard')}
              >
                Open forum
                <ArrowRight size={16} aria-hidden />
              </button>
            ) : (
              <>
                <button
                  type='button'
                  className={styles.landing__btnGhost}
                  onClick={() => navigate('/auth')}
                >
                  Sign in
                </button>
                <button
                  type='button'
                  className={styles.landing__btnPrimary}
                  onClick={() => navigate('/auth')}
                >
                  Create account
                </button>
              </>
            )}
          </div>
        </div>
      </header>

      <main className={styles.landing__main}>
        <section className={styles.landing__hero}>
          <div className={styles.landing__heroInner}>
            <div className={styles.landing__heroCopy}>
              <Motion.p
                className={styles.landing__eyebrow}
                initial={{ opacity: 0, y: 8 }}
                animate={{ opacity: 1, y: 0 }}
              >
                <Sparkles size={14} aria-hidden />
                Keyword search + embedding similarity
              </Motion.p>
              <Motion.h1
                className={styles.landing__title}
                initial={{ opacity: 0, y: 12 }}
```

```
                animate={{ opacity: 1, y: 0 }}
                transition={{ delay: 0.05 }}
              >
                A calm place for{' '}
                <span className={styles.landing__titleAccent}>
                  technical Q&A
                </span>
              </Motion.h1>
              <Motion.p
                className={styles.landing__lead}
                initial={{ opacity: 0, y: 12 }}
                animate={{ opacity: 1, y: 0 }}
                transition={{ delay: 0.1 }}
              >
                Post with enough context for peers to help in one pass. Search
                the archive by phrase or by meaning, keep your threads in one
                place, and ground questions in{' '}
                <strong className={styles.landing__leadStrong}>
                  course documents
                </strong>{' '}
                with retrieval-augmented generation (RAG) so answers cite the
                right syllabus, readings, and handouts.
              </Motion.p>
              <Motion.div
                className={styles.landing__heroCtas}
                initial={{ opacity: 0, y: 12 }}
                animate={{ opacity: 1, y: 0 }}
                transition={{ delay: 0.15 }}
              >
                <button
                  type='button'
                  className={styles.landing__btnPrimary}
                  onClick={() =>
                    navigate(isAuthenticated ? '/dashboard' : '/auth')
                  }
                >
                  {isAuthenticated ? 'Go to home' : 'Get started'}
                  <ArrowRight size={16} aria-hidden />
                </button>
                {!isAuthenticated && (
                  <button
                    type='button'
                    className={styles.landing__btnOutline}
                    onClick={scrollToHowItWorks}
                  >
                    See how it works
                  </button>
                )}
              </Motion.div>
            </div>

            <aside
              className={styles.landing__heroPanel}
              aria-label='What you get'
            >
              <p className={styles.landing__heroPanelLabel}>At a glance</p>
              <ul className={styles.landing__heroPanelList}>
                <li>
                  <CheckCircle2 size={16} aria-hidden />
                  Markdown threads and replies
                </li>
                <li>
                  <CheckCircle2 size={16} aria-hidden />
                  Semantic search on question embeddings
                </li>
                <li>
                  <CheckCircle2 size={16} aria-hidden />
                  Optional AI draft tips when you ask or answer
                </li>
                <li>
```

```
                  <CheckCircle2 size={16} aria-hidden />
                  <span>
                    <strong className={styles.landing__heroPanelStrong}>
                      Course RAG:
                    </strong>{' '}
                    upload or sync course materials, retrieve the best chunks
                    for each question, and answer with citations, not generic
                    web text.
                  </span>
                </li>
              </ul>
            </aside>
          </div>
        </section>

        <section
          className={styles.landing__rag}
          id='course-rag'
          aria-labelledby='rag-heading'
        >
          <div className={styles.landing__sectionInner}>
            <p className={styles.landing__ragEyebrow}>
              Retrieval-augmented generation
            </p>
            <h2 className={styles.landing__sectionTitle} id='rag-heading'>
              How course RAG works with the forum
            </h2>
            <p className={styles.landing__sectionLead}>
              Forum search already helps you find <em>similar questions</em>{' '}
              from peers. RAG goes further: it finds{' '}
              <em>evidence inside your own documents</em> (readings, rubrics,
              lab specs) and surfaces those snippets when you write or review an
              answer. That keeps AI assistance on-policy for Evangadi-style
              courses and reduces “confident but wrong” generic answers.
            </p>
            <div className={styles.landing__ragPipeline}>
              <div className={styles.landing__ragStep}>
                <span className={styles.landing__ragStepIcon} aria-hidden>
                  <FileText size={20} />
                </span>
                <h3 className={styles.landing__ragStepTitle}>Ingest & chunk</h3>
                <p className={styles.landing__ragStepText}>
                  Upload or connect course files; split them into overlapping
                  chunks and store embeddings the same way we already embed
                  questions, so retrieval stays fast and auditable.
                </p>
              </div>
              <div className={styles.landing__ragStep}>
                <span className={styles.landing__ragStepIcon} aria-hidden>
                  <Database size={20} />
                </span>
                <h3 className={styles.landing__ragStepTitle}>
                  Retrieve at question time
                </h3>
                <p className={styles.landing__ragStepText}>
                  When you open Ask or run a search, the app pulls the
                  top-matching chunks from the cohort corpus (with scores), not
                  just other threads. That is ideal for “what does the syllabus
                  say about…” style questions.
                </p>
              </div>
              <div className={styles.landing__ragStep}>
                <span className={styles.landing__ragStepIcon} aria-hidden>
                  <Sparkles size={20} />
                </span>
                <h3 className={styles.landing__ragStepTitle}>
                  Grounded responses
                </h3>
                <p className={styles.landing__ragStepText}>
                  Downstream prompts quote or summarize only from retrieved
```

```
                spans, with room for instructors to review sources. The UI
                makes it obvious when an answer drew on RAG versus peer
                replies alone.
              </p>
            </div>
          </div>
          <p className={styles.landing__ragFootnote}>
            Live forum threads, semantic question search, draft/fit AI
            helpers, and this RAG pipeline work together: uploads and access
            control live in the Knowledge base per cohort, and RAG-backed
            context shows up in the same thread view you already use.
          </p>
        </div>
      </section>

      {!isAuthenticated && (
        <>
          <section className={styles.landing__capabilities}>
            <div className={styles.landing__sectionInner}>
              <h2 className={styles.landing__sectionTitle}>
                Built for cohort coursework
              </h2>
              <p className={styles.landing__sectionLead}>
                Same patterns you use after sign-in, without a separate
                “marketing product.”
              </p>
              <div className={styles.landing__cardGrid}>
                <article className={styles.landing__card}>
                  <div className={styles.landing__cardIcon} aria-hidden>
                    <Search size={22} strokeWidth={1.75} />
                  </div>
                  <h3 className={styles.landing__cardTitle}>
                    Find related work
                  </h3>
                  <p className={styles.landing__cardBody}>
                    Keyword filters for exact matches, plus similarity search
                    when you are still shaping the right vocabulary.
                  </p>
                </article>
                <article className={styles.landing__card}>
                  <div className={styles.landing__cardIcon} aria-hidden>
                    <MessageSquare size={22} strokeWidth={1.75} />
                  </div>
                  <h3 className={styles.landing__cardTitle}>
                    Readable threads
                  </h3>
                  <p className={styles.landing__cardBody}>
                    Questions and answers stay structured so the group can
                    reuse explanations before exams and interviews.
                  </p>
                </article>
                <article className={styles.landing__card}>
                  <div className={styles.landing__cardIcon} aria-hidden>
                    <Sparkles size={22} strokeWidth={1.75} />
                  </div>
                  <h3 className={styles.landing__cardTitle}>
                    Lightweight AI help
                  </h3>
                  <p className={styles.landing__cardBody}>
                    Suggestions on your question draft and a quick relevance
                    check on answer drafts. Always your choice to apply or
                    post.
                  </p>
                </article>
                <article className={`${styles.landing__card} `}>
                  <div className={styles.landing__cardIcon} aria-hidden>
                    <Layers size={22} strokeWidth={1.75} />
                  </div>
                  <h3 className={styles.landing__cardTitle}>
                    RAG over your course library
```

```
                    </h3>
                    <p className={styles.landing__cardBody}>
                      Instructors and cohorts add PDFs, syllabi, and notes into
                      a controlled corpus. When you ask, the system retrieves
                      the most relevant passages and attaches them to the
                      prompt, so explanations stay tied to your class materials,
                      not the open web.
                    </p>
                  </article>
                </div>
              </div>
            </section>

            <section
              className={styles.landing__process}
              id='how-it-works'
              aria-labelledby='how-heading'
            >
              <div className={styles.landing__sectionInner}>
                <h2 className={styles.landing__sectionTitle} id='how-heading'>
                  How it works
                </h2>
                <p className={styles.landing__sectionLead}>
                  Four steps from question to searchable knowledge for the next
                  person.
                </p>
                <ol className={styles.landing__steps}>
                  <li className={styles.landing__step}>
                    <span className={styles.landing__stepIcon} aria-hidden>
                      <PenSquare size={18} />
                    </span>
                    <div>
                      <h3 className={styles.landing__stepTitle}>
                        Ask with context
                      </h3>
                      <p className={styles.landing__stepText}>
                        Title, environment, errors, and what you tried, so peers
                        reproduce before they teach.
                      </p>
                    </div>
                  </li>
                  <li className={styles.landing__step}>
                    <span className={styles.landing__stepIcon} aria-hidden>
                      <MessageSquare size={18} />
                    </span>
                    <div>
                      <h3 className={styles.landing__stepTitle}>Get answers</h3>
                      <p className={styles.landing__stepText}>
                        Replies live in one thread with markdown and code
                        blocks, visible to everyone in the cohort.
                      </p>
                    </div>
                  </li>
                  <li className={styles.landing__step}>
                    <span className={styles.landing__stepIcon} aria-hidden>
                      <Search size={18} />
                    </span>
                    <div>
                      <h3 className={styles.landing__stepTitle}>
                        Search two ways
                      </h3>
                      <p className={styles.landing__stepText}>
                        Classic text search on the feed, or semantic search when
                        you want “questions like this one.”
                      </p>
                    </div>
                  </li>
                  <li className={styles.landing__step}>
                    <span className={styles.landing__stepIcon} aria-hidden>
                      <Library size={18} />
```

```
                    </span>
                    <div>
                      <h3 className={styles.landing__stepTitle}>
                        Own your trail
                      </h3>
                      <p className={styles.landing__stepText}>
                        Your topics list keeps authorship clear. The Knowledge
                        base hosts uploads and RAG retrieval so answers can cite
                        your materials. See <strong>Course RAG</strong> above
                        for the full pipeline.
                      </p>
                    </div>
                  </li>
                </ol>
              </div>
            </section>

            <section className={styles.landing__cta}>
              <div className={styles.landing__ctaInner}>
                <h2 className={styles.landing__ctaTitle}>Ready when you are</h2>
                <p className={styles.landing__ctaText}>
                  Create a free learner account to post, reply, and search the
                  forum index.
                </p>
                <button
                  type='button'
                  className={styles.landing__btnPrimary}
                  onClick={() => navigate('/auth')}
                >
                  Create free account
                  <ArrowRight size={16} aria-hidden />
                </button>
              </div>
            </section>
          </>
        )}

        {isAuthenticated && (
          <section className={styles.landing__welcomeBack}>
            <div className={styles.landing__sectionInner}>
              <p className={styles.landing__eyebrow}>Signed in</p>
              <h2 className={styles.landing__sectionTitle}>
                Back to your workspace
              </h2>
              <p className={styles.landing__sectionLead}>
                Home has the live feed, shortcuts, and search. Your topics lists
                only threads you started. Course-document RAG (ingest, retrieve,
                cite) ties the Knowledge base to threads. Scroll to{' '}
                <strong>Course RAG</strong> on this page for the full picture.
              </p>
              <button
                type='button'
                className={styles.landing__btnPrimary}
                onClick={() => navigate('/dashboard')}
              >
                Open forum home
                <ArrowRight size={16} aria-hidden />
              </button>
            </div>
          </section>
        )}
      </main>

      <footer className={styles.landing__footer}>
        <div className={styles.landing__footerInner}>
          <div>
            <p className={styles.landing__footerBrand}>Evangadi Forum</p>
            <p className={styles.landing__footerMeta}>
              © {new Date().getFullYear()} · Learner-led Q&A
            </p>
```

```
          </div>
          <div className={styles.landing__footerLinks}>
            <button
              type='button'
              className={styles.landing__footerLink}
              onClick={() => navigate('/auth')}
            >
              Sign in
            </button>
            <span className={styles.landing__footerDot} aria-hidden>
              .
            </span>
            <a href='#' className={styles.landing__footerLinkAnchor}>
              Privacy
            </a>
            <span className={styles.landing__footerDot} aria-hidden>
              .
            </span>
            <a href='#' className={styles.landing__footerLinkAnchor}>
              Terms
            </a>
          </div>
        </div>
      </footer>
    </div>
  );
}

```

PAGE · CSS frontend/src/pages/Landing/Landing.module.css

Landing Page Styles

Hero styling, gradient backgrounds, and responsive CTA layout.

Lines: 680 Size: 14.7 KB Format: .CSS

```css
/**
 * Landing: public marketing. Uses same tokens as Layout/Dashboard/Auth:
 * --surface, --background, --border, --radius-*, --text-*, --primary.
 * Width: --app-content-max for horizontal rhythm with `.layout__mainInner`.
 */

.landing {
  min-height: 100vh;
  display: flex;
  flex-direction: column;
  background-color: var(--background);
  color: var(--text-primary);
}

/* Header */
.landing__header {
  position: sticky;
  top: 0;
  z-index: 50;
  background-color: var(--surface);
  border-bottom: 1px solid var(--border);
  box-shadow: var(--shadow-sm);
}

.landing__headerInner {
  max-width: var(--app-content-max, 80rem);
  margin: 0 auto;
  padding: var(--spacing-md) var(--spacing-lg);
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  justify-content: space-between;
  gap: var(--spacing-md);
}

.landing__brand {
  display: flex;
  align-items: flex-start;
  gap: var(--spacing-md);
  background: none;
  border: none;
  cursor: pointer;
  text-align: left;
  padding: var(--spacing-xs);
  margin: calc(-1 * var(--spacing-xs));
  border-radius: var(--radius-md);
  transition: background-color var(--transition-fast);
}

.landing__brand:hover {
  background-color: var(--surface-hover);
}

.landing__brand:focus-visible {
  outline: 2px solid var(--primary);
  outline-offset: 2px;
}

.landing__brandMark {
  flex-shrink: 0;
  width: 2.375rem;
```

```
  height: 2.375rem;
  display: flex;
  align-items: center;
  justify-content: center;
  border-radius: var(--radius-md);
  background: linear-gradient(
    145deg,
    var(--primary) 0%,
    var(--primary-hover) 100%
  );
  color: var(--text-inverse);
  box-shadow: 0 1px 2px rgba(15, 23, 42, 0.08);
}

.landing__brandText {
  display: flex;
  flex-direction: column;
  gap: 0.125rem;
  min-width: 0;
  padding-top: 0.125rem;
}

.landing__brandName {
  font-size: var(--text-base);
  font-weight: var(--font-semibold);
  color: var(--tertiary);
  letter-spacing: -0.02em;
}

.landing__brandLine {
  font-size: 0.6875rem;
  color: var(--text-tertiary);
  line-height: 1.35;
}

.landing__nav {
  display: flex;
  align-items: center;
  gap: var(--spacing-xs);
  flex: 1 1 auto;
  justify-content: center;
}

@media (min-width: 900px) {
  .landing__nav {
    flex: 0 1 auto;
    justify-content: center;
  }
}

.landing__navLink {
  padding: var(--spacing-sm) var(--spacing-md);
  font-size: var(--text-sm);
  font-weight: var(--font-medium);
  color: var(--text-secondary);
  background: none;
  border: none;
  border-radius: var(--radius-md);
  cursor: pointer;
  transition: color var(--transition-fast),
    background-color var(--transition-fast);
}

.landing__navLink:hover {
  color: var(--text-primary);
  background-color: var(--surface-container-low);
}

.landing__headerActions {
  display: flex;
```

```
  align-items: center;
  gap: var(--spacing-sm);
  flex-shrink: 0;
}

/* Buttons */
.landing__btnPrimary {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: var(--spacing-sm);
  padding: var(--spacing-sm) var(--spacing-lg);
  font-size: var(--text-sm);
  font-weight: var(--font-semibold);
  color: var(--text-inverse);
  background-color: var(--primary);
  border: none;
  border-radius: var(--radius-md);
  cursor: pointer;
  box-shadow: var(--shadow-sm);
  transition: background-color var(--transition-fast),
    box-shadow var(--transition-fast);
}

.landing__btnPrimary:hover {
  background-color: var(--primary-hover);
}

.landing__btnGhost {
  padding: var(--spacing-sm) var(--spacing-md);
  font-size: var(--text-sm);
  font-weight: var(--font-medium);
  color: var(--text-secondary);
  background: none;
  border: none;
  border-radius: var(--radius-md);
  cursor: pointer;
  transition: color var(--transition-fast);
}

.landing__btnGhost:hover {
  color: var(--text-primary);
}

.landing__btnOutline {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  padding: var(--spacing-sm) var(--spacing-lg);
  font-size: var(--text-sm);
  font-weight: var(--font-medium);
  color: var(--text-primary);
  background-color: var(--surface);
  border: 1px solid var(--border);
  border-radius: var(--radius-md);
  cursor: pointer;
  transition: border-color var(--transition-fast),
    background-color var(--transition-fast);
}

.landing__btnOutline:hover {
  border-color: var(--border-hover);
  background-color: var(--surface-container-low);
}

/* Main */
.landing__main {
  flex: 1;
}

```

```
.landing__sectionInner {
  max-width: var(--app-content-max, 80rem);
  margin: 0 auto;
  padding-left: var(--spacing-lg);
  padding-right: var(--spacing-lg);
}

/* Hero */
.landing__hero {
  padding: var(--spacing-2xl) 0 var(--spacing-3xl);
  background-color: var(--surface);
}

.landing__heroInner {
  max-width: var(--app-content-max, 80rem);
  margin: 0 auto;
  padding: 0 var(--spacing-lg);
  display: grid;
  gap: var(--spacing-2xl);
  align-items: start;
}

@media (min-width: 960px) {
  .landing__heroInner {
    grid-template-columns: 1fr minmax(16rem, 22rem);
    gap: var(--spacing-3xl);
    align-items: center;
  }
}

.landing__heroCopy {
  min-width: 0;
}

.landing__eyebrow {
  display: inline-flex;
  align-items: center;
  gap: var(--spacing-sm);
  margin: 0 0 var(--spacing-md);
  padding: 0.25rem 0.75rem;
  font-size: 0.625rem;
  font-weight: var(--font-semibold);
  letter-spacing: 0.08em;
  text-transform: uppercase;
  color: var(--text-placeholder);
  background-color: var(--surface-container-low);
  border-radius: var(--radius-full);
  border: 1px solid var(--border);
  width: fit-content;
}

.landing__title {
  margin: 0 0 var(--spacing-md);
  font-size: clamp(1.75rem, 4vw, 2.5rem);
  font-weight: var(--font-bold);
  line-height: 1.15;
  letter-spacing: -0.03em;
  color: var(--tertiary);
}

.landing__titleAccent {
  color: var(--primary);
}

.landing__lead {
  margin: 0 0 var(--spacing-xl);
  max-width: 36rem;
  font-size: var(--text-base);
  line-height: 1.6;
  color: var(--text-secondary);
```

```
}

.landing__leadStrong {
  font-weight: var(--font-semibold);
  color: var(--text-primary);
}

.landing__heroCtas {
  display: flex;
  flex-wrap: wrap;
  gap: var(--spacing-md);
}

.landing__heroPanel {
  padding: var(--spacing-lg);
  background-color: var(--background);
  border: 1px solid var(--border);
  border-radius: var(--radius-lg);
  box-shadow: var(--shadow-sm);
}

.landing__heroPanelLabel {
  margin: 0 0 var(--spacing-md);
  font-size: var(--text-xs);
  font-weight: var(--font-semibold);
  text-transform: uppercase;
  letter-spacing: 0.06em;
  color: var(--text-tertiary);
}

.landing__heroPanelList {
  margin: 0;
  padding: 0;
  list-style: none;
  display: flex;
  flex-direction: column;
  gap: var(--spacing-md);
}

.landing__heroPanelList li {
  display: flex;
  align-items: flex-start;
  gap: var(--spacing-sm);
  font-size: var(--text-sm);
  line-height: 1.5;
  color: var(--text-secondary);
}

.landing__heroPanelList svg {
  flex-shrink: 0;
  margin-top: 0.125rem;
  color: var(--primary);
}

.landing__heroPanelStrong {
  font-weight: var(--font-semibold);
  color: var(--text-primary);
}

/* Course RAG (all visitors) */
.landing__rag {
  padding: var(--spacing-3xl) 0;
  background-color: var(--surface-container-low);
  border-top: 1px solid var(--border);
  box-shadow: inset 0 1px 0 rgba(255, 255, 255, 0.6);
}

.landing__ragEyebrow {
  margin: 0 0 var(--spacing-sm);
  font-size: 0.625rem;
```

```
  font-weight: var(--font-semibold);
  letter-spacing: 0.1em;
  text-transform: uppercase;
  color: var(--primary-dark);
}

.landing__rag .landing__sectionLead em {
  font-style: normal;
  font-weight: var(--font-medium);
  color: var(--text-primary);
}

.landing__ragPipeline {
  display: grid;
  gap: var(--spacing-lg);
  margin-bottom: var(--spacing-xl);
  grid-template-columns: 1fr;
}

@media (min-width: 900px) {
  .landing__ragPipeline {
    grid-template-columns: repeat(3, 1fr);
  }
}

.landing__ragStep {
  padding: var(--spacing-lg);
  background-color: var(--surface);
  border: 1px solid var(--border);
  border-radius: var(--radius-lg);
  box-shadow: var(--shadow-sm);
}

.landing__ragStepIcon {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 2.5rem;
  height: 2.5rem;
  margin-bottom: var(--spacing-md);
  border-radius: var(--radius-md);
  background-color: var(--primary-light);
  color: var(--primary-dark);
}

.landing__ragStepTitle {
  margin: 0 0 var(--spacing-xs);
  font-size: var(--text-sm);
  font-weight: var(--font-semibold);
  color: var(--text-primary);
}

.landing__ragStepText {
  margin: 0;
  font-size: var(--text-sm);
  line-height: 1.55;
  color: var(--text-secondary);
}

.landing__ragFootnote {
  margin: 0;
  padding: var(--spacing-md) var(--spacing-lg);
  font-size: var(--text-sm);
  line-height: 1.55;
  color: var(--text-secondary);
  background-color: var(--surface);
  border: 1px solid var(--border);
  border-radius: var(--radius-md);
  border-left: 3px solid var(--primary);
}
```

```
412
413 .landing__ragFootnote strong {
414   color: var(--text-primary);
415   font-weight: var(--font-semibold);
416 }
417
418 /* Capabilities */
419 .landing__capabilities {
420   padding: var(--spacing-3xl) 0;
421   background-color: var(--background);
422   border-top: 1px solid var(--border);
423 }
424
425 .landing__sectionTitle {
426   margin: 0 0 var(--spacing-sm);
427   font-size: var(--text-xl);
428   font-weight: var(--font-semibold);
429   color: var(--text-primary);
430   letter-spacing: -0.02em;
431 }
432
433 .landing__sectionLead {
434   margin: 0 0 var(--spacing-xl);
435   max-width: 40rem;
436   font-size: var(--text-sm);
437   line-height: 1.55;
438   color: var(--text-secondary);
439 }
440
441 .landing__cardGrid {
442   display: grid;
443   gap: var(--spacing-lg);
444   grid-template-columns: 1fr;
445 }
446
447 @media (min-width: 640px) {
448   .landing__cardGrid {
449     grid-template-columns: repeat(2, 1fr);
450   }
451 }
452
453 @media (min-width: 1100px) {
454   .landing__cardGrid {
455     grid-template-columns: repeat(4, 1fr);
456   }
457 }
458
459 .landing__card {
460   padding: var(--spacing-lg);
461   background-color: var(--surface);
462   border: 1px solid var(--border);
463   border-radius: var(--radius-lg);
464   box-shadow: var(--shadow-sm);
465   transition: border-color var(--transition-fast),
466     box-shadow var(--transition-fast);
467 }
468
469 .landing__card:hover {
470   border-color: var(--border-hover);
471   box-shadow: var(--shadow-md);
472 }
473
474 .landing__cardIcon {
475   width: 2.75rem;
476   height: 2.75rem;
477   display: flex;
478   align-items: center;
479   justify-content: center;
480   margin-bottom: var(--spacing-md);
481   border-radius: var(--radius-md);
```

```
  background-color: var(--surface-container-low);
  color: var(--primary);
}

.landing__cardTitle {
  margin: 0 0 var(--spacing-xs);
  font-size: var(--text-base);
  font-weight: var(--font-semibold);
  color: var(--text-primary);
}

.landing__cardBody {
  margin: 0;
  font-size: var(--text-sm);
  line-height: 1.55;
  color: var(--text-secondary);
}

/* Process */
.landing__process {
  padding: var(--spacing-3xl) 0;
  background-color: var(--surface);
  border-top: 1px solid var(--border);
}

.landing__steps {
  margin: 0;
  padding: 0;
  list-style: none;
  display: grid;
  gap: var(--spacing-lg);
}

@media (min-width: 768px) {
  .landing__steps {
    grid-template-columns: repeat(2, 1fr);
  }
}

.landing__step {
  display: flex;
  gap: var(--spacing-md);
  padding: var(--spacing-lg);
  background-color: var(--background);
  border: 1px solid var(--border);
  border-radius: var(--radius-lg);
}

.landing__stepIcon {
  flex-shrink: 0;
  width: 2.5rem;
  height: 2.5rem;
  display: flex;
  align-items: center;
  justify-content: center;
  border-radius: var(--radius-md);
  background-color: var(--surface-container-low);
  color: var(--primary);
}

.landing__stepTitle {
  margin: 0 0 var(--spacing-xs);
  font-size: var(--text-sm);
  font-weight: var(--font-semibold);
  color: var(--text-primary);
}

.landing__stepText {
  margin: 0;
  font-size: var(--text-sm);
```

```
  line-height: 1.55;
  color: var(--text-secondary);
}

/* CTA */
.landing__cta {
  padding: var(--spacing-3xl) 0;
  background-color: var(--background);
  border-top: 1px solid var(--border);
}

.landing__ctaInner {
  max-width: var(--app-content-max, 80rem);
  margin: 0 auto;
  padding: var(--spacing-2xl) var(--spacing-lg);
  text-align: center;
  background-color: var(--surface);
  border: 1px solid var(--border);
  border-radius: var(--radius-lg);
  box-shadow: var(--shadow-sm);
}

.landing__ctaTitle {
  margin: 0 0 var(--spacing-sm);
  font-size: var(--text-xl);
  font-weight: var(--font-semibold);
  color: var(--text-primary);
}

.landing__ctaText {
  margin: 0 auto var(--spacing-lg);
  max-width: 28rem;
  font-size: var(--text-sm);
  color: var(--text-secondary);
  line-height: 1.55;
}

/* Signed-in strip */
.landing__welcomeBack {
  padding: var(--spacing-3xl) 0;
  background-color: var(--surface);
}

.landing__welcomeBack .landing__sectionInner {
  padding: var(--spacing-2xl) var(--spacing-lg);
  background-color: var(--background);
  border: 1px solid var(--border);
  border-radius: var(--radius-lg);
  max-width: 40rem;
  margin-left: auto;
  margin-right: auto;
}

/* Footer */
.landing__footer {
  margin-top: auto;
  padding: var(--spacing-xl) var(--spacing-lg);
  background-color: var(--surface);
  border-top: 1px solid var(--border);
}

.landing__footerInner {
  max-width: var(--app-content-max, 80rem);
  margin: 0 auto;
  display: flex;
  flex-direction: column;
  gap: var(--spacing-md);
  align-items: flex-start;
}

```

```
@media (min-width: 640px) {
  .landing__footerInner {
    flex-direction: row;
    align-items: center;
    justify-content: space-between;
  }
}

.landing__footerBrand {
  margin: 0;
  font-size: var(--text-sm);
  font-weight: var(--font-semibold);
  color: var(--tertiary);
}

.landing__footerMeta {
  margin: 0.25rem 0 0;
  font-size: var(--text-xs);
  color: var(--text-tertiary);
}

.landing__footerLinks {
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  gap: var(--spacing-sm);
  font-size: var(--text-xs);
}

.landing__footerLink {
  padding: 0;
  border: none;
  background: none;
  font-size: inherit;
  color: var(--text-secondary);
  cursor: pointer;
  text-decoration: underline;
  text-underline-offset: 2px;
}

.landing__footerLink:hover {
  color: var(--primary);
}

.landing__footerLinkAnchor {
  color: var(--text-secondary);
  text-decoration: none;
}

.landing__footerLinkAnchor:hover {
  color: var(--primary);
  text-decoration: underline;
}

.landing__footerDot {
  color: var(--text-placeholder);
  user-select: none;
}

```

PAGE · AUTH frontend/src/pages/Auth/Auth.jsx

Authentication Page (Login & Register)

Toggleable login and signup forms with inline validation and auth error display.

Lines: 380 Size: 13.6 KB Format: .JSX

```jsx
/**
 * Auth: combined login + register form; switches mode without changing routes.
 */
import { useState } from 'react';

import { motion as Motion, AnimatePresence } from 'framer-motion';
import { useNavigate, useLocation } from 'react-router-dom';
import {
  Sparkles,
  Code,
  ArrowRight,
  Eye,
  EyeOff,
  MessageSquare,
} from 'lucide-react';
import { useAuth } from '../../contexts/AuthContext';
import styles from './Auth.module.css';

export default function Auth() {
  const navigate = useNavigate();
  const location = useLocation();
  const { register, login } = useAuth();
  const [isLogin, setIsLogin] = useState(true);

  // Registration form state
  const [firstName, setFirstName] = useState('');
  const [lastName, setLastName] = useState('');
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [showPassword, setShowPassword] = useState(false);

  // Error and loading state
  const [error, setError] = useState(null);
  const [loading, setLoading] = useState(false);
  const [successMessage, setSuccessMessage] = useState(null);

  // Handle form submission for both login and registration
  const handleSubmit = async e => {
    e.preventDefault();
    setError(null);
    setSuccessMessage(null);
    const normalizedEmail = email.trim().toLowerCase();
    // regex for email validation
    const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

    if (!normalizedEmail) {
      setError('Email is required.');
      return;
    }

    if (!emailPattern.test(normalizedEmail)) {
      setError('Please enter a valid email address.');
      return;
    }

    if (!password.trim()) {
      setError('Password is required.');
      return;
    }

    const trimmedFirstName = firstName.trim();
```

```
    const trimmedLastName = lastName.trim();

    if (!isLogin) {
      if (!trimmedFirstName) {
        setError('First name is required.');
        return;
      }
      if (trimmedFirstName.length < 3) {
        setError('First name must be at least 3 characters long.');
        return;
      }
      if (!trimmedLastName) {
        setError('Last name is required.');
        return;
      }
      if (trimmedLastName.length < 3) {
        setError('Last name must be at least 3 characters long.');
        return;
      }
      if (password.length < 6) {
        setError('Password must be at least 6 characters long.');
        return;
      }
    }

    setLoading(true);

    try {
      if (isLogin) {
        // Login flow
        await login({ email: normalizedEmail, password });
        setSuccessMessage('Sign-in successful. Redirecting...');
        // Clear form fields
        setEmail('');
        setPassword('');
        setShowPassword(false);
        // Delay redirect to show successful message
        await new Promise(resolve => setTimeout(resolve, 1000));

        // Check location state for original URL after login
        // Redirect to original URL if present, otherwise dashboard
        const from = location.state?.from?.pathname || '/dashboard';
        navigate(from, { replace: true });
      } else {
        // Registration flow
        await register({
          firstName: trimmedFirstName,
          lastName: trimmedLastName,
          email: normalizedEmail,
          password,
        });
        setSuccessMessage('Registration successful! Please log in.');
        // Clear form fields
        setFirstName('');
        setLastName('');
        setEmail('');
        setPassword('');
        // Automatically switch to login form after 1.5 seconds
        setTimeout(() => {
          setIsLogin(true);
          setSuccessMessage(null);
        }, 1500);
      }
    } catch (err) {
      setError(err.message || 'An unexpected error occurred.');
    } finally {
      setLoading(false);
    }
  };

```

```
  return (
    <div className={styles.auth}>
      {/* Left: Info Section */}
      <section className={styles.auth__info}>
        <div className={styles.auth__infoContent}>
          <header className={styles.auth__infoHeader}>
            <div
              className={styles.auth__infoBranding}
              onClick={() => navigate('/')}
              title='Go to Home'
              role='button'
              tabIndex={0}
              onKeyDown={e => {
                if (e.key === 'Enter' || e.key === ' ') {
                  e.preventDefault();
                  navigate('/');
                }
              }}
            >
              <div className={styles.auth__infoLogo} aria-hidden>
                <MessageSquare
                  className={styles.auth__infoLogoIcon}
                  size={22}
                />
              </div>
              <div className={styles.auth__infoBrandCopy}>
                <p className={styles.auth__infoTitle}>Evangadi Forum</p>
                <p className={styles.auth__infoTagline}>
                  Learn together. Ask with context.
                </p>
              </div>
            </div>
            <p className={styles.auth__infoDescription}>
              Sign in to post technical questions, follow threads, and search
              the forum with both keyword and AI similarity modes, built for
              Evangadi coursework and peer review.
            </p>
          </header>

          <div className={styles.auth__features}>
            <div className={styles.auth__feature}>
              <div className={styles.auth__featureIcon}>
                <Sparkles size={20} />
              </div>
              <div className={styles.auth__featureContent}>
                <h3 className={styles.auth__featureTitle}>Visible reasoning</h3>
                <p className={styles.auth__featureDescription}>
                  Threads stay readable: markdown, code blocks, and replies
                  build a mini knowledge base your cohort can revisit before
                  exams.
                </p>
              </div>
            </div>
            <div className={styles.auth__feature}>
              <div className={styles.auth__featureIcon}>
                <Code size={20} />
              </div>
              <div className={styles.auth__featureContent}>
                <h3 className={styles.auth__featureTitle}>
                  Low-friction workflow
                </h3>
                <p className={styles.auth__featureDescription}>
                  One layout for asking, answering, and scanning search results,
                  so you spend energy on the problem, not on hunting controls.
                </p>
              </div>
            </div>
          </div>

          <div className={styles.auth__infoFooter}>
```

```
            <div className={styles.auth__infoFooterContent}>
              <div className={styles.auth__infoAvatars}>
                {[1, 2, 3].map(i => (
                  <img
                    key={i}
                    src={`https://picsum.photos/seed/${i + 50}/100/100`}
                    className={styles.auth__infoAvatar}
                    alt='u'
                    referrerPolicy='no-referrer'
                  />
                ))}
              </div>
              <span className={styles.auth__infoBadge}>
                Evangadi cohorts · weekly stand-ups · office-hour style help
              </span>
            </div>
          </div>
        </div>
      </section>

      {/* Right: Auth Forms */}
      <section className={styles.auth__formSection}>
        <div className={styles.auth__formContainer}>
          <AnimatePresence mode='wait'>
            <Motion.div
              key={isLogin ? 'login' : 'register'}
              initial={{ opacity: 0, y: 10 }}
              animate={{ opacity: 1, y: 0 }}
              exit={{ opacity: 0, y: -10 }}
              transition={{ duration: 0.2 }}
            >
              <div className={styles.auth__formHeader}>
                <h2 className={styles.auth__formTitle}>
                  {isLogin ? 'Sign in to your account' : 'Create an account'}
                </h2>
                <p className={styles.auth__formSubtitle}>
                  {isLogin
                    ? 'Enter your email address and password to continue.'
                    : 'Complete the form below to create your account.'}
                </p>
              </div>

              <form className={styles.auth__form} onSubmit={handleSubmit}>
                {!isLogin && (
                  <>
                    <div className={styles.auth__inputGroup}>
                      <label htmlFor='firstName' className={styles.auth__label}>
                        First Name
                      </label>
                      <input
                        id='firstName'
                        type='text'
                        placeholder='First name'
                        className={styles.auth__input}
                        value={firstName}
                        onChange={e => setFirstName(e.target.value)}
                      />
                    </div>

                    <div className={styles.auth__inputGroup}>
                      <label htmlFor='lastName' className={styles.auth__label}>
                        Last Name
                      </label>
                      <input
                        id='lastName'
                        type='text'
                        placeholder='Last name'
                        className={styles.auth__input}
                        value={lastName}
                        onChange={e => setLastName(e.target.value)}
```

```
                      />
                    </div>
                  </>
                )}

                <div className={styles.auth__inputGroup}>
                  <label htmlFor='email' className={styles.auth__label}>
                    Email Address
                  </label>
                  <input
                    id='email'
                    type='email'
                    placeholder='Enter your email address'
                    className={styles.auth__input}
                    value={email}
                    onChange={e => setEmail(e.target.value)}
                  />
                </div>

                <div className={styles.auth__inputGroup}>
                  <div className={styles.auth__labelRow}>
                    <label htmlFor='password' className={styles.auth__label}>
                      Password
                    </label>
                  </div>
                  <div className={styles.auth__passwordWrap}>
                    <input
                      id='password'
                      type={showPassword ? 'text' : 'password'}
                      placeholder='••••••••'
                      className={`${styles.auth__input} ${styles.auth__inputPassword}`}
                      value={password}
                      onChange={e => setPassword(e.target.value)}
                    />
                    <button
                      type='button'
                      className={styles.auth__passwordToggle}
                      onClick={() => setShowPassword(v => !v)}
                      aria-label={
                        showPassword ? 'Hide password' : 'Show password'
                      }
                      aria-pressed={showPassword}
                    >
                      {showPassword ? (
                        <EyeOff size={18} aria-hidden />
                      ) : (
                        <Eye size={18} aria-hidden />
                      )}
                    </button>
                  </div>
                </div>

                {successMessage && (
                  <div className={styles.auth__success}>{successMessage}</div>
                )}

                {error && <div className={styles.auth__error}>{error}</div>}

                <div className={styles.auth__buttonContainer}>
                  <button
                    type='submit'
                    className={`${styles.auth__button} ${styles['auth__button--primary']}`}
                    disabled={loading}
                  >
                    {loading
                      ? 'Processing...'
                      : isLogin
                      ? 'Sign In'
                      : 'Create Account'}
                    {!loading && (
```

```
                      <ArrowRight
                        size={16}
                        className={styles.auth__buttonIcon}
                      />
                    )}
                  </button>
                </div>

                <div className={styles.auth__divider}>
                  <div className={styles.auth__dividerLine}>
                    <div className={styles.auth__dividerBorder}></div>
                  </div>
                  <div className={styles.auth__dividerText}>
                    Additional options
                  </div>
                </div>
              </form>

              <footer className={styles.auth__formFooter}>
                <p className={styles.auth__formFooterText}>
                  {isLogin
                    ? "Don't have an account?"
                    : 'Already have an account?'}
                  <button
                    onClick={() => setIsLogin(!isLogin)}
                    className={styles.auth__formFooterLink}
                  >
                    {isLogin ? 'Create an account' : 'Back to sign in'}
                  </button>
                </p>
              </footer>
            </Motion.div>
          </AnimatePresence>
        </div>
      </section>
    </div>
  );
}
```

PAGE · CSS frontend/src/pages/Auth/Auth.module.css

Auth Page Styles

Auth card layout, input fields, toggle buttons, and validation state colors.

Lines: 538 Size: 10.6 KB Format: .CSS

```css
/* Auth Page Styles - BEM Naming Convention */

/* Main container */
.auth {
  min-height: 100vh;
  display: flex;
  flex-direction: column;
  background-color: var(--background);
}

@media (min-width: 768px) {
  .auth {
    flex-direction: row;
  }
}

/* Info Section (Left Side) */
.auth__info {
  display: flex;
  flex-direction: column;
  justify-content: center;
  padding: 3rem 1.5rem;
  background: linear-gradient(
    155deg,
    var(--primary) 0%,
    var(--primary-hover) 48%,
    var(--primary-dark) 100%
  );
  color: var(--text-inverse);
}

@media (min-width: 768px) {
  .auth__info {
    width: 45%;
    padding: 4rem;
  }
}

.auth__infoContent {
  max-width: var(--app-prose-max, 32rem);
  margin: 0 auto;
}

.auth__infoHeader {
  margin-bottom: 2.5rem;
}

/* Brand row: matches app Sidebar hierarchy (logo + title + tagline). */
.auth__infoBranding {
  display: flex;
  align-items: flex-start;
  gap: var(--spacing-md);
  cursor: pointer;
  padding: var(--spacing-sm);
  margin: calc(-1 * var(--spacing-sm)) calc(-1 * var(--spacing-sm)) 0;
  border-radius: var(--radius-lg);
  transition: background-color var(--transition-fast);
}

.auth__infoBranding:hover {
  background-color: rgba(255, 255, 255, 0.08);
```

```
}

.auth__infoBranding:focus {
  outline: none;
}

.auth__infoBranding:focus-visible {
  outline: 2px solid rgba(255, 255, 255, 0.9);
  outline-offset: 2px;
}

.auth__infoLogo {
  flex-shrink: 0;
  width: 2.5rem;
  height: 2.5rem;
  display: flex;
  align-items: center;
  justify-content: center;
  background-color: rgba(255, 255, 255, 0.96);
  border-radius: var(--radius-md);
  box-shadow: 0 1px 3px rgba(15, 23, 42, 0.12);
}

.auth__infoLogoIcon {
  color: var(--primary);
}

.auth__infoBrandCopy {
  min-width: 0;
  display: flex;
  flex-direction: column;
  gap: 0.25rem;
  padding-top: 0.125rem;
}

.auth__infoTitle {
  margin: 0;
  font-size: var(--text-2xl);
  font-weight: var(--font-semibold);
  line-height: 1.2;
  color: var(--text-inverse);
  letter-spacing: -0.02em;
}

.auth__infoTagline {
  margin: 0;
  font-size: var(--text-sm);
  font-weight: var(--font-normal);
  line-height: 1.4;
  color: rgba(255, 255, 255, 0.88);
}

.auth__infoDescription {
  margin: var(--spacing-lg) 0 0;
  font-size: var(--text-base);
  opacity: 0.92;
  line-height: 1.65;
  color: rgba(255, 255, 255, 0.95);
}

/* Features Section */
.auth__features {
  display: flex;
  flex-direction: column;
  gap: 1.5rem;
  margin-bottom: 3rem;
}

.auth__feature {
  display: flex;
```

```
  gap: 1rem;
  padding: 1.5rem;
  background-color: rgba(255, 255, 255, 0.08);
  border: 1px solid rgba(255, 255, 255, 0.14);
  border-radius: var(--radius-lg);
  backdrop-filter: blur(10px);
  transition:
    background-color var(--transition-base),
    border-color var(--transition-base),
    transform var(--transition-base);
}

.auth__feature:hover {
  background-color: rgba(255, 255, 255, 0.12);
  border-color: rgba(255, 255, 255, 0.22);
  transform: translateX(0.125rem);
}

.auth__featureIcon {
  flex-shrink: 0;
  width: 2.5rem;
  height: 2.5rem;
  display: flex;
  align-items: center;
  justify-content: center;
  background-color: rgba(255, 255, 255, 0.2);
  border-radius: var(--radius-md);
}

.auth__featureContent {
  flex: 1;
}

.auth__featureTitle {
  font-size: var(--text-lg);
  font-weight: var(--font-semibold);
  margin-bottom: 0.25rem;
  color: var(--text-inverse);
  letter-spacing: -0.02em;
}

.auth__featureDescription {
  font-size: var(--text-sm);
  color: rgba(255, 255, 255, 0.88);
  line-height: 1.55;
}

/* Info Footer */
.auth__infoFooter {
  padding-top: 2rem;
  border-top: 1px solid rgba(255, 255, 255, 0.2);
}

.auth__infoFooterContent {
  display: flex;
  align-items: center;
  gap: 1rem;
}

.auth__infoAvatars {
  display: flex;
  margin-left: -0.5rem;
}

.auth__infoAvatar {
  width: 2.5rem;
  height: 2.5rem;
  border-radius: 50%;
  border: 2px solid white;
  margin-left: -0.5rem;
```

```
  object-fit: cover;
}

.auth__infoBadge {
  font-size: var(--text-sm);
  font-weight: var(--font-medium);
  padding: 0.375rem 0.75rem;
  background-color: rgba(255, 255, 255, 0.2);
  border-radius: var(--radius-md);
  backdrop-filter: blur(10px);
}

/* Form Section (Right Side) */
.auth__formSection {
  display: flex;
  flex: 1;
  align-items: center;
  justify-content: center;
  padding: 2rem 1.5rem;
  background-color: var(--background);
  background-image: radial-gradient(
    ellipse 120% 80% at 50% 0%,
    rgba(255, 237, 213, 0.45) 0%,
    transparent 55%
  );
}

@media (min-width: 768px) {
  .auth__formSection {
    width: 55%;
    padding: 4rem;
  }
}

.auth__formContainer {
  width: 100%;
  max-width: var(--app-form-max, 28rem);
  padding: var(--spacing-xl);
  background-color: var(--surface);
  border: 1px solid var(--border);
  border-radius: var(--radius-xl);
  box-shadow:
    var(--shadow-md),
    inset 0 1px 0 rgba(255, 255, 255, 0.75);
}

@media (min-width: 768px) {
  .auth__formContainer {
    padding: var(--spacing-2xl);
  }
}

/* Form Header */
.auth__formHeader {
  margin-bottom: 1.75rem;
  text-align: left;
}

.auth__formTitle {
  font-size: var(--text-2xl);
  font-weight: var(--font-semibold);
  color: var(--text-primary);
  margin: 0 0 0.375rem;
  letter-spacing: -0.02em;
  line-height: 1.25;
}

.auth__formSubtitle {
  margin: 0;
  font-size: var(--text-sm);
```

```
  color: var(--text-secondary);
  line-height: 1.55;
}

/* Form */
.auth__form {
  display: flex;
  flex-direction: column;
  gap: 1.25rem;
}

/* Input Group */
.auth__inputGroup {
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
}

.auth__labelRow {
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.auth__label {
  font-size: var(--text-sm);
  font-weight: var(--font-medium);
  color: var(--text-primary);
}

.auth__labelLink {
  font-size: var(--text-sm);
  color: var(--primary);
  text-decoration: none;
  transition: var(--transition-fast);
}

.auth__labelLink:hover {
  color: var(--primary-container);
  text-decoration: underline;
}

.auth__input {
  width: 100%;
  padding: 0.75rem 1rem;
  font-size: var(--text-base);
  color: var(--text-primary);
  background-color: var(--surface);
  border: 1px solid var(--border);
  border-radius: var(--radius-md);
  outline: none;
  transition: var(--transition-fast);
}

.auth__input::placeholder {
  color: var(--text-tertiary);
}

.auth__input:hover {
  border-color: var(--border-hover);
}

.auth__passwordWrap {
  position: relative;
  width: 100%;
}

.auth__inputPassword {
  padding-right: 2.75rem;
}
```

```
342
343 .auth__passwordToggle {
344   position: absolute;
345   right: 0.5rem;
346   top: 50%;
347   transform: translateY(-50%);
348   display: flex;
349   align-items: center;
350   justify-content: center;
351   padding: 0.25rem;
352   color: var(--text-secondary);
353   background: none;
354   border: none;
355   border-radius: var(--radius-sm);
356   cursor: pointer;
357   transition: var(--transition-fast);
358 }
359
360 .auth__passwordToggle:hover {
361   color: var(--text-primary);
362   background-color: var(--surface-container-low);
363 }
364
365 .auth__passwordToggle:focus {
366   outline: none;
367 }
368
369 .auth__passwordToggle:focus-visible {
370   outline: 2px solid var(--primary);
371   outline-offset: 2px;
372 }
373
374 /* Button Container */
375 .auth__buttonContainer {
376   margin-top: 0.5rem;
377 }
378
379 /* Error Message */
380 .auth__error {
381   padding: 0.75rem 1rem;
382   font-size: var(--text-sm);
383   color: #dc2626;
384   background-color: #fef2f2;
385   border: 1px solid #fecaca;
386   border-radius: var(--radius-md);
387   line-height: 1.5;
388 }
389
390 /* Success Message */
391 .auth__success {
392   padding: 0.75rem 1rem;
393   font-size: var(--text-sm);
394   color: #16a34a;
395   background-color: #f0fdf4;
396   border: 1px solid #bbf7d0;
397   border-radius: var(--radius-md);
398   line-height: 1.5;
399 }
400
401 /* Buttons */
402 .auth__button {
403   width: 100%;
404   display: flex;
405   align-items: center;
406   justify-content: center;
407   gap: 0.5rem;
408   padding: 0.75rem 1.5rem;
409   font-size: var(--text-base);
410   font-weight: var(--font-medium);
411   border-radius: var(--radius-md);
```

```
  border: none;
  cursor: pointer;
  transition: var(--transition-fast);
  outline: none;
}

.auth__button:active {
  transform: scale(0.98);
}

/* Primary Button */
.auth__button--primary {
  background-color: var(--primary);
  color: white;
  box-shadow: var(--shadow-sm);
}

.auth__button--primary:hover {
  background-color: var(--primary-container);
  box-shadow: var(--shadow-md);
}

.auth__button--primary:focus {
  box-shadow: var(--shadow-focus);
}

.auth__buttonIcon {
  transition: var(--transition-fast);
}

.auth__button--primary:hover .auth__buttonIcon {
  transform: translateX(0.125rem);
}

/* Social Button */
.auth__button--social {
  background-color: var(--surface);
  color: var(--text-primary);
  border: 1px solid var(--border);
}

.auth__button--social:hover {
  background-color: var(--surface-container-low);
  border-color: var(--border-hover);
}

.auth__button--social:focus {
  border-color: var(--primary);
  box-shadow: 0 0 0 3px rgba(249, 115, 22, 0.15);
}

/* Divider */
.auth__divider {
  position: relative;
  margin: 1.5rem 0;
  text-align: center;
}

.auth__dividerLine {
  position: absolute;
  top: 50%;
  left: 0;
  right: 0;
  height: 1px;
}

.auth__dividerBorder {
  height: 100%;
  background-color: var(--border);
}
```

```
482
483 .auth__dividerText {
484   position: relative;
485   display: inline-block;
486   padding: 0 1rem;
487   font-size: 0.625rem;
488   font-weight: var(--font-semibold);
489   letter-spacing: 0.08em;
490   text-transform: uppercase;
491   color: var(--text-placeholder);
492   background-color: var(--surface);
493 }
494
495 /* Social Buttons Container */
496 .auth__socialButtons {
497   display: flex;
498   gap: 0.75rem;
499 }
500
501 /* Form Footer */
502 .auth__formFooter {
503   margin-top: 1.75rem;
504   text-align: left;
505 }
506
507 .auth__formFooterText {
508   font-size: var(--text-sm);
509   color: var(--text-secondary);
510   display: flex;
511   flex-wrap: wrap;
512   align-items: baseline;
513   justify-content: flex-start;
514   gap: 0.35rem;
515 }
516
517 .auth__formFooterLink {
518   color: var(--primary);
519   font-weight: var(--font-medium);
520   background: none;
521   border: none;
522   cursor: pointer;
523   padding: 0;
524   text-decoration: none;
525   transition: var(--transition-fast);
526 }
527
528 .auth__formFooterLink:hover {
529   color: var(--primary-container);
530   text-decoration: underline;
531 }
532
533 .auth__formFooterLink:focus {
534   outline: 2px solid var(--primary);
535   outline-offset: 2px;
536   border-radius: var(--radius-sm);
537 }
538
```

PAGE · DASHBOARD frontend/src/pages/Dashboard/Dashboard.jsx

Main Dashboard & Question Feed

Question feed with exact keyword search, semantic search mode, and question listing.

Lines: 306 Size: 10.4 KB Format: .JSX

```jsx
import { useState, useEffect, useMemo, useCallback } from 'react';
import { useNavigate, useSearchParams } from 'react-router-dom';
import {
  PenSquare,
  MessageSquare,
  BookOpen,
  Sparkles,
  Search,
  X,
  RefreshCw,
  AlertCircle,
  ArrowRight,
} from 'lucide-react';
import { useAuth } from '../../contexts/AuthContext';
import { questionService } from '../../services/question/question.service';
import QuestionCard from '../../components/QuestionCard/QuestionCard';
import { isAuthoredByUser } from '../../lib/utils';
import styles from './Dashboard.module.css';

export default function Dashboard() {
  const { user } = useAuth();
  const navigate = useNavigate();
  const [searchParams, setSearchParams] = useSearchParams();

  const [questions, setQuestions] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  const keywordQuery = searchParams.get('q') || '';
  const semanticQuery = searchParams.get('semantic') || '';
  const isSearchActive = Boolean(keywordQuery || semanticQuery);

  const fetchQuestions = useCallback(async () => {
    setLoading(true);
    setError(null);
    try {
      if (semanticQuery.trim()) {
        const res = await questionService.searchQuestionsSemantic(semanticQuery.trim());
        setQuestions(res.questions);
      } else if (keywordQuery.trim()) {
        const res = await questionService.getQuestions({ search: keywordQuery.trim() });
        setQuestions(res.questions);
      } else {
        const res = await questionService.getQuestions();
        setQuestions(res.questions);
      }
    } catch (err) {
      console.error('Failed to load questions:', err);
      setError(err.message || 'Unable to load discussion questions.');
    } finally {
      setLoading(false);
    }
  }, [keywordQuery, semanticQuery]);

  useEffect(() => {
    fetchQuestions();
  }, [fetchQuestions]);

  const clearSearch = () => {
    setSearchParams({});
  };
```

```
62
63    // Metrics computation for stats row
64    const stats = useMemo(() => {
65      const totalQuestions = questions.length;
66      let totalReplies = 0;
67      let unanswered = 0;
68      let authoredByMe = 0;
69
70      questions.forEach(q => {
71        const count = q.answerCount ?? q.answers?.length ?? 0;
72        totalReplies += count;
73        if (count === 0) unanswered += 1;
74        if (isAuthoredByUser(q, user)) authoredByMe += 1;
75      });
76
77      return {
78        totalQuestions,
79        totalReplies,
80        unanswered,
81        authoredByMe,
82      };
83    }, [questions, user]);
84
85    const firstName = user?.firstName?.trim();
86    const greeting = firstName ? `Good to see you, ${firstName}.` : 'Welcome to the forum.';
87
88    return (
89      <div className={styles.dashboard}>
90        {/* Top Welcome / Hero Card */}
91        <section className={styles.heroCard} aria-labelledby='hero-greeting'>
92          <div className={styles.heroHeader}>
93            <span className={styles.heroEyebrow}>FORUM HOME</span>
94            <h1 id='hero-greeting' className={styles.heroTitle}>
95              {greeting}
96            </h1>
97            <p className={styles.heroSubtitle}>
98              Start a topic, revisit your own threads, or skim the live feed. Search above
99              works from any page once you are back on Home.
100           </p>
101         </div>
102
103         {/* Quick Action Tiles */}
104         <div className={styles.quickTiles}>
105           <button
106             type='button'
107             className={styles.quickTile}
108             onClick={() => navigate('/questions/ask')}
109           >
110             <div className={styles.quickTileIconWrap}>
111               <PenSquare size={20} className={styles.quickTileIcon} />
112             </div>
113             <div className={styles.quickTileContent}>
114               <strong className={styles.quickTileTitle}>New question</strong>
115               <span className={styles.quickTileDesc}>
116                 Share context, errors, and what you already tried
117               </span>
118             </div>
119           </button>
120
121           <button
122             type='button'
123             className={styles.quickTile}
124             onClick={() => navigate('/my-questions')}
125           >
126             <div className={styles.quickTileIconWrap}>
127               <MessageSquare size={20} className={styles.quickTileIcon} />
128             </div>
129             <div className={styles.quickTileContent}>
130               <strong className={styles.quickTileTitle}>Your topics</strong>
131               <span className={styles.quickTileDesc}>
```

```
                Filtered list of threads you authored
              </span>
            </div>
          </button>

          <button
            type='button'
            className={styles.quickTile}
            onClick={() => navigate('/rag-documents')}
          >
            <div className={styles.quickTileIconWrap}>
              <BookOpen size={20} className={styles.quickTileIcon} />
            </div>
            <div className={styles.quickTileContent}>
              <strong className={styles.quickTileTitle}>Knowledge base</strong>
              <span className={styles.quickTileDesc}>
                Course library, uploads, and retrieval-backed context for threads
              </span>
            </div>
          </button>
        </div>

        {/* Stats Row */}
        <div className={styles.statsSection}>
          <p className={styles.statsLabel}>
            Figures below describe the newest threads in this feed (up to 100 from the API).
          </p>
          <div className={styles.statsGrid}>
            <div className={styles.statCard}>
              <span className={styles.statName}>Questions</span>
              <span className={styles.statValue}>{stats.totalQuestions}</span>
            </div>
            <div className={styles.statCard}>
              <span className={styles.statName}>Replies</span>
              <span className={styles.statValue}>{stats.totalReplies}</span>
            </div>
            <div className={styles.statCard}>
              <span className={styles.statName}>Unanswered</span>
              <span className={styles.statValue}>{stats.unanswered}</span>
            </div>
            <div className={styles.statCard}>
              <span className={styles.statName}>Yours</span>
              <span className={styles.statValue}>{stats.authoredByMe}</span>
            </div>
          </div>
        </div>
      </section>

      {/* Discussion Feed Section */}
      <section className={styles.feedSection} aria-label='Discussion Feed'>
        <div className={styles.feedHeader}>
          <div>
            <h2 className={styles.feedTitle}>Discussion feed</h2>
            <p className={styles.feedSubtitle}>
              Your threads use a slim left accent in this list.
            </p>
          </div>

          <div className={styles.feedActions}>
            {isSearchActive ? (
              <div className={styles.searchPill}>
                {semanticQuery ? (
                  <>
                    <Sparkles size={14} className={styles.semanticIcon} />
                    <span>AI Semantic: "{semanticQuery}"</span>
                  </>
                ) : (
                  <>
                    <Search size={14} />
                    <span>Keyword: "{keywordQuery}"</span>
```

```
                  </>
                )}
                <button
                  type='button'
                  onClick={clearSearch}
                  className={styles.clearSearchBtn}
                  aria-label='Clear search filter'
                >
                  <X size={14} />
                </button>
              </div>
            ) : (
              <span className={styles.newestBadge}>NEWEST THREADS</span>
            )}
          </div>
        </div>

        {/* Loading State */}
        {loading && (
          <div className={styles.loadingContainer}>
            {[1, 2, 3, 4].map(idx => (
              <div key={idx} className={styles.skeletonCard}>
                <div className={styles.skeletonAvatar} />
                <div className={styles.skeletonContent}>
                  <div className={styles.skeletonTitle} />
                  <div className={styles.skeletonText} />
                  <div className={styles.skeletonFooter} />
                </div>
              </div>
            ))}
          </div>
        )}

        {/* Error State */}
        {!loading && error && (
          <div className={styles.errorBanner} role='alert'>
            <AlertCircle size={20} className={styles.errorIcon} />
            <div className={styles.errorContent}>
              <p className={styles.errorMessage}>{error}</p>
              <button
                type='button'
                onClick={fetchQuestions}
                className={styles.retryBtn}
              >
                <RefreshCw size={14} />
                Try again
              </button>
            </div>
          </div>
        )}

        {/* Empty State */}
        {!loading && !error && questions.length === 0 && (
          <div className={styles.emptyCard}>
            <div className={styles.emptyIconWrap}>
              <MessageSquare size={32} />
            </div>
            <h3 className={styles.emptyTitle}>
              {isSearchActive ? 'No matching questions found' : 'No questions yet'}
            </h3>
            <p className={styles.emptyText}>
              {isSearchActive
                ? `We couldn't find any threads matching "${keywordQuery || semanticQuery}". Try adjusting your terms or browse the complete feed.`
                : 'Be the first in your cohort to ask a question! Clear questions with context help everyone learn.'}
            </p>
            <div className={styles.emptyActions}>
              {isSearchActive ? (
                <button
                  type='button'
```

```
                  onClick={clearSearch}
                  className={styles.emptySecondaryBtn}
                >
                  Show all questions
                </button>
              ) : (
                <button
                  type='button'
                  onClick={() => navigate('/questions/ask')}
                  className={styles.emptyPrimaryBtn}
                >
                  Ask a Question
                  <ArrowRight size={16} />
                </button>
              )}
            </div>
          </div>
        )}

        {/* Question List */}
        {!loading && !error && questions.length > 0 && (
          <div className={styles.questionList}>
            {questions.map(q => (
              <QuestionCard
                key={q.questionHash || q.id}
                question={q}
                currentUser={user}
              />
            ))}
          </div>
        )}
      </section>
    </div>
  );
}

```

PAGE · CSS frontend/src/pages/Dashboard/Dashboard.module.css

Dashboard Page Styles

Search bar controls, semantic search toggle, and feed list layout.

Lines: 437 Size: 8.3 KB Format: .CSS

```css
.dashboard {
  display: flex;
  flex-direction: column;
  gap: var(--spacing-xl, 2rem);
  width: 100%;
}

/* Hero Section */
.heroCard {
  background-color: var(--surface, #ffffff);
  border: 1px solid var(--border, #e2e8f0);
  border-radius: var(--radius-xl, 1rem);
  padding: 1.75rem 2rem;
  display: flex;
  flex-direction: column;
  gap: 1.5rem;
}

.heroHeader {
  display: flex;
  flex-direction: column;
  gap: 0.35rem;
}

.heroEyebrow {
  font-size: 0.6875rem;
  font-weight: 700;
  letter-spacing: 0.08em;
  color: var(--primary-dark, #c2410c);
}

.heroTitle {
  font-size: 1.625rem;
  font-weight: 700;
  color: var(--text-primary, #0f172a);
  margin: 0;
  letter-spacing: -0.02em;
}

.heroSubtitle {
  font-size: 0.9375rem;
  color: var(--text-secondary, #475569);
  margin: 0;
  line-height: 1.5;
}

/* Quick Action Tiles */
.quickTiles {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 1rem;
}

@media (max-width: 900px) {
  .quickTiles {
    grid-template-columns: 1fr;
  }
}

.quickTile {
  display: flex;
```

```
  align-items: flex-start;
  gap: 1rem;
  padding: 1.125rem 1.25rem;
  background-color: var(--surface-container-low, #f8fafc);
  border: 1px solid var(--border, #e2e8f0);
  border-radius: var(--radius-lg, 0.75rem);
  text-align: left;
  cursor: pointer;
  transition: all 0.15s ease;
}

.quickTile:hover {
  background-color: var(--surface, #ffffff);
  border-color: var(--primary, #f97316);
  box-shadow: var(--shadow-sm, 0 1px 2px rgba(0, 0, 0, 0.05));
  transform: translateY(-1px);
}

.quickTileIconWrap {
  width: 2.5rem;
  height: 2.5rem;
  border-radius: var(--radius-md, 0.5rem);
  background-color: var(--surface, #ffffff);
  border: 1px solid var(--border, #e2e8f0);
  display: flex;
  align-items: center;
  justify-content: center;
  color: var(--primary, #f97316);
  flex-shrink: 0;
}

.quickTileContent {
  display: flex;
  flex-direction: column;
  gap: 0.25rem;
}

.quickTileTitle {
  font-size: 0.9375rem;
  font-weight: 600;
  color: var(--text-primary, #0f172a);
}

.quickTileDesc {
  font-size: 0.8125rem;
  color: var(--text-tertiary, #64748b);
  line-height: 1.4;
}

/* Stats Row */
.statsSection {
  border-top: 1px solid var(--border, #e2e8f0);
  padding-top: 1.25rem;
  display: flex;
  flex-direction: column;
  gap: 0.75rem;
}

.statsLabel {
  font-size: 0.8125rem;
  color: var(--text-tertiary, #64748b);
  margin: 0;
}

.statsGrid {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 1rem;
}

```

```
@media (max-width: 680px) {
  .statsGrid {
    grid-template-columns: repeat(2, 1fr);
  }
}

.statCard {
  background-color: var(--surface-container-low, #f8fafc);
  border: 1px solid var(--border, #e2e8f0);
  border-radius: var(--radius-md, 0.5rem);
  padding: 0.875rem 1rem;
  display: flex;
  flex-direction: column;
  gap: 0.25rem;
}

.statName {
  font-size: 0.75rem;
  font-weight: 500;
  color: var(--text-tertiary, #64748b);
}

.statValue {
  font-size: 1.5rem;
  font-weight: 700;
  color: var(--text-primary, #0f172a);
  letter-spacing: -0.02em;
}

/* Feed Section */
.feedSection {
  display: flex;
  flex-direction: column;
  gap: 1rem;
}

.feedHeader {
  display: flex;
  align-items: flex-end;
  justify-content: space-between;
  gap: 1rem;
  flex-wrap: wrap;
}

.feedTitle {
  font-size: 1.25rem;
  font-weight: 700;
  color: var(--text-primary, #0f172a);
  margin: 0;
}

.feedSubtitle {
  font-size: 0.875rem;
  color: var(--text-secondary, #475569);
  margin: 0.25rem 0 0 0;
}

.newestBadge {
  font-size: 0.6875rem;
  font-weight: 700;
  letter-spacing: 0.05em;
  padding: 0.35rem 0.75rem;
  background-color: var(--primary, #f97316);
  color: #ffffff;
  border-radius: var(--radius-full, 9999px);
}

.searchPill {
  display: inline-flex;
  align-items: center;
```

```
  gap: 0.5rem;
  padding: 0.35rem 0.75rem;
  background-color: var(--primary-light, #ffedd5);
  color: var(--primary-dark, #c2410c);
  border: 1px solid rgba(249, 115, 22, 0.3);
  border-radius: var(--radius-full, 9999px);
  font-size: 0.8125rem;
  font-weight: 500;
}

.semanticIcon {
  color: var(--primary, #f97316);
}

.clearSearchBtn {
  background: none;
  border: none;
  color: var(--primary-dark, #c2410c);
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 0.15rem;
  cursor: pointer;
  border-radius: var(--radius-full, 9999px);
}

.clearSearchBtn:hover {
  background-color: rgba(0, 0, 0, 0.08);
}

/* Question List */
.questionList {
  display: flex;
  flex-direction: column;
  gap: 0.875rem;
}

/* Loading Skeleton */
.loadingContainer {
  display: flex;
  flex-direction: column;
  gap: 0.875rem;
}

.skeletonCard {
  display: flex;
  align-items: flex-start;
  gap: 1rem;
  padding: 1.25rem 1.5rem;
  background-color: var(--surface, #ffffff);
  border: 1px solid var(--border, #e2e8f0);
  border-radius: var(--radius-lg, 0.75rem);
}

.skeletonAvatar {
  width: 2.5rem;
  height: 2.5rem;
  border-radius: var(--radius-full, 9999px);
  background-color: var(--surface-container-high, #e2e8f0);
  animation: pulse 1.5s infinite ease-in-out;
  flex-shrink: 0;
}

.skeletonContent {
  flex: 1;
  display: flex;
  flex-direction: column;
  gap: 0.75rem;
}

```

```
.skeletonTitle {
  height: 1.25rem;
  width: 55%;
  background-color: var(--surface-container-high, #e2e8f0);
  border-radius: var(--radius-sm, 0.25rem);
  animation: pulse 1.5s infinite ease-in-out;
}

.skeletonText {
  height: 1rem;
  width: 85%;
  background-color: var(--surface-container-high, #e2e8f0);
  border-radius: var(--radius-sm, 0.25rem);
  animation: pulse 1.5s infinite ease-in-out;
}

.skeletonFooter {
  height: 0.875rem;
  width: 30%;
  background-color: var(--surface-container-high, #e2e8f0);
  border-radius: var(--radius-sm, 0.25rem);
  animation: pulse 1.5s infinite ease-in-out;
}

@keyframes pulse {
  0% {
    opacity: 0.6;
  }
  50% {
    opacity: 1;
  }
  100% {
    opacity: 0.6;
  }
}

/* Error State */
.errorBanner {
  display: flex;
  align-items: flex-start;
  gap: 0.875rem;
  padding: 1.25rem 1.5rem;
  background-color: var(--error-light, #fee2e2);
  border: 1px solid rgba(239, 68, 68, 0.3);
  border-radius: var(--radius-lg, 0.75rem);
  color: var(--error, #ef4444);
}

.errorIcon {
  flex-shrink: 0;
  margin-top: 0.125rem;
}

.errorContent {
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
}

.errorMessage {
  margin: 0;
  font-size: 0.9375rem;
  font-weight: 500;
  color: #991b1b;
}

.retryBtn {
  display: inline-flex;
  align-items: center;
  gap: 0.35rem;
```

```
  font-size: 0.8125rem;
  font-weight: 600;
  padding: 0.35rem 0.75rem;
  background-color: #ffffff;
  border: 1px solid #f87171;
  border-radius: var(--radius-sm, 0.25rem);
  color: #b91c1c;
  cursor: pointer;
  align-self: flex-start;
}

.retryBtn:hover {
  background-color: #fef2f2;
}

/* Empty State */
.emptyCard {
  padding: 3rem 2rem;
  text-align: center;
  background-color: var(--surface, #ffffff);
  border: 1px dashed var(--border, #e2e8f0);
  border-radius: var(--radius-xl, 1rem);
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  gap: 0.75rem;
  max-width: 36rem;
  margin: 1rem auto;
}

.emptyIconWrap {
  width: 3.5rem;
  height: 3.5rem;
  border-radius: var(--radius-full, 9999px);
  background-color: var(--surface-container-low, #f1f5f9);
  display: flex;
  align-items: center;
  justify-content: center;
  color: var(--text-tertiary, #64748b);
  margin-bottom: 0.5rem;
}

.emptyTitle {
  font-size: 1.125rem;
  font-weight: 600;
  color: var(--text-primary, #0f172a);
  margin: 0;
}

.emptyText {
  font-size: 0.9375rem;
  color: var(--text-secondary, #475569);
  line-height: 1.5;
  margin: 0;
  max-width: 28rem;
}

.emptyActions {
  margin-top: 0.75rem;
}

.emptyPrimaryBtn {
  display: inline-flex;
  align-items: center;
  gap: 0.5rem;
  padding: 0.625rem 1.25rem;
  background-color: var(--primary, #f97316);
  color: #ffffff;
  border: none;
```

```
  border-radius: var(--radius-md, 0.5rem);
  font-size: 0.875rem;
  font-weight: 600;
  cursor: pointer;
  transition: background-color 0.15s ease;
}

.emptyPrimaryBtn:hover {
  background-color: var(--primary-hover, #ea580c);
}

.emptySecondaryBtn {
  padding: 0.5rem 1rem;
  background-color: var(--surface, #ffffff);
  border: 1px solid var(--border, #e2e8f0);
  border-radius: var(--radius-md, 0.5rem);
  color: var(--text-primary, #0f172a);
  font-size: 0.875rem;
  font-weight: 500;
  cursor: pointer;
}

.emptySecondaryBtn:hover {
  background-color: var(--surface-hover, #f8fafc);
}

```

PAGE · POST QUESTION frontend/src/pages/PostQuestion/PostQuestion.jsx

Ask Question Page with AI Draft Coach

Question composition form integrated with real-time Gemini draft coaching suggestions.

Lines: 403 Size: 13.5 KB Format: .JSX

```jsx
import { useState, useRef } from 'react';
import { useNavigate } from 'react-router-dom';
import {
  Sparkles,
  Send,
  Bold,
  Italic,
  Code,
  Link as LinkIcon,
  AlertCircle,
  CheckCircle2,
  X,
  Loader2,
} from 'lucide-react';
import { questionService } from '../../services/question/question.service';
import styles from './PostQuestion.module.css';

export default function PostQuestion() {
  const navigate = useNavigate();
  const textareaRef = useRef(null);

  const [title, setTitle] = useState('');
  const [content, setContent] = useState('');

  // Validation errors
  const [touched, setTouched] = useState({ title: false, content: false });
  const [generalError, setGeneralError] = useState(null);

  // AI Draft Coach state
  const [coachLoading, setCoachLoading] = useState(false);
  const [coachTips, setCoachTips] = useState(null);
  const [coachError, setCoachError] = useState(null);

  // Form submission state
  const [submitting, setSubmitting] = useState(false);

  // Validation rules
  const isTitleValid = title.trim().length >= 5 && title.trim().length <= 255;
  const isContentValid = content.trim().length >= 10;
  const titleError =
    touched.title && !isTitleValid
      ? 'Question title must be at least 5 characters'
      : null;
  const contentError =
    touched.content && !isContentValid
      ? 'Question content must be at least 10 characters'
      : null;

  // Markdown formatting tool helper
  const insertMarkdown = (prefix, suffix = '') => {
    const textarea = textareaRef.current;
    if (!textarea) return;

    const start = textarea.selectionStart;
    const end = textarea.selectionEnd;
    const text = textarea.value;
    const selected = text.substring(start, end);

    let replacement = '';
    if (prefix === '```\n') {
      replacement = `\`\`\`\n${selected || '// code here'}\n\`\`\``;
```

```
    } else if (prefix === '[') {
      replacement = `[${selected || 'link text'}](https://)`;
    } else {
      replacement = `${prefix}${selected || 'text'}${suffix}`;
    }

    const nextContent =
      text.substring(0, start) + replacement + text.substring(end);
    setContent(nextContent);

    setTimeout(() => {
      textarea.focus();
      textarea.setSelectionRange(
        start + prefix.length,
        start + prefix.length + (selected.length || (prefix === '```\n' ? 12 : 4)),
      );
    }, 0);
  };

  // Run AI Draft Coach
  const handleGetCoachFeedback = async () => {
    setCoachError(null);
    if (title.trim().length < 5 || content.trim().length < 10) {
      setTouched({ title: true, content: true });
      setCoachError(
        'Please enter a title (min 5 chars) and draft description (min 10 chars) so the AI Coach can review it.',
      );
      return;
    }

    setCoachLoading(true);
    try {
      const res = await questionService.generateQuestionDraftCoach({
        title: title.trim(),
        content: content.trim(),
      });
      setCoachTips(res.tips);
    } catch (err) {
      console.error('AI coach error:', err);
      setCoachError(
        err.message || 'AI coach is temporarily unavailable. You can still post your question.',
      );
    } finally {
      setCoachLoading(false);
    }
  };

  // Submit question
  const handleSubmit = async e => {
    e.preventDefault();
    setTouched({ title: true, content: true });
    setGeneralError(null);

    if (!isTitleValid || !isContentValid) {
      return;
    }

    setSubmitting(true);
    try {
      const created = await questionService.createQuestion({
        title: title.trim(),
        content: content.trim(),
      });

      const hash = created?.questionHash || created?.id;
      if (hash) {
        navigate(`/questions/${hash}`);
      } else {
        navigate('/dashboard');
      }
```

```
    } catch (err) {
      console.error('Failed to post question:', err);
      setGeneralError(
        err.message || 'Failed to publish your question. Please check your connection and try again.',
      );
    } finally {
      setSubmitting(false);
    }
  };

  return (
    <div className={styles.container}>
      {/* Top Guidance Card */}
      <section className={styles.guidanceCard} aria-labelledby='ask-cohort-heading'>
        <div className={styles.guidanceHeader}>
          <span className={styles.guidanceEyebrow}>ASK THE COHORT</span>
          <h1 id='ask-cohort-heading' className={styles.guidanceTitle}>
            Publish to the forum
          </h1>
          <p className={styles.guidanceLead}>
            Public threads help the whole cohort. Write as if a classmate will debug
            your issue tomorrow. They only know what you put on the page.
          </p>
        </div>

        {/* Checklist Callout */}
        <div className={styles.calloutBox}>
          <h2 className={styles.calloutTitle}>
            Write questions people can answer in one pass
          </h2>
          <p className={styles.calloutText}>
            Mentors volunteer their time. Give them runnable context, expected vs
            actual behavior, and a tight scope so they can reproduce the issue without
            guessing your setup.
          </p>

          <div className={styles.checklistGroup}>
            <h3 className={styles.checklistHeading}>Checklist before you post</h3>
            <ul className={styles.checklist}>
              <li>
                <strong>Title as a headline</strong> that states the symptom and tech stack (e.g., “React 19: state re
sets after navigation”).
              </li>
              <li>
                <strong>Repro steps</strong> numbered, with environment (OS, browser, Node version) when it matters.
              </li>
              <li>
                <strong>Minimal code</strong> in fenced markdown blocks; trim unrelated lines so readers scan faster.
              </li>
              <li>
                <strong>Exact errors</strong> copied verbatim, including stack trace snippets when debugging backend r
outes.
              </li>
            </ul>
          </div>

          <div className={styles.rulesGroup}>
            <h3 className={styles.checklistHeading}>Validation rules (enforced by the form)</h3>
            <ul className={styles.checklist}>
              <li>
                <strong>Title length:</strong> Must be between 5 and 255 characters.
              </li>
              <li>
                <strong>Body length:</strong> Must contain a minimum of 10 characters detailing your problem.
              </li>
              <li>
                <strong>Single topic:</strong> Split unrelated bugs into separate threads so search and embeddings sta
y precise.
              </li>
            </ul>
```

```
            </div>
          </div>
        </section>

        {/* Question Form Card */}
        <section className={styles.formCard}>
          <form className={styles.form} onSubmit={handleSubmit} noValidate>
            {generalError && (
              <div className={styles.errorAlert} role='alert'>
                <AlertCircle size={18} />
                <span>{generalError}</span>
              </div>
            )}

            {/* Title Field */}
            <div className={styles.fieldGroup}>
              <label htmlFor='question-title' className={styles.label}>
                Title
              </label>
              <p className={styles.fieldHelper}>
                Be specific and imagine you're asking a question to another person.
              </p>
              <input
                id='question-title'
                type='text'
                value={title}
                onChange={e => setTitle(e.target.value)}
                onBlur={() => setTouched(prev => ({ ...prev, title: true }))}
                placeholder='e.g. How do I handle state management using Context API in React?'
                className={`${styles.input} ${titleError ? styles['input--error'] : ''}`}
                maxLength={255}
              />
              {titleError && <p className={styles.errorText}>{titleError}</p>}
            </div>

            {/* Content Field with Markdown Toolbar */}
            <div className={styles.fieldGroup}>
              <label htmlFor='question-content' className={styles.label}>
                What are the details of your problem?
              </label>
              <p className={styles.fieldHelper}>
                Introduce the problem and expand on what you put in the title. Minimum 10 characters.
              </p>

              <div className={`${styles.editorWrap} ${contentError ? styles['editorWrap--error'] : ''}`}>
                <div className={styles.toolbar}>
                  <div className={styles.toolbarButtons}>
                    <button
                      type='button'
                      className={styles.toolBtn}
                      onClick={() => insertMarkdown('**', '**')}
                      title='Bold'
                    >
                      <Bold size={15} />
                    </button>
                    <button
                      type='button'
                      className={styles.toolBtn}
                      onClick={() => insertMarkdown('*', '*')}
                      title='Italic'
                    >
                      <Italic size={15} />
                    </button>
                    <button
                      type='button'
                      className={styles.toolBtn}
                      onClick={() => insertMarkdown('```\n', '\n```')}
                      title='Code block'
                    >
                      <Code size={15} />
```

```
                  </button>
                  <button
                    type='button'
                    className={styles.toolBtn}
                    onClick={() => insertMarkdown('[', '](url)')}
                    title='Insert link'
                  >
                    <LinkIcon size={15} />
                  </button>
                </div>

                <span className={styles.charCount}>
                  {content.length} characters
                </span>
              </div>

              <textarea
                id='question-content'
                ref={textareaRef}
                value={content}
                onChange={e => setContent(e.target.value)}
                onBlur={() => setTouched(prev => ({ ...prev, content: true }))}
                rows={12}
                placeholder='Include all the information someone would need to answer your question... You can use Markdown to format your code!'
                className={styles.textarea}
              />
            </div>
            {contentError && <p className={styles.errorText}>{contentError}</p>}
          </div>

          {/* AI Coach Trigger */}
          <div className={styles.aiCoachRow}>
            <button
              type='button'
              onClick={handleGetCoachFeedback}
              disabled={coachLoading || submitting}
              className={styles.coachBtn}
            >
              {coachLoading ? (
                <>
                  <Loader2 size={16} className={styles.spinner} />
                  Analyzing draft...
                </>
              ) : (
                <>
                  <Sparkles size={16} />
                  AI suggestions
                </>
              )}
            </button>
            <span className={styles.coachCaption}>
              Suggestions only. You still choose what to post.
            </span>
          </div>

          {/* AI Coach Feedback Panel */}
          {coachError && (
            <div className={styles.coachErrorBanner}>
              <AlertCircle size={16} />
              <span>{coachError}</span>
              <button
                type='button'
                onClick={() => setCoachError(null)}
                className={styles.dismissBtn}
              >
                <X size={14} />
              </button>
            </div>
          )}
```

```
          {coachTips && coachTips.length > 0 && (
            <div className={styles.coachFeedbackBox}>
              <div className={styles.coachFeedbackHeader}>
                <div className={styles.coachFeedbackTitleWrap}>
                  <Sparkles size={16} className={styles.coachSparkle} />
                  <strong className={styles.coachFeedbackTitle}>
                    AI Draft Coach suggestions:
                  </strong>
                </div>
                <button
                  type='button'
                  onClick={() => setCoachTips(null)}
                  className={styles.dismissBtn}
                  aria-label='Dismiss suggestions'
                >
                  <X size={16} />
                </button>
              </div>

              <ul className={styles.coachTipsList}>
                {coachTips.map((tip, idx) => (
                  <li key={idx} className={styles.coachTipItem}>
                    <CheckCircle2 size={15} className={styles.tipCheckIcon} />
                    <span>{tip}</span>
                  </li>
                ))}
              </ul>
            </div>
          )}

          {/* Form Action Buttons */}
          <div className={styles.actionRow}>
            <button
              type='button'
              className={styles.cancelBtn}
              onClick={() => navigate('/dashboard')}
              disabled={submitting}
            >
              Cancel
            </button>

            <button
              type='submit'
              className={styles.submitBtn}
              disabled={submitting}
            >
              {submitting ? (
                <>
                  <Loader2 size={16} className={styles.spinner} />
                  Posting...
                </>
              ) : (
                <>
                  Post Question
                  <Send size={16} />
                </>
              )}
            </button>
          </div>
        </form>
      </section>
    </div>
  );
}
```

PAGE · CSS frontend/src/pages/PostQuestion/PostQuestion.module.css

Post Question Styles

Draft coach suggestion card, textarea layout, and submit button styles.

Lines: 417 Size: 7.8 KB Format: .CSS

```css
.container {
  display: flex;
  flex-direction: column;
  gap: var(--spacing-xl, 2rem);
  width: 100%;
}

/* Guidance Section */
.guidanceCard {
  background-color: var(--surface, #ffffff);
  border: 1px solid var(--border, #e2e8f0);
  border-radius: var(--radius-xl, 1rem);
  padding: 1.75rem 2rem;
  display: flex;
  flex-direction: column;
  gap: 1.5rem;
}

.guidanceHeader {
  display: flex;
  flex-direction: column;
  gap: 0.35rem;
}

.guidanceEyebrow {
  font-size: 0.6875rem;
  font-weight: 700;
  letter-spacing: 0.08em;
  color: var(--primary-dark, #c2410c);
}

.guidanceTitle {
  font-size: 1.625rem;
  font-weight: 700;
  color: var(--text-primary, #0f172a);
  margin: 0;
  letter-spacing: -0.02em;
}

.guidanceLead {
  font-size: 0.9375rem;
  color: var(--text-secondary, #475569);
  margin: 0;
  line-height: 1.5;
}

/* Callout Box */
.calloutBox {
  background-color: var(--surface-container-low, #f8fafc);
  border: 1px solid var(--border, #e2e8f0);
  border-left: 4px solid var(--primary, #f97316);
  border-radius: var(--radius-lg, 0.75rem);
  padding: 1.5rem 1.75rem;
  display: flex;
  flex-direction: column;
  gap: 1rem;
}

.calloutTitle {
  font-size: 1.0625rem;
  font-weight: 700;
```

```
  color: var(--text-primary, #0f172a);
  margin: 0;
}

.calloutText {
  font-size: 0.875rem;
  color: var(--text-secondary, #475569);
  margin: 0;
  line-height: 1.5;
}

.checklistHeading {
  font-size: 0.875rem;
  font-weight: 600;
  color: var(--text-primary, #0f172a);
  margin: 0 0 0.5rem 0;
}

.checklist {
  margin: 0;
  padding-left: 1.25rem;
  display: flex;
  flex-direction: column;
  gap: 0.375rem;
  font-size: 0.8125rem;
  color: var(--text-secondary, #475569);
  line-height: 1.5;
}

.checklist li strong {
  color: var(--text-primary, #0f172a);
}

/* Form Card */
.formCard {
  background-color: var(--surface, #ffffff);
  border: 1px solid var(--border, #e2e8f0);
  border-radius: var(--radius-xl, 1rem);
  padding: 2rem;
}

.form {
  display: flex;
  flex-direction: column;
  gap: 1.5rem;
}

.fieldGroup {
  display: flex;
  flex-direction: column;
  gap: 0.375rem;
}

.label {
  font-size: 0.9375rem;
  font-weight: 600;
  color: var(--text-primary, #0f172a);
}

.fieldHelper {
  font-size: 0.8125rem;
  color: var(--text-tertiary, #64748b);
  margin: 0 0 0.375rem 0;
}

.input {
  width: 100%;
  padding: 0.75rem 1rem;
  font-size: 0.9375rem;
  color: var(--text-primary, #0f172a);
```

```
  background-color: #ffffff;
  border: 1px solid var(--border, #cbd5e1);
  border-radius: var(--radius-md, 0.5rem);
  transition: border-color 0.15s ease, box-shadow 0.15s ease;
  outline: none;
  font-family: inherit;
}

.input:focus {
  border-color: var(--primary, #f97316);
  box-shadow: 0 0 0 3px rgba(249, 115, 22, 0.15);
}

.input--error {
  border-color: var(--error, #ef4444);
}

.errorText {
  font-size: 0.8125rem;
  color: var(--error, #ef4444);
  margin: 0.25rem 0 0 0;
  font-weight: 500;
}

/* Markdown Toolbar & Textarea */
.editorWrap {
  border: 1px solid var(--border, #cbd5e1);
  border-radius: var(--radius-md, 0.5rem);
  overflow: hidden;
  transition: border-color 0.15s ease, box-shadow 0.15s ease;
  background-color: #ffffff;
}

.editorWrap:focus-within {
  border-color: var(--primary, #f97316);
  box-shadow: 0 0 0 3px rgba(249, 115, 22, 0.15);
}

.editorWrap--error {
  border-color: var(--error, #ef4444);
}

.toolbar {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 0.5rem 0.75rem;
  background-color: var(--surface-container-low, #f8fafc);
  border-bottom: 1px solid var(--border, #e2e8f0);
}

.toolbarButtons {
  display: flex;
  align-items: center;
  gap: 0.25rem;
}

.toolBtn {
  background: none;
  border: none;
  padding: 0.35rem 0.5rem;
  border-radius: var(--radius-sm, 0.25rem);
  color: var(--text-secondary, #475569);
  cursor: pointer;
  display: flex;
  align-items: center;
  justify-content: center;
  transition: background-color 0.15s ease;
}

```

```
.toolBtn:hover {
  background-color: var(--surface-container-high, #e2e8f0);
  color: var(--text-primary, #0f172a);
}

.charCount {
  font-size: 0.75rem;
  color: var(--text-tertiary, #64748b);
}

.textarea {
  width: 100%;
  padding: 0.875rem 1rem;
  font-size: 0.9375rem;
  color: var(--text-primary, #0f172a);
  background-color: transparent;
  border: none;
  outline: none;
  resize: vertical;
  min-height: 14rem;
  font-family: inherit;
  line-height: 1.6;
}

/* AI Coach Row */
.aiCoachRow {
  display: flex;
  align-items: center;
  gap: 0.75rem;
  flex-wrap: wrap;
}

.coachBtn {
  display: inline-flex;
  align-items: center;
  gap: 0.4rem;
  padding: 0.5rem 0.875rem;
  background-color: var(--surface-container-low, #f1f5f9);
  border: 1px solid var(--border, #cbd5e1);
  border-radius: var(--radius-md, 0.5rem);
  font-size: 0.8125rem;
  font-weight: 600;
  color: var(--text-secondary, #334155);
  cursor: pointer;
  transition: all 0.15s ease;
}

.coachBtn:hover:not(:disabled) {
  background-color: var(--primary-light, #ffedd5);
  border-color: var(--primary, #f97316);
  color: var(--primary-dark, #c2410c);
}

.coachBtn:disabled {
  opacity: 0.6;
  cursor: not-allowed;
}

.coachCaption {
  font-size: 0.8125rem;
  color: var(--text-tertiary, #64748b);
}

.spinner {
  animation: spin 1s linear infinite;
}

@keyframes spin {
  from {
    transform: rotate(0deg);
```

```
  }
  to {
    transform: rotate(360deg);
  }
}

/* AI Coach Feedback Box */
.coachFeedbackBox {
  background-color: #f0fdf4;
  border: 1px solid #bbf7d0;
  border-radius: var(--radius-lg, 0.75rem);
  padding: 1.25rem 1.5rem;
  display: flex;
  flex-direction: column;
  gap: 0.75rem;
}

.coachFeedbackHeader {
  display: flex;
  align-items: center;
  justify-content: space-between;
}

.coachFeedbackTitleWrap {
  display: flex;
  align-items: center;
  gap: 0.5rem;
}

.coachSparkle {
  color: #16a34a;
}

.coachFeedbackTitle {
  font-size: 0.875rem;
  color: #166534;
}

.coachTipsList {
  margin: 0;
  padding: 0;
  list-style: none;
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
}

.coachTipItem {
  display: flex;
  align-items: flex-start;
  gap: 0.5rem;
  font-size: 0.8125rem;
  color: #14532d;
  line-height: 1.4;
}

.tipCheckIcon {
  color: #16a34a;
  flex-shrink: 0;
  margin-top: 0.125rem;
}

.coachErrorBanner {
  display: flex;
  align-items: center;
  gap: 0.5rem;
  padding: 0.75rem 1rem;
  background-color: var(--error-light, #fee2e2);
  border: 1px solid rgba(239, 68, 68, 0.3);
  border-radius: var(--radius-md, 0.5rem);
```

```
  color: #b91c1c;
  font-size: 0.8125rem;
}

.dismissBtn {
  margin-left: auto;
  background: none;
  border: none;
  color: inherit;
  cursor: pointer;
  padding: 0.2rem;
  display: flex;
  align-items: center;
}

/* Action Buttons */
.actionRow {
  display: flex;
  align-items: center;
  justify-content: flex-end;
  gap: 1rem;
  margin-top: 1rem;
}

.cancelBtn {
  padding: 0.625rem 1.25rem;
  background: none;
  border: none;
  color: var(--text-secondary, #475569);
  font-size: 0.875rem;
  font-weight: 500;
  cursor: pointer;
  border-radius: var(--radius-md, 0.5rem);
  transition: color 0.15s ease;
}

.cancelBtn:hover:not(:disabled) {
  color: var(--text-primary, #0f172a);
}

.submitBtn {
  display: inline-flex;
  align-items: center;
  gap: 0.5rem;
  padding: 0.625rem 1.5rem;
  background-color: var(--primary, #f97316);
  color: #ffffff;
  border: none;
  border-radius: var(--radius-md, 0.5rem);
  font-size: 0.875rem;
  font-weight: 600;
  cursor: pointer;
  transition: background-color 0.15s ease;
}

.submitBtn:hover:not(:disabled) {
  background-color: var(--primary-hover, #ea580c);
}

.submitBtn:disabled {
  opacity: 0.6;
  cursor: not-allowed;
}

.errorAlert {
  display: flex;
  align-items: center;
  gap: 0.5rem;
  padding: 0.875rem 1rem;
  background-color: var(--error-light, #fee2e2);
```

```
  border: 1px solid rgba(239, 68, 68, 0.3);
  border-radius: var(--radius-md, 0.5rem);
  color: var(--error, #ef4444);
  font-size: 0.875rem;
}

```

PAGE · QUESTION DETAIL frontend/src/pages/QuestionDetail/QuestionDetail.jsx

Question Detail & Answer Workspace

Full question viewer, list of answers, answer form with AI fit assessment, and similar question recommendations.

Lines: 605 Size: 20.6 KB Format: .JSX

```jsx
import { useState, useEffect, useRef, useCallback } from 'react';
import { useParams, useNavigate, Link } from 'react-router-dom';
import {
  ArrowLeft,
  Share2,
  MessageSquare,
  Sparkles,
  Send,
  Bold,
  Italic,
  Code,
  Link as LinkIcon,
  Check,
  AlertCircle,
  Loader2,
} from 'lucide-react';
import ReactMarkdown from 'react-markdown';
import { useAuth } from '../../contexts/AuthContext';
import { questionService } from '../../services/question/question.service';
import { answerService } from '../../services/answer/answer.service';
import {
  timeAgo,
  formatDate,
  getInitials,
  getAvatarColor,
  isAuthoredByUser,
} from '../../lib/utils';
import RagAnswerBody from '../../components/RagAnswerBody/RagAnswerBody';
import styles from './QuestionDetail.module.css';

export default function QuestionDetail() {
  const { questionHash, id } = useParams();
  const activeHash = questionHash || id;
  const navigate = useNavigate();
  const { user } = useAuth();
  const textareaRef = useRef(null);

  const [question, setQuestion] = useState(null);
  const [answers, setAnswers] = useState([]);
  const [relatedQuestions, setRelatedQuestions] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  // New Answer Form state
  const [answerContent, setAnswerContent] = useState('');
  const [submittingAnswer, setSubmittingAnswer] = useState(false);
  const [submitError, setSubmitError] = useState(null);
  const [copiedShare, setCopiedShare] = useState(false);

  // AI Answer Fit state
  const [fitChecking, setFitChecking] = useState(false);
  const [fitResult, setFitResult] = useState(null);
  const [fitError, setFitError] = useState(null);

  const fetchQuestionDetails = useCallback(async () => {
    if (!activeHash) return;
    setLoading(true);
    setError(null);
    try {
      const data = await questionService.getSingleQuestion(activeHash);
      setQuestion(data.question);
```

```
      setAnswers(data.answers || []);

      // Fetch related questions in parallel
      if (data.question) {
        questionService
          .findSimilarQuestions(activeHash, data.question.title)
          .then(related => {
            const filtered = related.filter(
              r => (r.questionHash || r.id) !== activeHash,
            );
            setRelatedQuestions(filtered.slice(0, 5));
          })
          .catch(() => setRelatedQuestions([]));
      }
    } catch (err) {
      console.error('Failed to load question detail:', err);
      setError(err.message || 'Unable to load question details.');
    } finally {
      setLoading(false);
    }
  }, [activeHash]);

  useEffect(() => {
    fetchQuestionDetails();
    setAnswerContent('');
    setFitResult(null);
    setFitError(null);
    setSubmitError(null);
  }, [fetchQuestionDetails]);

  // Handle Share link copy
  const handleShare = async () => {
    try {
      await navigator.clipboard.writeText(window.location.href);
      setCopiedShare(true);
      setTimeout(() => setCopiedShare(false), 2000);
    } catch {
      // Fallback
      setCopiedShare(false);
    }
  };

  // Markdown insertion helper
  const insertMarkdown = (prefix, suffix = '') => {
    const textarea = textareaRef.current;
    if (!textarea) return;

    const start = textarea.selectionStart;
    const end = textarea.selectionEnd;
    const text = textarea.value;
    const selected = text.substring(start, end);

    let replacement = '';
    if (prefix === '```\n') {
      replacement = `\`\`\`\n${selected || '// code here'}\n\`\`\``;
    } else if (prefix === '[') {
      replacement = `[${selected || 'link text'}](https://)`;
    } else {
      replacement = `${prefix}${selected || 'text'}${suffix}`;
    }

    const nextContent =
      text.substring(0, start) + replacement + text.substring(end);
    setAnswerContent(nextContent);

    setTimeout(() => {
      textarea.focus();
      textarea.setSelectionRange(
        start + prefix.length,
        start + prefix.length + (selected.length || (prefix === '```\n' ? 12 : 4)),
```

```
      );
    }, 0);
  };

  // Check Answer Fit with AI
  const handleCheckFit = async () => {
    setFitError(null);
    if (answerContent.trim().length < 20) {
      setFitError('You need at least 20 characters before evaluating answer fit.');
      return;
    }

    setFitChecking(true);
    try {
      const result = await questionService.assessAnswerFit(
        activeHash,
        answerContent.trim(),
      );
      setFitResult(result);
    } catch (err) {
      console.error('Fit assessment error:', err);
      setFitError(
        err.message || 'AI fit check is temporarily unavailable. You can still post your answer.',
      );
    } finally {
      setFitChecking(false);
    }
  };

  // Submit Answer
  const handleSubmitAnswer = async e => {
    e.preventDefault();
    setSubmitError(null);

    if (answerContent.trim().length < 20) {
      setSubmitError('Answer must be at least 20 characters long.');
      return;
    }

    setSubmittingAnswer(true);
    try {
      const questionId = question?.id || activeHash;
      const createdAnswer = await answerService.postAnswer({
        questionId,
        content: answerContent.trim(),
      });

      // Append answer to list or refresh
      if (createdAnswer) {
        setAnswers(prev => [...prev, createdAnswer]);
      } else {
        await fetchQuestionDetails();
      }

      // Reset form
      setAnswerContent('');
      setFitResult(null);
    } catch (err) {
      console.error('Failed to post answer:', err);
      setSubmitError(
        err.message || 'Failed to submit answer. Please check your connection and try again.',
      );
    } finally {
      setSubmittingAnswer(false);
    }
  };

  const isOwner = isAuthoredByUser(question, user);

  if (loading) {
```

```
    return (
      <div className={styles.container}>
        <div className={styles.backLinkWrap}>
          <button
            type='button'
            className={styles.backBtn}
            onClick={() => navigate('/dashboard')}
          >
            <ArrowLeft size={16} />
            Back to feed
          </button>
        </div>

        <div className={styles.layoutColumns}>
          <div className={styles.mainColumn}>
            <div className={styles.skeletonCard}>
              <div className={styles.skeletonHeader} />
              <div className={styles.skeletonTitle} />
              <div className={styles.skeletonBody} />
            </div>
          </div>
          <div className={styles.sideColumn}>
            <div className={styles.skeletonSidebar} />
          </div>
        </div>
      </div>
    );
  }

  if (error || !question) {
    return (
      <div className={styles.container}>
        <div className={styles.backLinkWrap}>
          <button
            type='button'
            className={styles.backBtn}
            onClick={() => navigate('/dashboard')}
          >
            <ArrowLeft size={16} />
            Back to feed
          </button>
        </div>

        <div className={styles.errorCard}>
          <AlertCircle size={32} className={styles.errorIcon} />
          <h2 className={styles.errorTitle}>Question Not Found</h2>
          <p className={styles.errorMessage}>
            {error || 'This question might have been deleted or the link is invalid.'}
          </p>
          <button
            type='button'
            className={styles.primaryBtn}
            onClick={() => navigate('/dashboard')}
          >
            Browse all questions
          </button>
        </div>
      </div>
    );
  }

  const authorFirst = question.author?.firstName || question.firstName || 'Anonymous';
  const authorLast = question.author?.lastName || question.lastName || '';
  const authorInitials = getInitials(authorFirst, authorLast);
  const authorTheme = getAvatarColor(authorFirst + authorLast + (question.author?.id || ''));
  const postedDate = question.createdAt
    ? `Posted ${formatDate(question.createdAt)}`
    : 'Recently posted';

  return (
```

```
    <div className={styles.container}>
      {/* Back to feed Navigation */}
      <div className={styles.backLinkWrap}>
        <button
          type='button'
          className={styles.backBtn}
          onClick={() => navigate('/dashboard')}
        >
          <ArrowLeft size={16} />
          Back to feed
        </button>
      </div>

      <div className={styles.layoutColumns}>
        {/* Main Left Column */}
        <div className={styles.mainColumn}>
          {/* Main Question Card */}
          <article className={styles.questionCard} aria-labelledby='question-title'>
            <div className={styles.authorRow}>
              <div
                className={styles.avatar}
                style={{ backgroundColor: authorTheme.bg, color: authorTheme.color }}
                aria-hidden
              >
                {authorInitials}
              </div>
              <div className={styles.authorInfo}>
                <strong className={styles.authorName}>
                  {authorFirst} {authorLast}
                </strong>
                <span className={styles.postedDate}>{postedDate}</span>
              </div>
            </div>

            <h1 id='question-title' className={styles.questionTitle}>
              {question.title}
            </h1>

            <div className={styles.questionContent}>
              <RagAnswerBody>{question.content}</RagAnswerBody>
            </div>

            <div className={styles.questionActions}>
              <button
                type='button'
                className={styles.shareBtn}
                onClick={handleShare}
                title='Share link to this question'
              >
                {copiedShare ? (
                  <>
                    <Check size={15} className={styles.shareSuccessIcon} />
                    <span>Copied link</span>
                  </>
                ) : (
                  <>
                    <Share2 size={15} />
                    <span>Share</span>
                  </>
                )}
              </button>

              <span className={styles.answerCountBadge}>
                <MessageSquare size={15} />
                <span>
                  {answers.length} {answers.length === 1 ? 'Answer' : 'Answers'}
                </span>
              </span>
            </div>
          </article>
```

```
          {/* Community Answers Section */}
          <section className={styles.answersSection} aria-label='Community Answers'>
            <h2 className={styles.answersHeading}>
              Community Answers ({answers.length})
            </h2>

            {/* Empty Answers State */}
            {answers.length === 0 ? (
              <div className={styles.emptyAnswersCard}>
                <div className={styles.emptyIconWrap}>
                  <MessageSquare size={28} />
                </div>
                <strong className={styles.emptyAnswersTitle}>
                  Be the first to help!
                </strong>
                <p className={styles.emptyAnswersText}>
                  This question is waiting for an expert like you. Share your knowledge and earn
                  reputation points.
                </p>
              </div>
            ) : (
              <div className={styles.answersList}>
                {answers.map((answer, index) => {
                  const ansFirst =
                    answer.author?.firstName || answer.firstName || 'Community';
                  const ansLast = answer.author?.lastName || answer.lastName || 'Member';
                  const ansInitials = getInitials(ansFirst, ansLast);
                  const ansTheme = getAvatarColor(ansFirst + ansLast + (answer.author?.id || index));
                  const ansDate = answer.createdAt
                    ? formatDate(answer.createdAt)
                    : timeAgo(answer.createdAt);

                  return (
                    <article key={answer.id || index} className={styles.answerCard}>
                      <div className={styles.answerHeader}>
                        <div
                          className={styles.avatarSm}
                          style={{
                            backgroundColor: ansTheme.bg,
                            color: ansTheme.color,
                          }}
                          aria-hidden
                        >
                          {ansInitials}
                        </div>
                        <div className={styles.authorInfo}>
                          <strong className={styles.authorName}>
                            {ansFirst} {ansLast}
                          </strong>
                          <span className={styles.postedDate}>{ansDate}</span>
                        </div>
                      </div>

                      <div className={styles.answerBody}>
                        <RagAnswerBody>{answer.content}</RagAnswerBody>
                      </div>
                    </article>
                  );
                })}
              </div>
            )}

            {/* Answer Form or Author Notice */}
            {isOwner ? (
              <div className={styles.ownerNoticeBox}>
                <AlertCircle size={18} className={styles.noticeIcon} />
                <div>
                  <strong className={styles.noticeTitle}>
                    You cannot answer your own question
```

```
                    </strong>
                    <p className={styles.noticeText}>
                      To clarify your question or add new details, you can ask a follow-up or
                      participate in the thread replies.
                    </p>
                  </div>
                </div>
              ) : (
                <div className={styles.contributeCard}>
                  <h3 className={styles.contributeTitle}>Contribute an answer</h3>

                  <form onSubmit={handleSubmitAnswer} className={styles.answerForm}>
                    {submitError && (
                      <div className={styles.errorBanner} role='alert'>
                        <AlertCircle size={16} />
                        <span>{submitError}</span>
                      </div>
                    )}

                    <div className={styles.editorWrap}>
                      <div className={styles.toolbar}>
                        <div className={styles.toolbarButtons}>
                          <button
                            type='button'
                            className={styles.toolBtn}
                            onClick={() => insertMarkdown('**', '**')}
                            title='Bold'
                          >
                            <Bold size={15} />
                          </button>
                          <button
                            type='button'
                            className={styles.toolBtn}
                            onClick={() => insertMarkdown('*', '*')}
                            title='Italic'
                          >
                            <Italic size={15} />
                          </button>
                          <button
                            type='button'
                            className={styles.toolBtn}
                            onClick={() => insertMarkdown('```\n', '\n```')}
                            title='Code block'
                          >
                            <Code size={15} />
                          </button>
                          <button
                            type='button'
                            className={styles.toolBtn}
                            onClick={() => insertMarkdown('[', '](url)')}
                            title='Insert link'
                          >
                            <LinkIcon size={15} />
                          </button>
                        </div>

                        <span className={styles.charCount}>
                          {answerContent.length} characters
                        </span>
                      </div>

                      <textarea
                        ref={textareaRef}
                        value={answerContent}
                        onChange={e => setAnswerContent(e.target.value)}
                        rows={8}
                        placeholder='Type your answer here... You can use Markdown to format your code!'
                        className={styles.answerTextarea}
                      />
                    </div>
```

```
                    {/* AI Answer Fit Row */}
                    <div className={styles.aiFitRow}>
                      <button
                        type='button'
                        onClick={handleCheckFit}
                        disabled={fitChecking || submittingAnswer}
                        className={styles.checkFitBtn}
                      >
                        {fitChecking ? (
                          <>
                            <Loader2 size={15} className={styles.spinner} />
                            Checking fit...
                          </>
                        ) : (
                          <>
                            <Sparkles size={15} />
                            Check draft fit
                          </>
                        )}
                      </button>
                      <span className={styles.fitCaption}>
                        Relevance only. Not grading correctness. You need at least 20 characters.
                      </span>
                    </div>

                    {fitError && (
                      <div className={styles.fitErrorText}>
                        <AlertCircle size={14} />
                        <span>{fitError}</span>
                      </div>
                    )}

                    {/* AI Fit Feedback Panel */}
                    {fitResult && (
                      <div
                        className={`${styles.fitResultBox} ${
                          fitResult.level === 'strong'
                            ? styles['fitResultBox--strong']
                            : fitResult.level === 'weak'
                            ? styles['fitResultBox--weak']
                            : styles['fitResultBox--partial']
                        }`}
                      >
                        <div className={styles.fitResultHeader}>
                          <Sparkles size={16} />
                          <span className={styles.fitBadge}>
                            {fitResult.level === 'strong'
                              ? 'Strong match'
                              : fitResult.level === 'weak'
                              ? 'Weak match'
                              : 'Partial match'}
                          </span>
                        </div>
                        <p className={styles.fitNote}>{fitResult.note}</p>
                      </div>
                    )}

                    {/* Submit Button */}
                    <div className={styles.submitRow}>
                      <button
                        type='submit'
                        disabled={submittingAnswer || answerContent.trim().length < 20}
                        className={styles.submitAnswerBtn}
                      >
                        {submittingAnswer ? (
                          <>
                            <Loader2 size={16} className={styles.spinner} />
                            Posting...
                          </>
```

```
                    ) : (
                      'Post Your Answer'
                    )}
                  </button>
                </div>
              </form>
            </div>
          )}
        </section>
      </div>

      {/* Right Column: Related Questions Sidebar */}
      <aside className={styles.sideColumn} aria-label='Related Questions'>
        <div className={styles.relatedBox}>
          <h3 className={styles.relatedTitle}>Related Questions</h3>

          {relatedQuestions.length === 0 ? (
            <p className={styles.noRelatedText}>
              No related questions found.
            </p>
          ) : (
            <div className={styles.relatedList}>
              {relatedQuestions.map(rel => {
                const relHash = rel.questionHash || rel.id;
                const relFirst = rel.author?.firstName || rel.firstName || 'User';
                const relLast = rel.author?.lastName || rel.lastName || '';
                const relDate = rel.createdAt
                  ? formatDate(rel.createdAt)
                  : '';

                return (
                  <div key={relHash} className={styles.relatedCard}>
                    <Link
                      to={`/questions/${relHash}`}
                      className={styles.relatedCardTitle}
                    >
                      {rel.title}
                    </Link>
                    <div className={styles.relatedMeta}>
                      <span>{`${relFirst} ${relLast}`.trim()}</span>
                      {relDate && <span>{relDate}</span>}
                    </div>
                  </div>
                );
              })}
            </div>
          )}
        </div>
      </aside>
    </div>
  </div>
  );
}

```

PAGE · CSS frontend/src/pages/QuestionDetail/QuestionDetail.module.css

Question Detail Styles

Thread layout, author header, answer fit badge, and similar questions panel.

Lines: 663 Size: 12.3 KB Format: .CSS

```css
.container {
  display: flex;
  flex-direction: column;
  gap: 1.5rem;
  width: 100%;
}

.backLinkWrap {
  display: flex;
  align-items: center;
}

.backBtn {
  display: inline-flex;
  align-items: center;
  gap: 0.5rem;
  background: none;
  border: none;
  color: var(--text-secondary, #475569);
  font-size: 0.875rem;
  font-weight: 500;
  cursor: pointer;
  padding: 0.25rem 0;
  transition: color 0.15s ease;
}

.backBtn:hover {
  color: var(--primary, #f97316);
}

.layoutColumns {
  display: grid;
  grid-template-columns: 1fr 20rem;
  gap: 1.75rem;
  align-items: flex-start;
}

@media (max-width: 960px) {
  .layoutColumns {
    grid-template-columns: 1fr;
  }
}

.mainColumn {
  display: flex;
  flex-direction: column;
  gap: 2rem;
  min-width: 0;
}

/* Question Card */
.questionCard {
  background-color: var(--surface, #ffffff);
  border: 1px solid var(--border, #e2e8f0);
  border-radius: var(--radius-xl, 1rem);
  padding: 2rem;
  display: flex;
  flex-direction: column;
  gap: 1.25rem;
}

```

```
.authorRow {
  display: flex;
  align-items: center;
  gap: 0.875rem;
}

.avatar {
  width: 2.75rem;
  height: 2.75rem;
  border-radius: var(--radius-full, 9999px);
  display: flex;
  align-items: center;
  justify-content: center;
  font-weight: 600;
  font-size: 0.9375rem;
  flex-shrink: 0;
}

.avatarSm {
  width: 2.25rem;
  height: 2.25rem;
  border-radius: var(--radius-full, 9999px);
  display: flex;
  align-items: center;
  justify-content: center;
  font-weight: 600;
  font-size: 0.8125rem;
  flex-shrink: 0;
}

.authorInfo {
  display: flex;
  flex-direction: column;
  gap: 0.125rem;
}

.authorName {
  font-size: 0.9375rem;
  color: var(--text-primary, #0f172a);
}

.postedDate {
  font-size: 0.75rem;
  color: var(--text-tertiary, #64748b);
}

.questionTitle {
  font-size: 1.5rem;
  font-weight: 700;
  line-height: 1.35;
  color: var(--text-primary, #0f172a);
  margin: 0;
  letter-spacing: -0.02em;
}

.questionContent {
  color: var(--text-secondary, #334155);
  line-height: 1.7;
}

.questionActions {
  display: flex;
  align-items: center;
  gap: 0.75rem;
  border-top: 1px solid var(--border, #e2e8f0);
  padding-top: 1.25rem;
  margin-top: 0.5rem;
}

.shareBtn {
```

```
  display: inline-flex;
  align-items: center;
  gap: 0.4rem;
  padding: 0.4rem 0.875rem;
  background-color: var(--surface-container-low, #f8fafc);
  border: 1px solid var(--border, #cbd5e1);
  border-radius: var(--radius-md, 0.5rem);
  color: var(--text-secondary, #475569);
  font-size: 0.8125rem;
  font-weight: 500;
  cursor: pointer;
  transition: all 0.15s ease;
}

.shareBtn:hover {
  background-color: var(--surface, #ffffff);
  border-color: var(--primary, #f97316);
  color: var(--primary, #f97316);
}

.shareSuccessIcon {
  color: var(--success, #10b981);
}

.answerCountBadge {
  display: inline-flex;
  align-items: center;
  gap: 0.4rem;
  padding: 0.4rem 0.875rem;
  background-color: var(--surface-container-low, #f8fafc);
  border: 1px solid var(--border, #cbd5e1);
  border-radius: var(--radius-md, 0.5rem);
  color: var(--text-secondary, #475569);
  font-size: 0.8125rem;
  font-weight: 500;
}

/* Answers Section */
.answersSection {
  display: flex;
  flex-direction: column;
  gap: 1.25rem;
}

.answersHeading {
  font-size: 1.25rem;
  font-weight: 700;
  color: var(--text-primary, #0f172a);
  margin: 0;
}

.answersList {
  display: flex;
  flex-direction: column;
  gap: 1rem;
}

.answerCard {
  background-color: var(--surface, #ffffff);
  border: 1px solid var(--border, #e2e8f0);
  border-radius: var(--radius-xl, 1rem);
  padding: 1.75rem;
  display: flex;
  flex-direction: column;
  gap: 1rem;
}

.answerHeader {
  display: flex;
  align-items: center;
```

```
  gap: 0.75rem;
}

.answerBody {
  color: var(--text-secondary, #334155);
  line-height: 1.7;
}

/* Empty Answers State */
.emptyAnswersCard {
  background-color: var(--surface-container-low, #f8fafc);
  border: 1px dashed var(--border, #e2e8f0);
  border-radius: var(--radius-xl, 1rem);
  padding: 3rem 2rem;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  text-align: center;
  gap: 0.5rem;
}

.emptyIconWrap {
  width: 3.5rem;
  height: 3.5rem;
  border-radius: var(--radius-full, 9999px);
  background-color: #ffffff;
  border: 1px solid var(--border, #e2e8f0);
  display: flex;
  align-items: center;
  justify-content: center;
  color: var(--text-tertiary, #64748b);
  margin-bottom: 0.5rem;
}

.emptyAnswersTitle {
  font-size: 1.0625rem;
  color: var(--text-primary, #0f172a);
}

.emptyAnswersText {
  font-size: 0.875rem;
  color: var(--text-secondary, #475569);
  max-width: 24rem;
  margin: 0;
  line-height: 1.5;
}

/* Owner Notice Box */
.ownerNoticeBox {
  background-color: var(--surface-container-low, #f8fafc);
  border: 1px solid var(--border, #cbd5e1);
  border-radius: var(--radius-lg, 0.75rem);
  padding: 1.25rem 1.5rem;
  display: flex;
  align-items: flex-start;
  gap: 0.875rem;
  color: var(--text-secondary, #475569);
}

.noticeIcon {
  color: var(--primary, #f97316);
  flex-shrink: 0;
  margin-top: 0.125rem;
}

.noticeTitle {
  font-size: 0.9375rem;
  color: var(--text-primary, #0f172a);
}
```

```
.noticeText {
  margin: 0.25rem 0 0 0;
  font-size: 0.8125rem;
  line-height: 1.4;
}

/* Contribute Answer Card */
.contributeCard {
  background-color: var(--surface, #ffffff);
  border: 1px solid var(--border, #e2e8f0);
  border-radius: var(--radius-xl, 1rem);
  padding: 1.75rem 2rem;
  display: flex;
  flex-direction: column;
  gap: 1.25rem;
}

.contributeTitle {
  font-size: 1.125rem;
  font-weight: 700;
  color: var(--text-primary, #0f172a);
  margin: 0;
}

.answerForm {
  display: flex;
  flex-direction: column;
  gap: 1rem;
}

.editorWrap {
  border: 1px solid var(--border, #cbd5e1);
  border-radius: var(--radius-md, 0.5rem);
  overflow: hidden;
  background-color: #ffffff;
}

.editorWrap:focus-within {
  border-color: var(--primary, #f97316);
  box-shadow: 0 0 0 3px rgba(249, 115, 22, 0.15);
}

.toolbar {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 0.5rem 0.75rem;
  background-color: var(--surface-container-low, #f8fafc);
  border-bottom: 1px solid var(--border, #e2e8f0);
}

.toolbarButtons {
  display: flex;
  align-items: center;
  gap: 0.25rem;
}

.toolBtn {
  background: none;
  border: none;
  padding: 0.35rem 0.5rem;
  border-radius: var(--radius-sm, 0.25rem);
  color: var(--text-secondary, #475569);
  cursor: pointer;
  display: flex;
  align-items: center;
  justify-content: center;
}

```

```
.toolBtn:hover {
  background-color: var(--surface-container-high, #e2e8f0);
  color: var(--text-primary, #0f172a);
}

.charCount {
  font-size: 0.75rem;
  color: var(--text-tertiary, #64748b);
}

.answerTextarea {
  width: 100%;
  padding: 0.875rem 1rem;
  font-size: 0.9375rem;
  color: var(--text-primary, #0f172a);
  background-color: transparent;
  border: none;
  outline: none;
  resize: vertical;
  min-height: 8rem;
  font-family: inherit;
  line-height: 1.6;
}

.aiFitRow {
  display: flex;
  align-items: center;
  gap: 0.75rem;
  flex-wrap: wrap;
}

.checkFitBtn {
  display: inline-flex;
  align-items: center;
  gap: 0.4rem;
  padding: 0.45rem 0.875rem;
  background-color: var(--surface-container-low, #f1f5f9);
  border: 1px solid var(--border, #cbd5e1);
  border-radius: var(--radius-md, 0.5rem);
  font-size: 0.8125rem;
  font-weight: 600;
  color: var(--text-secondary, #334155);
  cursor: pointer;
  transition: all 0.15s ease;
}

.checkFitBtn:hover:not(:disabled) {
  background-color: var(--primary-light, #ffedd5);
  border-color: var(--primary, #f97316);
  color: var(--primary-dark, #c2410c);
}

.checkFitBtn:disabled {
  opacity: 0.6;
  cursor: not-allowed;
}

.fitCaption {
  font-size: 0.8125rem;
  color: var(--text-tertiary, #64748b);
}

.fitErrorText {
  display: flex;
  align-items: center;
  gap: 0.35rem;
  font-size: 0.8125rem;
  color: var(--error, #ef4444);
}

```

```css
/* Fit Result Box */
.fitResultBox {
  border-radius: var(--radius-lg, 0.75rem);
  padding: 1rem 1.25rem;
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
}

.fitResultBox--strong {
  background-color: #f0fdf4;
  border: 1px solid #bbf7d0;
  color: #166534;
}

.fitResultBox--partial {
  background-color: #fffbeb;
  border: 1px solid #fde68a;
  color: #92400e;
}

.fitResultBox--weak {
  background-color: #fff1f2;
  border: 1px solid #fecdd3;
  color: #9f1239;
}

.fitResultHeader {
  display: flex;
  align-items: center;
  gap: 0.5rem;
  font-weight: 700;
  font-size: 0.875rem;
}

.fitNote {
  margin: 0;
  font-size: 0.8125rem;
  line-height: 1.5;
}

.submitRow {
  display: flex;
  justify-content: flex-end;
  margin-top: 0.5rem;
}

.submitAnswerBtn {
  padding: 0.625rem 1.5rem;
  background-color: var(--primary, #f97316);
  color: #ffffff;
  border: none;
  border-radius: var(--radius-md, 0.5rem);
  font-size: 0.875rem;
  font-weight: 600;
  cursor: pointer;
  transition: background-color 0.15s ease;
  display: inline-flex;
  align-items: center;
  gap: 0.5rem;
}

.submitAnswerBtn:hover:not(:disabled) {
  background-color: var(--primary-hover, #ea580c);
}

.submitAnswerBtn:disabled {
  opacity: 0.5;
  cursor: not-allowed;
}
```

```css

/* Right Column (Sidebar) */
.sideColumn {
  display: flex;
  flex-direction: column;
  gap: 1rem;
}

.relatedBox {
  background-color: var(--surface, #ffffff);
  border: 1px solid var(--border, #e2e8f0);
  border-radius: var(--radius-xl, 1rem);
  padding: 1.25rem 1.5rem;
  display: flex;
  flex-direction: column;
  gap: 1rem;
}

.relatedTitle {
  font-size: 1rem;
  font-weight: 700;
  color: var(--text-primary, #0f172a);
  margin: 0;
}

.relatedList {
  display: flex;
  flex-direction: column;
  gap: 0.75rem;
}

.relatedCard {
  background-color: var(--surface-container-low, #f8fafc);
  border: 1px solid var(--border, #e2e8f0);
  border-radius: var(--radius-md, 0.5rem);
  padding: 0.875rem 1rem;
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
  transition: border-color 0.15s ease;
}

.relatedCard:hover {
  border-color: var(--primary, #f97316);
}

.relatedCardTitle {
  font-size: 0.875rem;
  font-weight: 600;
  color: var(--primary-dark, #c2410c);
  text-decoration: none;
  line-height: 1.4;
}

.relatedCardTitle:hover {
  text-decoration: underline;
}

.relatedMeta {
  display: flex;
  align-items: center;
  justify-content: space-between;
  font-size: 0.75rem;
  color: var(--text-tertiary, #64748b);
}

.noRelatedText {
  font-size: 0.8125rem;
  color: var(--text-tertiary, #64748b);
  margin: 0;
```

```
}

/* Loading Skeleton */
.skeletonCard {
  background-color: var(--surface, #ffffff);
  border: 1px solid var(--border, #e2e8f0);
  border-radius: var(--radius-xl, 1rem);
  padding: 2rem;
  display: flex;
  flex-direction: column;
  gap: 1.25rem;
}

.skeletonHeader {
  height: 2.5rem;
  width: 40%;
  background-color: var(--surface-container-high, #e2e8f0);
  border-radius: var(--radius-sm, 0.25rem);
  animation: pulse 1.5s infinite ease-in-out;
}

.skeletonTitle {
  height: 2rem;
  width: 75%;
  background-color: var(--surface-container-high, #e2e8f0);
  border-radius: var(--radius-sm, 0.25rem);
  animation: pulse 1.5s infinite ease-in-out;
}

.skeletonBody {
  height: 10rem;
  width: 100%;
  background-color: var(--surface-container-high, #e2e8f0);
  border-radius: var(--radius-sm, 0.25rem);
  animation: pulse 1.5s infinite ease-in-out;
}

.skeletonSidebar {
  background-color: var(--surface, #ffffff);
  border: 1px solid var(--border, #e2e8f0);
  border-radius: var(--radius-xl, 1rem);
  height: 15rem;
  animation: pulse 1.5s infinite ease-in-out;
}

/* Error Card */
.errorCard {
  background-color: var(--surface, #ffffff);
  border: 1px solid var(--border, #e2e8f0);
  border-radius: var(--radius-xl, 1rem);
  padding: 3rem 2rem;
  text-align: center;
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 0.75rem;
}

.errorIcon {
  color: var(--error, #ef4444);
}

.errorTitle {
  font-size: 1.25rem;
  font-weight: 700;
  margin: 0;
  color: var(--text-primary, #0f172a);
}

.errorMessage {
```

```
  font-size: 0.9375rem;
  color: var(--text-secondary, #475569);
  margin: 0;
}

.primaryBtn {
  margin-top: 0.5rem;
  padding: 0.5rem 1.25rem;
  background-color: var(--primary, #f97316);
  color: #ffffff;
  border: none;
  border-radius: var(--radius-md, 0.5rem);
  font-size: 0.875rem;
  font-weight: 600;
  cursor: pointer;
}

.spinner {
  animation: spin 1s linear infinite;
}

@keyframes spin {
  from {
    transform: rotate(0deg);
  }
  to {
    transform: rotate(360deg);
  }
}

@keyframes pulse {
  0% {
    opacity: 0.6;
  }
  50% {
    opacity: 1;
  }
  100% {
    opacity: 0.6;
  }
}

```

PAGE · MY QUESTIONS `frontend/src/pages/MyQuestions/MyQuestions.jsx`

My Questions Page

Lists all questions authored by the currently logged-in user.

Lines: 133 Size: 4.2 KB Format: .JSX

```jsx
import { useState, useEffect, useCallback } from 'react';
import { useNavigate } from 'react-router-dom';
import { Plus, MessageSquare, AlertCircle, RefreshCw, ArrowRight } from 'lucide-react';
import { useAuth } from '../../contexts/AuthContext';
import { questionService } from '../../services/question/question.service';
import QuestionCard from '../../components/QuestionCard/QuestionCard';
import styles from './MyQuestions.module.css';

export default function MyQuestions() {
  const { user } = useAuth();
  const navigate = useNavigate();

  const [questions, setQuestions] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  const fetchMyQuestions = useCallback(async () => {
    setLoading(true);
    setError(null);
    try {
      const res = await questionService.getQuestions({ mine: true });
      setQuestions(res.questions);
    } catch (err) {
      console.error('Failed to fetch my questions:', err);
      setError(err.message || 'Unable to load your topics.');
    } finally {
      setLoading(false);
    }
  }, []);

  useEffect(() => {
    fetchMyQuestions();
  }, [fetchMyQuestions]);

  return (
    <div className={styles.container}>
      {/* Top Header Card */}
      <section className={styles.headerCard} aria-labelledby='my-topics-heading'>
        <div className={styles.headerContent}>
          <span className={styles.headerEyebrow}>YOUR WORKSPACE</span>
          <h1 id='my-topics-heading' className={styles.headerTitle}>
            Your topics
          </h1>
          <p className={styles.headerDesc}>
            Only questions you created. Open one to read answers or add follow-ups.
            Rows use the same left accent as your threads on Home.
          </p>
        </div>

        <button
          type='button'
          className={styles.newQuestionBtn}
          onClick={() => navigate('/questions/ask')}
        >
          <Plus size={18} />
          New question
        </button>
      </section>

      {/* Loading Skeleton */}
      {loading && (
```

```
        <div className={styles.loadingContainer}>
          {[1, 2, 3].map(idx => (
            <div key={idx} className={styles.skeletonCard}>
              <div className={styles.skeletonAvatar} />
              <div className={styles.skeletonContent}>
                <div className={styles.skeletonTitle} />
                <div className={styles.skeletonText} />
                <div className={styles.skeletonFooter} />
              </div>
            </div>
          ))}
        </div>
      )}

      {/* Error State */}
      {!loading && error && (
        <div className={styles.errorBanner} role='alert'>
          <AlertCircle size={20} className={styles.errorIcon} />
          <div className={styles.errorContent}>
            <p className={styles.errorMessage}>{error}</p>
            <button
              type='button'
              onClick={fetchMyQuestions}
              className={styles.retryBtn}
            >
              <RefreshCw size={14} />
              Try again
            </button>
          </div>
        </div>
      )}

      {/* Empty State */}
      {!loading && !error && questions.length === 0 && (
        <div className={styles.emptyCard}>
          <div className={styles.emptyIconWrap}>
            <MessageSquare size={32} />
          </div>
          <h2 className={styles.emptyTitle}>
            You haven't posted any questions yet
          </h2>
          <p className={styles.emptyText}>
            When you ask a question, it will appear here so you can track answers and
            updates in one place.
          </p>
          <button
            type='button'
            onClick={() => navigate('/questions/ask')}
            className={styles.emptyCtaBtn}
          >
            Ask your first question
            <ArrowRight size={16} />
          </button>
        </div>
      )}

      {/* Questions List */}
      {!loading && !error && questions.length > 0 && (
        <div className={styles.questionList}>
          {questions.map(q => (
            <QuestionCard
              key={q.questionHash || q.id}
              question={q}
              currentUser={user}
            />
          ))}
        </div>
      )}
    </div>
  );
```

```
132 }
133
```

PAGE · CSS frontend/src/pages/MyQuestions/MyQuestions.module.css

My Questions Styles

User specific list formatting and empty state styling.

Lines: 251 Size: 4.8 KB Format: .CSS

```css
.container {
  display: flex;
  flex-direction: column;
  gap: var(--spacing-xl, 2rem);
  width: 100%;
}

.headerCard {
  background-color: var(--surface, #ffffff);
  border: 1px solid var(--border, #e2e8f0);
  border-radius: var(--radius-xl, 1rem);
  padding: 1.75rem 2rem;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 1.5rem;
  flex-wrap: wrap;
}

.headerContent {
  display: flex;
  flex-direction: column;
  gap: 0.35rem;
  max-width: 42rem;
}

.headerEyebrow {
  font-size: 0.6875rem;
  font-weight: 700;
  letter-spacing: 0.08em;
  color: var(--primary-dark, #c2410c);
}

.headerTitle {
  font-size: 1.625rem;
  font-weight: 700;
  color: var(--text-primary, #0f172a);
  margin: 0;
  letter-spacing: -0.02em;
}

.headerDesc {
  font-size: 0.9375rem;
  color: var(--text-secondary, #475569);
  margin: 0;
  line-height: 1.5;
}

.newQuestionBtn {
  display: inline-flex;
  align-items: center;
  gap: 0.5rem;
  padding: 0.625rem 1.25rem;
  background-color: var(--primary, #f97316);
  color: #ffffff;
  border: none;
  border-radius: var(--radius-md, 0.5rem);
  font-size: 0.875rem;
  font-weight: 600;
  cursor: pointer;
  transition: background-color 0.15s ease;
```

```
  white-space: nowrap;
}

.newQuestionBtn:hover {
  background-color: var(--primary-hover, #ea580c);
}

.questionList {
  display: flex;
  flex-direction: column;
  gap: 0.875rem;
}

/* Loading Skeleton */
.loadingContainer {
  display: flex;
  flex-direction: column;
  gap: 0.875rem;
}

.skeletonCard {
  display: flex;
  align-items: flex-start;
  gap: 1rem;
  padding: 1.25rem 1.5rem;
  background-color: var(--surface, #ffffff);
  border: 1px solid var(--border, #e2e8f0);
  border-radius: var(--radius-lg, 0.75rem);
}

.skeletonAvatar {
  width: 2.5rem;
  height: 2.5rem;
  border-radius: var(--radius-full, 9999px);
  background-color: var(--surface-container-high, #e2e8f0);
  animation: pulse 1.5s infinite ease-in-out;
  flex-shrink: 0;
}

.skeletonContent {
  flex: 1;
  display: flex;
  flex-direction: column;
  gap: 0.75rem;
}

.skeletonTitle {
  height: 1.25rem;
  width: 50%;
  background-color: var(--surface-container-high, #e2e8f0);
  border-radius: var(--radius-sm, 0.25rem);
  animation: pulse 1.5s infinite ease-in-out;
}

.skeletonText {
  height: 1rem;
  width: 80%;
  background-color: var(--surface-container-high, #e2e8f0);
  border-radius: var(--radius-sm, 0.25rem);
  animation: pulse 1.5s infinite ease-in-out;
}

.skeletonFooter {
  height: 0.875rem;
  width: 25%;
  background-color: var(--surface-container-high, #e2e8f0);
  border-radius: var(--radius-sm, 0.25rem);
  animation: pulse 1.5s infinite ease-in-out;
}

```

```css
/* Error Banner */
.errorBanner {
  display: flex;
  align-items: flex-start;
  gap: 0.875rem;
  padding: 1.25rem 1.5rem;
  background-color: var(--error-light, #fee2e2);
  border: 1px solid rgba(239, 68, 68, 0.3);
  border-radius: var(--radius-lg, 0.75rem);
  color: var(--error, #ef4444);
}

.errorIcon {
  flex-shrink: 0;
  margin-top: 0.125rem;
}

.errorContent {
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
}

.errorMessage {
  margin: 0;
  font-size: 0.9375rem;
  font-weight: 500;
  color: #991b1b;
}

.retryBtn {
  display: inline-flex;
  align-items: center;
  gap: 0.35rem;
  font-size: 0.8125rem;
  font-weight: 600;
  padding: 0.35rem 0.75rem;
  background-color: #ffffff;
  border: 1px solid #f87171;
  border-radius: var(--radius-sm, 0.25rem);
  color: #b91c1c;
  cursor: pointer;
  align-self: flex-start;
}

/* Empty State */
.emptyCard {
  padding: 3.5rem 2rem;
  text-align: center;
  background-color: var(--surface, #ffffff);
  border: 1px dashed var(--border, #e2e8f0);
  border-radius: var(--radius-xl, 1rem);
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  gap: 0.75rem;
  max-width: 36rem;
  margin: 1.5rem auto;
}

.emptyIconWrap {
  width: 3.5rem;
  height: 3.5rem;
  border-radius: var(--radius-full, 9999px);
  background-color: var(--surface-container-low, #f1f5f9);
  display: flex;
  align-items: center;
  justify-content: center;
  color: var(--text-tertiary, #64748b);
```

```
  margin-bottom: 0.5rem;
}

.emptyTitle {
  font-size: 1.125rem;
  font-weight: 600;
  color: var(--text-primary, #0f172a);
  margin: 0;
}

.emptyText {
  font-size: 0.9375rem;
  color: var(--text-secondary, #475569);
  line-height: 1.5;
  margin: 0;
  max-width: 28rem;
}

.emptyCtaBtn {
  display: inline-flex;
  align-items: center;
  gap: 0.5rem;
  padding: 0.625rem 1.25rem;
  background-color: var(--primary, #f97316);
  color: #ffffff;
  border: none;
  border-radius: var(--radius-md, 0.5rem);
  font-size: 0.875rem;
  font-weight: 600;
  cursor: pointer;
  transition: background-color 0.15s ease;
  margin-top: 0.5rem;
}

.emptyCtaBtn:hover {
  background-color: var(--primary-hover, #ea580c);
}

@keyframes pulse {
  0% {
    opacity: 0.6;
  }
  50% {
    opacity: 1;
  }
  100% {
    opacity: 0.6;
  }
}

```

PAGE · RAG DOCUMENTS frontend/src/pages/RagDocuments/RagDocuments.jsx

RAG Documents & PDF AI Workspace

Document manager: drag-and-drop PDF upload, embedded PDF viewer, and interactive AI Q&A panel.

Lines: 618 Size: 21.7 KB Format: .JSX

```jsx
import { useState, useEffect, useRef, useCallback } from 'react';
import {
  Upload,
  FileText,
  Trash2,
  Sparkles,
  Search,
  AlertCircle,
  CheckCircle2,
  Loader2,
  File,
  Layers,
} from 'lucide-react';
import { ragService } from '../../services/rag/rag.service';
import RagAnswerBody from '../../components/RagAnswerBody/RagAnswerBody';
import styles from './RagDocuments.module.css';

export default function RagDocuments() {
  const fileInputRef = useRef(null);

  // Document list state
  const [documents, setDocuments] = useState([]);
  const [loadingDocs, setLoadingDocs] = useState(true);
  const [docsError, setDocsError] = useState(null);

  // Active document selection
  const [activeDoc, setActiveDoc] = useState(null);
  const [pdfUrl, setPdfUrl] = useState(null);
  const [pdfLoading, setPdfLoading] = useState(false);

  // Upload state
  const [selectedFile, setSelectedFile] = useState(null);
  const [uploading, setUploading] = useState(false);
  const [uploadError, setUploadError] = useState(null);
  const [uploadSuccess, setUploadSuccess] = useState(null);

  // Semantic search state
  const [searchQuery, setSearchQuery] = useState('');
  const [searching, setSearching] = useState(false);
  const [searchResults, setSearchResults] = useState(null);
  const [searchError, setSearchError] = useState(null);

  // Ask AI state
  const [aiQuestion, setAiQuestion] = useState('');
  const [askingAi, setAskingAi] = useState(false);
  const [aiAnswer, setAiAnswer] = useState(null);
  const [aiError, setAiError] = useState(null);

  // Fetch documents list on mount
  const fetchDocuments = useCallback(async () => {
    setLoadingDocs(true);
    setDocsError(null);
    try {
      const list = await ragService.listDocuments();
      setDocuments(list);
      // Auto-select first document if none active
      if (list.length > 0 && !activeDoc) {
        setActiveDoc(list[0]);
      }
    } catch (err) {
      console.error('Failed to list RAG documents:', err);
```

```
      setDocsError(err.message || 'Unable to load documents from knowledge base.');
    } finally {
      setLoadingDocs(false);
    }
  }, [activeDoc]);

  useEffect(() => {
    fetchDocuments();
  }, [fetchDocuments]);

  // Load PDF Object URL when active document changes
  useEffect(() => {
    let currentUrl = null;
    if (activeDoc) {
      setPdfLoading(true);
      setSearchResults(null);
      setAiAnswer(null);
      setSearchError(null);
      setAiError(null);

      ragService
        .fetchPdfObjectUrl(activeDoc.id || activeDoc.documentId)
        .then(url => {
          currentUrl = url;
          setPdfUrl(url);
        })
        .catch(err => {
          console.warn('Could not fetch PDF object URL:', err);
          setPdfUrl(null);
        })
        .finally(() => {
          setPdfLoading(false);
        });
    } else {
      setPdfUrl(null);
    }

    return () => {
      if (currentUrl) {
        URL.revokeObjectURL(currentUrl);
      }
    };
  }, [activeDoc]);

  // File selection
  const handleFileChange = e => {
    const file = e.target.files?.[0];
    setUploadError(null);
    setUploadSuccess(null);
    if (!file) {
      setSelectedFile(null);
      return;
    }
    if (!file.name.toLowerCase().endsWith('.pdf')) {
      setUploadError('Only PDF documents are supported.');
      setSelectedFile(null);
      return;
    }
    setSelectedFile(file);
  };

  // Upload file
  const handleUpload = async () => {
    if (!selectedFile) return;
    setUploading(true);
    setUploadError(null);
    setUploadSuccess(null);

    try {
      // Upload returns immediately with status: 'processing'
```

```
      const newDoc = await ragService.uploadPdf(selectedFile);
      setUploadSuccess(`"${selectedFile.name}" uploaded! Processing in background…`);
      setSelectedFile(null);
      if (fileInputRef.current) {
        fileInputRef.current.value = '';
      }
      // Add doc to list immediately so user sees it
      setDocuments(prev => [newDoc, ...prev]);
      setActiveDoc(newDoc);
      setUploading(false);

      // Poll every 3s until status changes to 'ready' or 'failed'
      ragService.pollDocumentStatus(newDoc.id, updatedDoc => {
        setDocuments(prev =>
          prev.map(d => (d.id === updatedDoc.id ? updatedDoc : d)),
        );
        if (updatedDoc.id === newDoc.id) {
          setActiveDoc(updatedDoc);
        }
      }).then(finalDoc => {
        setDocuments(prev =>
          prev.map(d => (d.id === finalDoc.id ? finalDoc : d)),
        );
        setActiveDoc(finalDoc);
        if (finalDoc.status === 'ready') {
          setUploadSuccess(`"${finalDoc.title}" is ready!`);
        } else {
          setUploadError(`Processing failed: ${finalDoc.error_message || 'Unknown error'}`);
        }
      }).catch(err => {
        setUploadError(err.message);
      });
    } catch (err) {
      console.error('Upload error:', err);
      setUploadError(err.message || 'Failed to upload document.');
      setUploading(false);
    }
  };

  // Delete document
  const handleDeleteDoc = async (e, docId) => {
    e.stopPropagation();
    if (!window.confirm('Are you sure you want to remove this PDF from your workspace?')) {
      return;
    }

    try {
      await ragService.deleteDocument(docId);
      const remaining = documents.filter(d => (d.id || d.documentId) !== docId);
      setDocuments(remaining);
      if ((activeDoc?.id || activeDoc?.documentId) === docId) {
        setActiveDoc(remaining[0] || null);
      }
    } catch (err) {
      alert(err.message || 'Failed to delete document.');
    }
  };

  // Perform Semantic Search in Document
  const handleSearchSubmit = async e => {
    e.preventDefault();
    if (!searchQuery.trim() || !activeDoc) return;
    setSearching(true);
    setSearchError(null);

    try {
      const results = await ragService.searchInDocument(
        activeDoc.id || activeDoc.documentId,
        searchQuery.trim(),
      );
```

```
      setSearchResults(results);
    } catch (err) {
      console.error('Search in document error:', err);
      setSearchError(err.message || 'Semantic search encountered an error.');
    } finally {
      setSearching(false);
    }
  };

  // Ask AI about Document
  const handleAskSubmit = async e => {
    e.preventDefault();
    if (!aiQuestion.trim() || !activeDoc) return;
    setAskingAi(true);
    setAiError(null);

    try {
      const response = await ragService.queryDocument(
        activeDoc.id || activeDoc.documentId,
        aiQuestion.trim(),
      );
      setAiAnswer(response.answer);
    } catch (err) {
      console.error('Ask AI error:', err);
      setAiError(err.message || 'Could not get an answer.');
    } finally {
      setAskingAi(false);
    }
  };

  return (
    <div className={styles.container}>
      {/* Top Header Card */}
      <section className={styles.headerCard} aria-labelledby='rag-header-title'>
        <span className={styles.headerEyebrow}>KNOWLEDGE BASE</span>
        <h1 id='rag-header-title' className={styles.headerTitle}>
          Private PDF library
        </h1>
        <p className={styles.headerDesc}>
          Upload study or reference PDFs to your own workspace. Each file is indexed for
          semantic search and optional AI answers that cite passages from that document
          only. File size limits apply on the server; other users never see your uploads.
        </p>
      </section>

      {/* Two Column Layout */}
      <div className={styles.layoutColumns}>
        {/* Left Column: Library */}
        <aside className={styles.libraryColumn} aria-label='Document Library'>
          <div className={styles.libraryCard}>
            <div className={styles.libraryHeader}>
              <h2 className={styles.libraryTitle}>Library</h2>
              <p className={styles.librarySubtitle}>
                Add PDFs here. Processing runs once per upload.
              </p>
            </div>

            {/* Upload Dropzone Box */}
            <div className={styles.uploadBox}>
              <p className={styles.uploadPolicy}>
                Accepted format: PDF. Maximum file size is enforced by the server.
              </p>

              <input
                type='file'
                ref={fileInputRef}
                accept='.pdf,application/pdf'
                onChange={handleFileChange}
                className={styles.hiddenFileInput}
                id='pdf-upload-input'
```

```
              />

              <div className={styles.uploadActions}>
                <button
                  type='button'
                  onClick={() => fileInputRef.current?.click()}
                  className={styles.chooseFileBtn}
                  disabled={uploading}
                >
                  <FileText size={16} />
                  Choose file
                </button>

                <button
                  type='button'
                  onClick={handleUpload}
                  disabled={!selectedFile || uploading}
                  className={styles.uploadBtn}
                >
                  {uploading ? (
                    <>
                      <Loader2 size={16} className={styles.spinner} />
                      Uploading...
                    </>
                  ) : (
                    <>
                      <Upload size={16} />
                      Upload
                    </>
                  )}
                </button>
              </div>

              <span className={styles.selectedFileText}>
                {selectedFile ? selectedFile.name : 'No file selected.'}
              </span>

              {uploadError && (
                <div className={styles.uploadErrorText}>
                  <AlertCircle size={14} />
                  <span>{uploadError}</span>
                </div>
              )}

              {uploadSuccess && (
                <div className={styles.uploadSuccessText}>
                  <CheckCircle2 size={14} />
                  <span>{uploadSuccess}</span>
                </div>
              )}
            </div>

            {/* Document List */}
            <div className={styles.docListSection}>
              {loadingDocs && (
                <div className={styles.docLoading}>
                  <Loader2 size={20} className={styles.spinner} />
                  <span>Loading library...</span>
                </div>
              )}

              {docsError && (
                <div className={styles.docListError}>
                  <AlertCircle size={16} />
                  <span>{docsError}</span>
                </div>
              )}

              {!loadingDocs && documents.length === 0 && (
                <div className={styles.emptyDocList}>
```

```
                <File size={28} className={styles.emptyDocIcon} />
                <span>No documents uploaded yet.</span>
              </div>
            )}

            {!loadingDocs && documents.length > 0 && (
              <div className={styles.docList}>
                {documents.map(doc => {
                  const docId = doc.id || doc.documentId;
                  const isSelected =
                    (activeDoc?.id || activeDoc?.documentId) === docId;
                  const status = (doc.status || 'ready').toUpperCase();

                  return (
                    <div
                      key={docId}
                      onClick={() => setActiveDoc(doc)}
                      className={`${styles.docItem} ${
                        isSelected ? styles['docItem--active'] : ''
                      }`}
                      role='button'
                      tabIndex={0}
                      onKeyDown={e => {
                        if (e.key === 'Enter' || e.key === ' ') {
                          setActiveDoc(doc);
                        }
                      }}
                    >
                      <div className={styles.docItemInfo}>
                        <span className={styles.docItemTitle}>
                          {doc.title || doc.filename || 'Untitled document'}
                        </span>
                        <span
                          className={`${styles.statusBadge} ${
                            status === 'READY'
                              ? styles['statusBadge--ready']
                              : styles['statusBadge--processing']
                          }`}
                        >
                          {status}
                        </span>
                      </div>

                      <button
                        type='button'
                        onClick={e => handleDeleteDoc(e, docId)}
                        className={styles.deleteDocBtn}
                        title='Delete document'
                        aria-label={`Delete ${doc.title || doc.filename}`}
                      >
                        <Trash2 size={16} />
                      </button>
                    </div>
                  );
                })}
              </div>
            )}
          </div>
        </aside>

        {/* Right Column: Active Document View */}
        <main className={styles.activeColumn} aria-label='Active Document Content'>
          {!activeDoc ? (
            <div className={styles.noSelectionCard}>
              <Layers size={36} className={styles.noSelectionIcon} />
              <p className={styles.noSelectionText}>
                Choose a document from the library to open the reader, run semantic
                search over its text, and ask questions with AI-assisted answers grounded
                in that file.
```

```
              </p>
            </div>
          ) : (
            <div className={styles.activeContentWrap}>
              {/* Section 1: Reader (PDF Preview) */}
              <section className={styles.sectionCard} aria-labelledby='reader-heading'>
                <div className={styles.sectionHeader}>
                  <h2 id='reader-heading' className={styles.sectionTitle}>
                    Reader
                  </h2>
                  <p className={styles.sectionDesc}>
                    Inline preview of the selected PDF.
                  </p>
                </div>

                <div className={styles.previewContainer}>
                  {pdfLoading && (
                    <div className={styles.previewLoading}>
                      <Loader2 size={24} className={styles.spinner} />
                      <span>Loading PDF preview...</span>
                    </div>
                  )}

                  {!pdfLoading && pdfUrl ? (
                    <iframe
                      src={pdfUrl}
                      title={activeDoc.title || 'PDF Document'}
                      className={styles.pdfIframe}
                    />
                  ) : !pdfLoading ? (
                    <div className={styles.previewPlaceholder}>
                      <FileText size={40} className={styles.placeholderIcon} />
                      <strong className={styles.placeholderTitle}>
                        {activeDoc.title || activeDoc.filename}
                      </strong>
                      <p className={styles.placeholderSub}>
                        PDF preview is ready. If inline display is blocked by your browser,
                        you can still run semantic search and ask questions below.
                      </p>
                    </div>
                  ) : null}
                </div>
              </section>

              {/* Section 2: Semantic Search */}
              <section className={styles.sectionCard} aria-labelledby='search-heading'>
                <div className={styles.sectionHeader}>
                  <h2 id='search-heading' className={styles.sectionTitle}>
                    Semantic search
                  </h2>
                  <p className={styles.sectionDesc}>
                    Finds passages by meaning (embeddings), not only exact keywords.
                  </p>
                </div>

                <form onSubmit={handleSearchSubmit} className={styles.searchForm}>
                  <label htmlFor='rag-search-input' className={styles.inputLabel}>
                    Search query
                  </label>
                  <div className={styles.searchInputRow}>
                    <input
                      id='rag-search-input'
                      type='text'
                      value={searchQuery}
                      onChange={e => setSearchQuery(e.target.value)}
                      placeholder='Describe the topic or phrase you are looking for'
                      className={styles.textInput}
                    />
                    <button
                      type='submit'
```

```
                        disabled={searching || !searchQuery.trim()}
                        className={styles.primaryActionBtn}
                      >
                        {searching ? (
                          <>
                            <Loader2 size={16} className={styles.spinner} />
                            Searching...
                          </>
                        ) : (
                          <>
                            <Sparkles size={16} />
                            Search
                          </>
                        )}
                      </button>
                    </div>
                  </form>

                  {searchError && (
                    <div className={styles.errorAlert} role='alert'>
                      <AlertCircle size={16} />
                      <span>{searchError}</span>
                    </div>
                  )}

                  {/* Search Results */}
                  {searchResults && (
                    <div className={styles.searchResultsList}>
                      <h3 className={styles.resultsHeading}>
                        Relevant Excerpts ({searchResults.length})
                      </h3>
                      {searchResults.length === 0 ? (
                        <p className={styles.noResultsText}>
                          No matching passages found in this document.
                        </p>
                      ) : (
                        searchResults.map((result, idx) => (
                          <div key={idx} className={styles.excerptCard}>
                            <div className={styles.excerptHeader}>
                              {result.chunkIndex != null && (
                                <span className={styles.pageBadge}>
                                  Chunk {result.chunkIndex + 1}
                                </span>
                              )}
                              {(result.score ?? result.similarity) != null && (
                                <span className={styles.similarityBadge}>
                                  {Math.round((result.score ?? result.similarity) * 100)}% match
                                </span>
                              )}
                            </div>
                            <p className={styles.excerptContent}>
                              {result.excerpt || result.content || result.text}
                            </p>
                          </div>
                        ))
                      )}
                    </div>
                  )}
                </section>

                {/* Section 3: Ask with AI */}
                <section className={styles.sectionCard} aria-labelledby='ask-ai-heading'>
                  <div className={styles.sectionHeader}>
                    <h2 id='ask-ai-heading' className={styles.sectionTitle}>
                      Ask with AI
                    </h2>
                    <p className={styles.sectionDesc}>
                      Answers use only retrieved excerpts from this PDF, with citations
                      where possible. When the document includes code, the reply may show
                      it in formatted blocks you can copy.
```

```
                  </p>
                </div>

                <form onSubmit={handleAskSubmit} className={styles.askForm}>
                  <label htmlFor='rag-ai-input' className={styles.inputLabel}>
                    Question
                  </label>
                  <textarea
                    id='rag-ai-input'
                    rows={4}
                    value={aiQuestion}
                    onChange={e => setAiQuestion(e.target.value)}
                    placeholder='What does this PDF recommend for route persistence?'
                    className={styles.textareaInput}
                  />

                  <div className={styles.askActionRow}>
                    <button
                      type='submit'
                      disabled={askingAi || !aiQuestion.trim()}
                      className={styles.primaryActionBtn}
                    >
                      {askingAi ? (
                        <>
                          <Loader2 size={16} className={styles.spinner} />
                          Generating answer...
                        </>
                      ) : (
                        <>
                          <Sparkles size={16} />
                          Ask
                        </>
                      )}
                    </button>
                  </div>
                </form>

                {aiError && (
                  <div className={styles.errorAlert} role='alert'>
                    <AlertCircle size={16} />
                    <span>{aiError}</span>
                  </div>
                )}

                {/* AI Answer Display */}
                {aiAnswer && (
                  <div className={styles.aiAnswerCard}>
                    <div className={styles.aiAnswerHeader}>
                      <Sparkles size={18} className={styles.aiAnswerSparkle} />
                      <strong className={styles.aiAnswerTitle}>
                        Grounded AI Response:
                      </strong>
                    </div>
                    <div className={styles.aiAnswerBody}>
                      <RagAnswerBody>{aiAnswer}</RagAnswerBody>
                    </div>
                  </div>
                )}
              </section>
            </div>
          )}
        </main>
      </div>
    </div>
  );
}
```

PAGE · CSS frontend/src/pages/RagDocuments/RagDocuments.module.css

RAG Workspace Styles

Dual-pane layout (PDF preview + chat conversation panel) and citation markers.

Lines: 629 Size: 11.4 KB Format: .CSS

```css
.container {
  display: flex;
  flex-direction: column;
  gap: var(--spacing-xl, 2rem);
  width: 100%;
}

/* Header Card */
.headerCard {
  background-color: var(--surface, #ffffff);
  border: 1px solid var(--border, #e2e8f0);
  border-radius: var(--radius-xl, 1rem);
  padding: 1.75rem 2rem;
  display: flex;
  flex-direction: column;
  gap: 0.35rem;
}

.headerEyebrow {
  font-size: 0.6875rem;
  font-weight: 700;
  letter-spacing: 0.08em;
  color: var(--primary-dark, #c2410c);
}

.headerTitle {
  font-size: 1.625rem;
  font-weight: 700;
  color: var(--text-primary, #0f172a);
  margin: 0;
  letter-spacing: -0.02em;
}

.headerDesc {
  font-size: 0.9375rem;
  color: var(--text-secondary, #475569);
  margin: 0;
  line-height: 1.5;
  max-width: 50rem;
}

/* Two-column layout */
.layoutColumns {
  display: grid;
  grid-template-columns: 22rem 1fr;
  gap: 1.75rem;
  align-items: flex-start;
}

@media (max-width: 1024px) {
  .layoutColumns {
    grid-template-columns: 1fr;
  }
}

/* Library Column */
.libraryColumn {
  display: flex;
  flex-direction: column;
}

```

```
.libraryCard {
  background-color: var(--surface, #ffffff);
  border: 1px solid var(--border, #e2e8f0);
  border-radius: var(--radius-xl, 1rem);
  padding: 1.5rem;
  display: flex;
  flex-direction: column;
  gap: 1.25rem;
}

.libraryHeader {
  display: flex;
  flex-direction: column;
  gap: 0.25rem;
}

.libraryTitle {
  font-size: 1.125rem;
  font-weight: 700;
  color: var(--text-primary, #0f172a);
  margin: 0;
}

.librarySubtitle {
  font-size: 0.8125rem;
  color: var(--text-tertiary, #64748b);
  margin: 0;
}

/* Upload Box */
.uploadBox {
  background-color: var(--surface-container-low, #f8fafc);
  border: 1px dashed var(--border, #cbd5e1);
  border-radius: var(--radius-lg, 0.75rem);
  padding: 1.25rem;
  display: flex;
  flex-direction: column;
  gap: 0.75rem;
}

.uploadPolicy {
  font-size: 0.75rem;
  color: var(--text-tertiary, #64748b);
  margin: 0;
  line-height: 1.4;
}

.hiddenFileInput {
  display: none;
}

.uploadActions {
  display: flex;
  align-items: center;
  gap: 0.5rem;
}

.chooseFileBtn {
  display: inline-flex;
  align-items: center;
  gap: 0.35rem;
  padding: 0.45rem 0.875rem;
  background-color: #ffffff;
  border: 1px solid var(--border, #cbd5e1);
  border-radius: var(--radius-md, 0.5rem);
  color: var(--text-primary, #0f172a);
  font-size: 0.8125rem;
  font-weight: 500;
  cursor: pointer;
  transition: all 0.15s ease;
```

```
}

.chooseFileBtn:hover:not(:disabled) {
  background-color: var(--surface-hover, #f1f5f9);
  border-color: var(--border-hover, #94a3b8);
}

.uploadBtn {
  display: inline-flex;
  align-items: center;
  gap: 0.35rem;
  padding: 0.45rem 0.875rem;
  background-color: var(--primary, #f97316);
  color: #ffffff;
  border: none;
  border-radius: var(--radius-md, 0.5rem);
  font-size: 0.8125rem;
  font-weight: 600;
  cursor: pointer;
  transition: background-color 0.15s ease;
}

.uploadBtn:hover:not(:disabled) {
  background-color: var(--primary-hover, #ea580c);
}

.uploadBtn:disabled {
  opacity: 0.5;
  cursor: not-allowed;
}

.selectedFileText {
  font-size: 0.75rem;
  color: var(--text-tertiary, #64748b);
  font-style: italic;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.uploadErrorText {
  display: flex;
  align-items: center;
  gap: 0.35rem;
  font-size: 0.75rem;
  color: var(--error, #ef4444);
}

.uploadSuccessText {
  display: flex;
  align-items: center;
  gap: 0.35rem;
  font-size: 0.75rem;
  color: var(--success, #10b981);
}

/* Document List */
.docListSection {
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
}

.docLoading {
  display: flex;
  align-items: center;
  gap: 0.5rem;
  font-size: 0.8125rem;
  color: var(--text-tertiary, #64748b);
  padding: 1rem 0;
```

```
  justify-content: center;
}

.docListError {
  display: flex;
  align-items: center;
  gap: 0.35rem;
  font-size: 0.8125rem;
  color: var(--error, #ef4444);
}

.emptyDocList {
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 0.5rem;
  padding: 2rem 1rem;
  text-align: center;
  font-size: 0.8125rem;
  color: var(--text-tertiary, #64748b);
}

.emptyDocIcon {
  color: var(--text-placeholder, #cbd5e1);
}

.docList {
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
}

.docItem {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 0.75rem;
  padding: 0.75rem 1rem;
  background-color: var(--surface-container-low, #f8fafc);
  border: 1px solid var(--border, #e2e8f0);
  border-radius: var(--radius-md, 0.5rem);
  cursor: pointer;
  transition: all 0.15s ease;
}

.docItem:hover {
  background-color: #ffffff;
  border-color: var(--border-hover, #cbd5e1);
}

.docItem--active {
  border-color: var(--primary, #f97316);
  background-color: #ffffff;
  box-shadow: 0 0 0 1px var(--primary, #f97316);
}

.docItemInfo {
  display: flex;
  flex-direction: column;
  gap: 0.25rem;
  min-width: 0;
}

.docItemTitle {
  font-size: 0.875rem;
  font-weight: 600;
  color: var(--text-primary, #0f172a);
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
```

```
}

.statusBadge {
  font-size: 0.625rem;
  font-weight: 700;
  padding: 0.1rem 0.4rem;
  border-radius: var(--radius-sm, 0.25rem);
  align-self: flex-start;
  letter-spacing: 0.05em;
}

.statusBadge--ready {
  background-color: #dcfce7;
  color: #15803d;
}

.statusBadge--processing {
  background-color: #fef3c7;
  color: #b45309;
}

.deleteDocBtn {
  background: none;
  border: none;
  color: var(--text-tertiary, #94a3b8);
  cursor: pointer;
  padding: 0.35rem;
  border-radius: var(--radius-sm, 0.25rem);
  display: flex;
  align-items: center;
  justify-content: center;
  transition: color 0.15s ease;
}

.deleteDocBtn:hover {
  color: var(--error, #ef4444);
}

/* Right Column */
.activeColumn {
  min-width: 0;
}

.noSelectionCard {
  border: 1px dashed var(--border, #cbd5e1);
  background-color: var(--surface-container-low, #f8fafc);
  border-radius: var(--radius-xl, 1rem);
  padding: 4rem 2.5rem;
  text-align: center;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  gap: 1rem;
}

.noSelectionIcon {
  color: var(--text-tertiary, #94a3b8);
}

.noSelectionText {
  font-size: 0.9375rem;
  color: var(--text-secondary, #475569);
  max-width: 28rem;
  margin: 0;
  line-height: 1.6;
}

.activeContentWrap {
  display: flex;
```

```
  flex-direction: column;
  gap: 1.75rem;
}

/* Section Card */
.sectionCard {
  background-color: var(--surface, #ffffff);
  border: 1px solid var(--border, #e2e8f0);
  border-radius: var(--radius-xl, 1rem);
  padding: 1.75rem 2rem;
  display: flex;
  flex-direction: column;
  gap: 1.25rem;
}

.sectionHeader {
  display: flex;
  flex-direction: column;
  gap: 0.25rem;
}

.sectionTitle {
  font-size: 1.125rem;
  font-weight: 700;
  color: var(--text-primary, #0f172a);
  margin: 0;
}

.sectionDesc {
  font-size: 0.8125rem;
  color: var(--text-tertiary, #64748b);
  margin: 0;
}

/* PDF Preview */
.previewContainer {
  width: 100%;
  height: 24rem;
  background-color: var(--surface-container-low, #f8fafc);
  border: 1px solid var(--border, #e2e8f0);
  border-radius: var(--radius-md, 0.5rem);
  overflow: hidden;
  display: flex;
  align-items: center;
  justify-content: center;
}

.pdfIframe {
  width: 100%;
  height: 100%;
  border: none;
}

.previewLoading {
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 0.5rem;
  color: var(--text-tertiary, #64748b);
  font-size: 0.875rem;
}

.previewPlaceholder {
  display: flex;
  flex-direction: column;
  align-items: center;
  text-align: center;
  gap: 0.5rem;
  padding: 2rem;
  max-width: 24rem;
```

```
}

.placeholderIcon {
  color: var(--primary, #f97316);
}

.placeholderTitle {
  font-size: 0.9375rem;
  color: var(--text-primary, #0f172a);
}

.placeholderSub {
  font-size: 0.8125rem;
  color: var(--text-tertiary, #64748b);
  margin: 0;
  line-height: 1.4;
}

/* Search Form */
.searchForm,
.askForm {
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
}

.inputLabel {
  font-size: 0.8125rem;
  font-weight: 600;
  color: var(--text-secondary, #334155);
}

.searchInputRow {
  display: flex;
  gap: 0.75rem;
}

.textInput {
  flex: 1;
  padding: 0.625rem 1rem;
  font-size: 0.875rem;
  border: 1px solid var(--border, #cbd5e1);
  border-radius: var(--radius-md, 0.5rem);
  background-color: #ffffff;
  outline: none;
  transition: border-color 0.15s ease;
  font-family: inherit;
}

.textInput:focus,
.textareaInput:focus {
  border-color: var(--primary, #f97316);
  box-shadow: 0 0 0 3px rgba(249, 115, 22, 0.15);
}

.textareaInput {
  width: 100%;
  padding: 0.75rem 1rem;
  font-size: 0.875rem;
  border: 1px solid var(--border, #cbd5e1);
  border-radius: var(--radius-md, 0.5rem);
  background-color: #ffffff;
  outline: none;
  font-family: inherit;
  line-height: 1.5;
  resize: vertical;
}

.primaryActionBtn {
  display: inline-flex;
```

```
  align-items: center;
  gap: 0.4rem;
  padding: 0.625rem 1.25rem;
  background-color: var(--primary, #f97316);
  color: #ffffff;
  border: none;
  border-radius: var(--radius-md, 0.5rem);
  font-size: 0.875rem;
  font-weight: 600;
  cursor: pointer;
  transition: background-color 0.15s ease;
  white-space: nowrap;
}

.primaryActionBtn:hover:not(:disabled) {
  background-color: var(--primary-hover, #ea580c);
}

.primaryActionBtn:disabled {
  opacity: 0.6;
  cursor: not-allowed;
}

.askActionRow {
  display: flex;
  justify-content: flex-start;
  margin-top: 0.25rem;
}

.errorAlert {
  display: flex;
  align-items: center;
  gap: 0.5rem;
  padding: 0.75rem 1rem;
  background-color: var(--error-light, #fee2e2);
  border: 1px solid rgba(239, 68, 68, 0.3);
  border-radius: var(--radius-md, 0.5rem);
  color: var(--error, #ef4444);
  font-size: 0.8125rem;
}

/* Results */
.searchResultsList {
  display: flex;
  flex-direction: column;
  gap: 0.75rem;
  margin-top: 0.5rem;
}

.resultsHeading {
  font-size: 0.9375rem;
  font-weight: 600;
  color: var(--text-primary, #0f172a);
  margin: 0;
}

.noResultsText {
  font-size: 0.8125rem;
  color: var(--text-tertiary, #64748b);
  margin: 0;
}

.excerptCard {
  background-color: var(--surface-container-low, #f8fafc);
  border: 1px solid var(--border, #e2e8f0);
  border-radius: var(--radius-md, 0.5rem);
  padding: 0.875rem 1rem;
  display: flex;
  flex-direction: column;
  gap: 0.35rem;
```

```
}

.excerptHeader {
  display: flex;
  align-items: center;
  gap: 0.5rem;
}

.pageBadge {
  font-size: 0.6875rem;
  font-weight: 600;
  padding: 0.1rem 0.4rem;
  background-color: #ffffff;
  border: 1px solid var(--border, #cbd5e1);
  border-radius: var(--radius-sm, 0.25rem);
  color: var(--text-secondary, #475569);
}

.similarityBadge {
  font-size: 0.6875rem;
  font-weight: 600;
  padding: 0.1rem 0.4rem;
  background-color: #dbeafe;
  color: #1e40af;
  border-radius: var(--radius-sm, 0.25rem);
}

.excerptContent {
  font-size: 0.8125rem;
  color: var(--text-secondary, #334155);
  line-height: 1.5;
  margin: 0;
}

/* AI Answer Card */
.aiAnswerCard {
  background-color: var(--surface-container-low, #f8fafc);
  border: 1px solid var(--border, #cbd5e1);
  border-radius: var(--radius-lg, 0.75rem);
  padding: 1.25rem 1.5rem;
  display: flex;
  flex-direction: column;
  gap: 0.75rem;
  margin-top: 0.5rem;
}

.aiAnswerHeader {
  display: flex;
  align-items: center;
  gap: 0.5rem;
}

.aiAnswerSparkle {
  color: var(--primary, #f97316);
}

.aiAnswerTitle {
  font-size: 0.9375rem;
  color: var(--text-primary, #0f172a);
}

.aiAnswerBody {
  color: var(--text-secondary, #334155);
}

.spinner {
  animation: spin 1s linear infinite;
}

@keyframes spin {
```

```
  from {
    transform: rotate(0deg);
  }
  to {
    transform: rotate(360deg);
  }
}

```